Universidad Internacional de Valencia

Inteligencia Artificial multimodal para la recomendación de outfits y análisis de compatibilidad en moda

Titulación:
Máster en Inteligencia Artificial

Curso académico
2025 – 2026

Alumno/a: Ángel Fernández García
D.N.I: 79125286L

Director/a de TFM: Mónica Díaz Hernández

Convocatoria:

Primera

20 Julio 2026

De:
Planeta Formación y Universidades

Índice general

Índice de figuras IV

Índice de tablas VIII

Índice de código IX

Resumen 1

Abstract 2

1. Introducción 3

2. Descripción del problema 5
   2.1. Modelado del Estilo y la Armonía entre prendas 5
   2.2. Recuperación de Outfits basada en Texto (Text-to-Outfit) 5

3. Objetivos 7
   3.1. Objetivo Principal 7
   3.2. Objetivos Específicos 7
   3.3. Alcance 8
   3.4. Límites 8

4. Marco Teórico 10
   4.1. El problema de la compatibilidad de moda 10
      4.1.1. Delimitación conceptual: Similitud vs. Compatibilidad 10
      4.1.2. No transitividad en la combinación de prendas 11
   4.2. Representación matemática de ítems de moda 11
      4.2.1. Modelos de extracción visual 11
      4.2.2. Modelos de representación semántica y textual 13
      4.2.3. Fusión multimodal y representación categórica 14
      4.2.4. La etiqueta de categoría 15
   4.3. Aprendizaje métrico 16

4.3.1. Arquitecturas siamesas y funciones de pérdida 16
4.3.2. Nociones de distancia geométrica en espacios latentes 18
4.4. Tareas y métricas de evaluación estándar 18
4.5. Paradigmas de recomendación: enfoques basados en producto y enfoques personalizados 20
4.5.1. Fundamentos teóricos de la recomendación personalizada 20
4.6. Ecosistema de Datos, Viabilidad Práctica y Sesgos 21

5. Estado del arte 24
5.1. Evolución cronológica de los sistemas de recomendación de moda 24
5.1.1. Representación por pares 24
5.1.2. Modelos secuenciales (Bi-LSTM) 24
5.1.3. Embeddings condicionados por tipo 25
5.1.4. Aprendizaje de condiciones sin supervisión: SCE-Net 27
5.1.5. Modelado de subgrafos de outfits 28
5.1.6. Fusión modular y atención sobre conjuntos: Recommendation Engine . 30
5.1.7. Síntesis evolutiva y comparación de arquitecturas 32
5.2. Integración de lenguaje natural en la recomendación de moda: *Text-to-Outfit* . . 34
5.2.1. Búsqueda de prenda individual guiada por texto 34
5.2.2. Búsqueda de outfits completos guiada por texto: CLIP4Outfit 35
5.2.3. Perspectiva futura: de la recuperación a la generación mediante difusión 37

6. Metodología 40
6.1. Marco General de la Investigación y Formulación de Hipótesis 40
6.2. Estrategia de Selección y Diseño de Arquitecturas 41
6.3. Protocolo de Evaluación y Métricas Estándar 42
6.4. Diseño Experimental para la Validez de Transferencia 44

7. Desarrollo del Proyecto 46
7.1. Arquitectura del Sistema y Entorno de Ejecución 46
7.1.1. Infraestructura Hardware y Estrategias de Optimización de Memoria . . 46
7.1.2. Pipeline Tecnológico y Stack de Software 47
7.2. Pipeline de Procesamiento de Datos y Preparación de Corpora 48
7.2.1. Benchmark de Referencia: Polyvore Outfits 49
7.2.2. Desafíos Técnicos, Legales y Metodológicos en la Adquisición de Corpora Comerciales 52
7.3. Implementación Módulo a Módulo de las Arquitecturas 53
7.3.1. SCE-Net: Extracción e Integración de Máscaras Latentes 54
7.3.2. HGNN: Construcción del Hipergrafo y Paso de Mensajes 59
7.3.3. Recommendation Engine: Redes de Atención Transformer Multimodal . 63
7.4. Módulo de Búsqueda y Recomendación Text-to-Outfit 67
7.4.1. Recommendation Engine: Adaptación del Espacio Latente Multimodal . 68

7.4.2. CLIP4Outfit: Alineación Multimodal y Recuperación Contrastiva 70
7.5. Proceso de Entrenamiento, Convergencia y Reproducibilidad 75
7.5.1. Configuración del Entrenamiento e Hiperparámetros 75
7.5.2. Diagnóstico de Incidencias y Decisiones de Diseño 78
7.5.3. Dinámica de Convergencia y Estabilidad del Modelo 79
7.5.4. Protocolo de Reproducibilidad y Control de Aleatoriedad 86

8. Resultados Experimentales y Discusión 88
8.1. Protocolo Experimental y Métricas de Evaluación 89
8.2. Evaluación de Compatibilidad y Complementariedad 89
8.2.1. Comparación Cuantitativa y Estado del Arte 89
8.2.2. Análisis Cualitativo y Casos de Éxito 91
8.2.3. Análisis de Errores y Casos Límite (*Edge Cases*) 94
8.2.4. Análisis del Espacio de Representación 98
8.2.5. Interpretabilidad de mecanismos internos 99
8.3. Evaluación del Módulo Text-to-Outfit (Multimodal) 101
8.3.1. Resultados de Recuperación Semántica (*Text-to-Outfit*) 101
8.3.2. Inspección Cualitativa de Consultas con Referencia Conocida 103
8.3.3. Evaluación Cualitativa de Consultas Compuestas sin Referencia 106
8.4. Decisiones de Diseño y Diagnóstico de Componentes 109
8.4.1. Ablaciones Empíricas y Diagnóstico Interno de Arquitecturas 109
8.4.2. Justificación Conceptual y Limitaciones Transversales 110
8.5. Análisis de Eficiencia Computacional y Latencia de Inferencia 111
8.5.1. Modelos de Compatibilidad y Diagnóstico de Componentes 111
8.5.2. Módulo Text-to-Outfit: Latencia y Complejidad Espacial 111

9. Conclusiones y Trabajo Futuro 114
9.1. Consecución de Objetivos 114
9.2. Síntesis de Resultados y Principales Contribuciones 116
9.3. Limitaciones del Trabajo 116
9.4. Líneas de Trabajo Futuro 117
9.5. Consideraciones Finales 118

Bibliografía 120

A. Apéndize A: Declaración de uso de la IA 121
A.1. Apoyo al desarrollo técnico 121
A.2. Apoyo en la redacción y documentación 122
A.3. Uso responsable y verificación 122

Índice de figuras

3.1. Esquema general del sistema desarrollado 9

4.1. Arquitectura ResNet-18 original. Figura adaptada de Ramzan et al. (2019) . . . 12
4.2. Arquitectura del Vision Transformer (ViT). Figura tomada de Li (2025). 13
4.3. Arquitectura siamesa original basada en redes con retardo temporal para la comparación de características. Figura tomada de Bromley et al. (1993). 17

5.1. Arquitectura de embeddings condicionados por tipo (*type-aware*). A partir de un embedding general extraído por la red base, se aplican máscaras diagonales específicas para proyectar las prendas a subespacios de compatibilidad según el par de categorías. Figura adaptada de Vasileva et al. (2018). 26
5.2. Arquitectura de SCE-Net (*Similarity Condition Embedding Network*). Figura adaptada de Tan et al. (2019). 27
5.3. Arquitectura de HGNN (*Hypergraph Neural Network*). Figura adaptada de Gulati (2024). 31
5.4. Ilustración general de la arquitectura de CLIP4Outfit. Figura adaptada de Jang et al. (2023). 36

7.1. Flujo general del *pipeline* común a las tres arquitecturas de compatibilidad implementadas en este trabajo, desde la ingesta de datos (apartado 7.2) hasta la evaluación final (capítulo 8). La etapa de proyección y modelado arquitectónico (recuadro discontinuo) es el punto de divergencia entre modelos, detallado de forma individual para cada arquitectura en los apartados 7.3.1, 7.3.2 y 7.3.3. 54
7.2. Esquema del módulo `TypeAwareGate` basado en modulación de características FiLM (*Feature-wise Linear Modulation*) sigue a Perez et al. (2017), aplicado sobre espacios condicionados por categoría en la linea de Vasileva et al. (2018). El *embedding* de categoría e_{cat} proyecta de forma independiente un factor multiplicativo de escala γ y un sesgo aditivo de desplazamiento β, condicionando el *embedding* del ítem x antes de ser procesado por el Transformer del *Recommendation Engine*. 65

7.3. Arquitectura jerárquica de tres niveles de CLIP4Outfit, inspirada en el diseño propuesto por Jang et al. (2023). Las dos ramas paralelas (visual y textual) procesan los componentes del *outfit* y la consulta, evaluando la similitud cruzada en tres niveles semánticos progresivos (S_{item}, S_{style} y S_{outfit}). Las flechas punteadas indican la agregación de las tres métricas en la función de pérdida combinada $\mathcal{L}$. 73

7.4. Evolución de las componentes de optimización de SCE-Net durante 11 épocas de entrenamiento: (a) pérdida total combinada ($\mathcal{L}_{\text{total}}$); (b) pérdida *Triplet* ($\mathcal{L}_{\text{triplet}}$); (c) término de regularización L_1 sobre las máscaras de atención por subespacio; y (d) término de regularización L_2 sobre los *embeddings*. La línea vertical punteada verde señala la época 6, identificada mediante *early stopping* como el punto de óptima generalización en el conjunto de validación. 80

7.5. Dinámica de entrenamiento y validación de la arquitectura *HGNN*: (a) curvas de pérdida por ordenación ($\mathcal{L}_{\text{BPR}}$) y (b) métrica de compatibilidad AUC. La línea vertical discontinua indica el punto de parada anticipada (*early stopping*). 82

7.6. Dinámica de entrenamiento y validación de la arquitectura *Recommendation Engine*: (a) evolución de las pérdidas de entrenamiento ($\mathcal{L}_{\text{CE}}$ y $\mathcal{L}_{\text{FITB}}$); (b) métrica de compatibilidad CE-AUC sobre las particiones de entrenamiento y validación; y (c) precisión en la tarea de completado de conjuntos (*Fill-in-the-Blank*), comparada frente al umbral aleatorio de referencia ($0{,}25$). La línea vertical discontinua señala el punto de parada anticipada (*early stopping*). . . . 83

7.7. Comparativa de la dinámica de entrenamiento y validación de la arquitectura *CLIP4Outfit* bajo la configuración de *backbone* base (*Zero-shot*) frente a la variante adaptada al dominio de moda (*Fine-tuned*): (a) evolución de la pérdida contrastiva de entrenamiento; y (b) rendimiento de recuperación en validación medido mediante Recall@5. La estrella ($\star$) señala la posición del óptimo alcanzado por cada modelo y las líneas verticales discontinuas indican el momento de parada anticipada (*early stopping*). 84

7.8. Curvas de optimización y evaluación durante el entrenamiento de CLIP4Outfit: (a) evolución de la pérdida contrastiva ($\mathcal{L}$) sobre los conjuntos de entrenamiento (línea azul continua) y validación (línea naranja discontinua); (b) métrica de recuperación *Recall@5* ($\%$) sobre el conjunto de validación. La línea punteada roja vertical indica la parada anticipada (*early stopping*) en la época 15, tras agotar la paciencia de $P = 5$ épocas respecto al máximo alcanzado en *Recall@5*. 85

8.1. Comparativa de AUC y accuracy FITB frente al estado del arte 90

8.2. Ejemplo de acierto en la tarea *Fill-in-the-Blank* (FITB) mediante el modelo HGNN. Las puntuaciones asignadas a los candidatos (s) muestran una dispersión reducida entre la opción correcta y los distractores. 92

8.3. Ejemplo de acierto en la tarea *Fill-in-the-Blank* (FITB) mediante *Recommendation Engine*. El modelo asigna una puntuación significativamente más alta al candidato correcto ($s = 0{,}723$) frente a las alternativas. 92
8.4. Ejemplo de acierto en la tarea *Fill-in-the-Blank* (FITB) mediante SCE-Net. La distancia en el espacio latente (d) permite identificar de forma correcta la opción de referencia. 93
8.5. Clasificación correcta de compatibilidad (*True Positive*) en *Recommendation Engine* para un conjunto monocromático amarillo ($score = 0{,}995$). 93
8.6. Clasificación correcta de compatibilidad (*True Positive*) en SCE-Net para el mismo conjunto monocromático amarillo ($score = 0{,}282$). 94
8.7. Clasificación correcta de compatibilidad (*True Positive*) en SCE-Net para el mismo conjunto monocromático amarillo ($score = 0{,}282$). 95
8.8. Clasificación correcta de compatibilidad (*True Positive*) en SCE-Net para el mismo conjunto monocromático amarillo ($score = 0{,}282$). 95
8.9. Ejemplo de fallo en la tarea FITB mediante *Recommendation Engine*. El modelo selecciona el distractor de falda rosa palo ($s = 0{,}318$) frente al conjunto de referencia GT ($s = 0{,}310$) debido a la alta afinidad estilística y cromática de las prendas. 96
8.10. Ejemplo de fallo en la tarea FITB mediante SCE-Net en el mismo escenario. El modelo asigna menor distancia latente al distractor ($d = 0{,}267$) que a la prenda de referencia de tul rosa ($d = 0{,}271$). 97
8.11. Evaluación de compatibilidad con acierto (*True Positive*) en el modelo HGNN. Se observa una puntuación elevada ($s = 22{,}037$) asociada a la alta cardinalidad del conjunto (9 prendas). 97
8.12. Evaluación de compatibilidad con fallo de clasificación (*False Negative*) en el modelo HGNN. La puntuación reducida ($s = 2{,}981$) se ve influenciada por la baja cardinalidad del conjunto (3 prendas). 97
8.13. Proyección t-SNE bidimensional de los espacios latentes (300 ítems) aprendidos por HGNN ($0{,}6967$), *Recommendation Engine* ($0{,}1111$) y SCE-Net ($-0{,}1373$). Cada color representa una categoría semántica principal: azul oscuro (*heels*), azul claro (*sweater*), naranja (*dress*), melocotón (*earrings*), verde oscuro (*purse*), verde claro (*clutch*), rojo (*skirt*), rosa (*tote*), púrpura (*necklace*), lila (*pump*), marrón oscuro (*sunglasses*) y marrón claro (*bracelet*); los puntos grises corresponden a categorías secundarias. 98
8.14. Distribución media de los pesos asignados a las cinco máscaras de condición en SCE-Net para ocho combinaciones de categorías de prendas: *jewellery*↔*shoes* ($n = 114$), *bags*↔*shoes* ($n = 109$), *bags*↔*jewellery* ($n = 103$), *shoes*↔*tops* ($n = 79$), *bags*↔*tops* ($n = 71$), *jewellery*↔*tops* ($n = 66$), *bottoms*↔*shoes* ($n = 65$) y *bottoms*↔*tops* ($n = 62$). Se observa un perfil de pesos idéntico ($[0{,}248;\ 0{,}172;\ 0{,}216;\ 0{,}130;\ 0{,}235]$) e independiente de las categorías de entrada. 100

8.15. Resultados cualitativos de recomendación (*Top-1*) frente a la consulta textual *«Velvet Bomber Jackets»*. Comparativa entre las adaptaciones RE *Zero-Shot*, RE *Fine-Tuned*, CLIP4Outfit y el conjunto de referencia (*Ground Truth*). 104
8.16. Resultados cualitativos de recomendación (*Top-1*) frente a la consulta textual *«Sweater dress and boots»*. Comparativa entre las adaptaciones RE *Zero-Shot*, RE *Fine-Tuned*, CLIP4Outfit y el conjunto de referencia (*Ground Truth*). . 104
8.17. Resultados cualitativos de recomendación (*Top-1*) frente a la consulta textual *«Paris fashion week»*. Comparativa entre las adaptaciones RE *Zero-Shot*, RE *Fine-Tuned*, CLIP4Outfit y el conjunto de referencia (*Ground Truth*). 105
8.18. Resultados cualitativos de recomendación (*Top-1*) frente a la consulta textual *«Sweater dresses»*. Comparativa entre las adaptaciones RE *Zero-Shot*, RE *Fine-Tuned*, CLIP4Outfit y el conjunto de referencia (*Ground Truth*). 106
8.19. Resultados cualitativos de recomendación (*Top-1*) frente a la consulta compleja sin referencia *«Casual outfit for a rainy day but not too warm»*. Comparativa entre las adaptaciones de *Recommendation Engine Zero-Shot* y RE *Fine-Tuned* y CLIP4Outfit. 107
8.20. Resultados cualitativos de recomendación (*Top-1*) frente a la consulta compleja sin referencia *«Elegant evening look without black or heels a rainy day but not too warm»*. Comparativa entre las adaptaciones de *Recommendation Engine Zero-Shot* y RE *Fine-Tuned* y CLIP4Outfit. 108
8.21. Resultados cualitativos de recomendación (*Top-1*) frente a la consulta compleja sin referencia *«Streetwear inspired by 90s hip-hop with oversized silhouette»*. Comparativa entre las adaptaciones de *Recommendation Engine Zero-Shot* y RE *Fine-Tuned* y CLIP4Outfit. 108
8.22. Volumen de parámetros y latencia de inferencia 112

Índice de tablas

4.1. Comparativa de las métricas de distancia y espacios de representación en los artículos de referencia 19
4.2. Principales datasets empleados en la literatura de compatibilidad y recomendación de moda. 23

5.1. Cifras de AUC (Compatibility Prediction) y accuracy (Fill-In-The-Blank) reportadas por cada arquitectura, agrupadas por benchmark de evaluación. Las cifras solo son directamente comparables *dentro* de cada bloque, dado que cada uno emplea un conjunto de datos o split diferente. 33

7.1. Distribución volumétrica de la partición *nondisjoint* de Polyvore Outfits. 50
7.2. Porcentaje de valores nulos o ausentes por atributo descriptivo en Polyvore Outfits. 51
7.3. Configuración final de hiperparámetros de implementación para SCE-Net. . . . 59
7.4. Configuración final de hiperparámetros de implementación para HGNN. 63
7.5. Configuración final de hiperparámetros de implementación para Recommendation Engine. 68
7.6. Configuración de hiperparámetros para la adaptación de *Recommendation Engine*. 71
7.7. Configuración de hiperparámetros para la arquitectura CLIP4Outfit. 75
7.8. Distribución de pesos asignados a las $M = 5$ máscaras de condición en la época óptima (6). 81

8.1. Evaluación comparativa en el conjunto de prueba frente a los valores reportados en la literatura. 89
8.2. Resultados de recuperación semántica sobre el conjunto de test (criterio multipositivo). 101
8.3. Volumen de parámetros y latencia de inferencia en modelos de compatibilidad. 111
8.4. Desglose de parámetros y tiempo de inferencia para las variantes de *Text-to-Outfit*. 112

Índice de código

7.1. Máscaras de condición y rama de ponderación dinámica de SCE-Net. 55
7.2. Función de pérdida total de SCE-Net, combinando la *triplet loss* principal con los cuatro términos de regularización. 58
7.3. Cálculo de la laplaciana normalizada de hipergrafo. 60
7.4. Capa de propagación sobre el hipergrafo. 61
7.5. Bloque de codificador Transformer del *Recommendation Engine*, sin *positional encoding*. 65
7.6. Cálculo de la similitud *Weighted Tokenwise Interaction* (WTI), reutilizada en los niveles Ítem y Estilo de CLIP4Outfit. 72

Resumen

Este Trabajo Fin de Máster aborda la recomendación de moda, donde la compatibilidad de un conjunto no depende de cada prenda por separado sino de su interacción en textura, color, forma y contexto de uso. El trabajo se propone desarrollar y validar un sistema capaz de generar y combinar *embeddings* de imagen y descripción textual de prendas, con capacidad de generalización a distintas tareas de recomendación de *outfits*.

Con este fin, se implementan, entrenan y comparan bajo un protocolo homogéneo tres arquitecturas del estado del arte: *SCE-Net*, *HGNN* y *Recommendation Engine*, sobre el benchmark Polyvore Outfits (partición nondisjoint), en las tareas de *Fill-In-The-Blank* y *Compatibility Prediction*. Además, se desarrolla un módulo *Text-to-Outfit* que recupera outfits completos a partir de descripciones en lenguaje natural, contrastando una adaptación ligera del espacio latente de *Recommendation Engine* frente a la arquitectura contrastiva dedicada *CLIP4Outfit*.

Recommendation Engine resulta la arquitectura más robusta y equilibrada (AUC $= 0{,}896$, consistente con el estado del arte), frente a la mayor sensibilidad de *SCE-Net* y *HGNN* a la configuración de entrenamiento y a la escala del conjunto de datos, respectivamente. En *Text-to-Outfit*, la adaptación de *Recommendation Engine* alcanza Recall@5 $= 10{,}4\,\%$ frente a $5{,}8\,\%$ de *CLIP4Outfit*, con una fracción de sus parámetros y una latencia dos órdenes de magnitud inferior, evidenciando la transferibilidad de los espacios de compatibilidad hacia la búsqueda semántica.

Entre las principales aportaciones destacan la reconstrucción unificada del estado del arte y el primer *benchmarking* homogéneo entre los tres paradigmas bajo idénticas condiciones. La validación sobre catálogos comerciales reales mediante *web scraping*, planteada como objetivo inicial, no pudo completarse por restricciones legales ajenas al proyecto, quedando como línea prioritaria de trabajo futuro.

Palabras clave: recomendación de moda, compatibilidad de prendas, representación multimodal, redes neuronales de grafos, aprendizaje de métricas, recuperación *Text-to-Outfit*.

Abstract

This Master's Thesis addresses fashion recommendation, where the compatibility of an outfit does not depend on each garment in isolation but on their interaction in terms of texture, color, shape, and context of use. This work sets out to develop and validate a system capable of generating and combining image and textual-description *embeddings* for garments, with the ability to generalize across different *outfit* recommendation tasks.

To this end, three state-of-the-art architectures are implemented, trained, and compared under a homogeneous protocol: *SCE-Net*, *HGNN*, and *Recommendation Engine*, on the Polyvore Outfits benchmark (nondisjoint split), on the *Fill-In-The-Blank* and *Compatibility Prediction* tasks. In addition, a *Text-to-Outfit* module is developed to retrieve complete outfits from natural-language descriptions, contrasting a lightweight adaptation of the *Recommendation Engine* latent space against the dedicated contrastive architecture *CLIP4Outfit*.

Recommendation Engine proves to be the most robust and balanced architecture (AUC $=$ $0{,}896$, consistent with the state of the art), compared to the greater sensitivity of *SCE-Net* and *HGNN* to training configuration and dataset scale, respectively. On *Text-to-Outfit*, the *Recommendation Engine* adaptation reaches Recall@5 $= 10{,}4\,\%$ versus $5{,}8\,\%$ for *CLIP4Outfit*, using a fraction of its parameters and with an inference latency two orders of magnitude lower, evidencing the transferability of the learned compatibility spaces to semantic search.

Among the main contributions are the unified reconstruction of the state of the art and the first homogeneous *benchmarking* of the three paradigms under identical conditions. Validation on real commercial catalogs via *web scraping*, set out as an initial objective, could not be completed due to legal restrictions beyond the scope of the project, and is established as a priority line for future work.

Keywords: fashion recommendation, garment compatibility, multimodal representation, graph neural networks, metric learning, *Text-to-Outfit* retrieval.

Introducción 1

La industria de la moda lleva años digitalizándose, cada vez más usuarios realizan sus compras de forma *online* y entre los que acuden a tiendas, hay quienes posteriormente acuden a la web para realizar la adquisición. Las tiendas aún mantienen escaparates atractivos para captar clientes y buscan un trato cercano para mantenerlos; no obstante, son las grandes marcas quienes buscan aplicar estas estrategias en el marco digital. La IA ha pasado de gestionar la logística a ocupar el centro de la experiencia de compra.

Recomendar una película es relativamente directo: géneros, directores, valoraciones de usuarios similares. La moda no funciona así. La compatibilidad de un conjunto no depende de cada prenda por separado, sino de cómo interactúan entre sí. Texturas, formas, colores, incluso el contexto de uso, todo entra en juego simultáneamente.

El conjunto de datos Polyvore ha sido el benchmark de referencia en este campo durante años. Sobre él se han construido arquitecturas cada vez más sofisticadas: redes de grafos que modelan conjuntos como nodos interconectados, mecanismos de atención, sistemas de regresión profunda. Los resultados en tareas como Fill-in-the-blank or Compatibility Prediction han mejorado de forma sostenida.

Gran parte de esa investigación optimiza para superar el estado del arte por márgenes pequeños en condiciones controladas. Este proyecto parte de una premisa distinta: antes que maximizar una métrica aislada, interesa construir un sistema con un enfoque general, que funcione en condiciones reales.

De acuerdo con el estado del arte, trabajar simultáneamente con imagen y texto resuelve una limitación de los modelos de la capacidad de generalización frente a los que tan solo emplean imágenes. Algunas características como el material, la ocasión o el estilo (vintage, gótico, etc.) pueden ser difíciles o ambiguas. Este trabajo la utiliza para desarrollar una funcionalidad que va más allá de la recomendación clásica: construir un outfit completo a partir de una descripción en lenguaje natural.

Para comprobar en qué medida lo aprendido generaliza hacia la variabilidad y el ruido propios de cualquier plataforma de e-commerce, el trabajo analiza la viabilidad de evaluar el sistema con datos procedentes de web scraping. Aunque la aplicación práctica sobre catálogos comerciales reales se vio delimitada por restricciones legales externas, la diferencia entre un modelo que funciona en el laboratorio y uno desplegable radica en anticipar y diseñar soluciones adaptadas a dichos condicionantes de producción.

Descripción del problema 2

2.1. Modelado del Estilo y la Armonía entre prendas

Componer un conjunto de prendas que resulte armonioso es una tarea que los seres humanos resuelven de forma intuitiva, apoyándose en años de exposición a referentes visuales, tendencias y normas estéticas implícitas. Sin embargo, trasladar esta capacidad al plano computacional revela una complejidad multidimensional.

La primera dificultad radica en una distinción conceptual crítica: la diferencia entre similitud y compatibilidad. Dos camisetas blancas son similares entre sí, pero esa semejanza es estéril al estructurar un conjunto completo. En cambio, un pantalón vaquero y una blusa estampada son visualmente dispares y, no obstante, altamente compatibles. Un sistema limitado a identificar parecidos visuales es, por lo tanto, incapaz de modelar las reglas de composición de un conjunto.

A esto se suma la naturaleza no transitiva de la compatibilidad. Que una prenda A combine con una prenda B, y esta última con una prenda C, no garantiza la compatibilidad entre A y C. A diferencia de las métricas de similitud geométrica tradicionales, que asumen propiedades de transitividad, la coherencia de un conjunto depende de la interacción global de todos sus elementos y del contexto de uso, lo que invalida el análisis aislado por pares independientes.

Por último, el concepto de estilo es inherentemente subjetivo y carece de una definición operativa universal. La discrepancia estética, incluso entre profesionales de la moda, introduce una ambigüedad latente que complica tanto el diseño de las arquitecturas como su evaluación. Esto dificulta discernir si una recomendación es estéticamente óptima o si únicamente satisface una métrica cuantitativa desconectada de la percepción humana.

2.2. Recuperación de Outfits basada en Texto (Text-to-Outfit)

Una variante compleja de este dominio es la generación de un *outfit* completo a partir de una descripción en lenguaje natural (ej. «conjunto de oficina para la transición de verano a otoño»). Mientras que la evaluación clásica de compatibilidad asume la existencia de prendas

preseleccionadas, esta tarea invierte el punto de partida: debe traducir una intención lingüística abstracta en una selección concreta, visualmente coherente y disponible en un catálogo físico. Este planteamiento es especialmente valioso para el comercio electrónico, pues replica el comportamiento natural de búsqueda de un usuario y el servicio de un estilista personal sin requerir que el cliente aporte una referencia visual previa.

Sin embargo, esta aproximación añade una doble brecha de representación. En primer lugar, exige alinear la flexibilidad e imprecisión del lenguaje con la naturaleza concreta del catálogo de producto. Para salvar este obstáculo, las soluciones del estado del arte suelen adaptar arquitecturas de preentrenamiento contrastivo imagen-texto (similares a los modelos de recuperación texto-vídeo). No obstante, estas aproximaciones deben adecuar mecanismos de atención temporal (diseñados para fotogramas secuenciales) a un escenario no temporal donde lo que predomina es la consistencia de estilo.

En segundo lugar, estos sistemas operan bajo de catálogos cerrados, comportándose como motores de recuperación de información más que como agentes creativos. Esta limitación restringe la propuesta de combinaciones genuinamente novedosas y se ve agravada por la escasez de datos. Los conjuntos de datos disponibles para esta tarea multimodal, aunque asocian texto e imagen, son sustancialmente más reducidos que los corpus de dominio general, lo que limita la capacidad de los modelos para generalizar asociaciones complejas entre semántica y estética visual.

Objetivos 3

3.1. Objetivo Principal

Desarrollar y validar un sistema que genere y combine embeddings de imagen y descripciones de prendas. Con la capacidad de generalizar la captura de características a diferentes tareas dentro del contexto de recomendación de outfits. El sistema basado en aprendizaje profundo debe aplicar técnicas del estado del arte, demostrando su utilidad práctica en entornos reales

3.2. Objetivos Específicos

1. Analizar y comparar críticamente diferentes arquitecturas de vanguardia (GNN, modelos de regresión y redes siamesas) para la tarea de predicción de compatibilidad de prendas.

2. Implementar un espacio de representación multimodal que integre las características visuales de las prendas con la información semántica de sus descripciones textuales para enriquecer el aprendizaje del modelo.

3. Validar el rendimiento del sistema en las tareas estándar de la literatura: Compatibility Prediction y Fill-in-the-blank, utilizando el conjunto de datos Polyvore como base de referencia.

4. Diseñar e implementar un módulo de recolección de datos mediante web scraping para la evaluación del modelo en entornos comerciales reales, analizando la viabilidad técnica y los condicionantes legales asociados a su ejecución.

5. Desarrollar un módulo de interacción por texto que permita la recuperación y recomendación de prendas aprovechando los embeddings generados, facilitando una búsqueda basada en conceptos abstractos de moda.

3.3. Alcance

Con el fin de satisfacer los objetivos planteados y validar el éxito de este proyecto, el desarrollo se acota bajo las siguientes consideraciones y delimitaciones:

- **Generalización del Aprendizaje:** Evaluar la capacidad de transferencia del modelo sobre las tareas objetivo, planteando el análisis de viabilidad y los condicionantes técnicos y legales para su despliegue sobre entornos o catálogos comerciales reales.
- **Enfoque Tecnológico:** Se aclara que el desarrollo no abordará metodologías basadas en IA Generativa. El objetivo final se restringe rigurosamente a sistemas de recomendación y optimización combinatoria de prendas de vestir ya existentes dentro de un catálogo determinado.
- **Ausencia de Datos de Usuario y Personalización:** El modelo desarrollado opera de manera estrictamente agnóstica al perfil del consumidor, al no disponer de datos históricos, interacciones previas ni información personal de los usuarios. Esta falta de contextualización individual limita el sistema a ofrecer recomendaciones de propósito general basadas únicamente en las características intrínsecas de los productos, impidiendo el aprendizaje de estilos dinámicos o patrones de comportamiento específicos de los clientes.
- **Sesgo de Representación y Exclusión de Atributos Físicos:** Por motivaciones éticas y de privacidad, el sistema omite variables críticas como morfologías corporales, dimensiones anatómicas o tallaje. No obstante, esta ausencia de información técnica genera una limitación metodológica: se asume la existencia de un sesgo implícito en los vectores de características de las prendas, ya que los conjuntos de datos de entrenamiento estándar del sector tienden a reflejar patrones de diseño no inclusivos o estandarizados normativamente, lo cual restringe la equidad del modelo ante una diversidad real de cuerpos.
- **Desarrollo de Software:** El presente proyecto excluye explícitamente la elaboración de interfaces de usuario (front-end), plataformas de gestión o el despliegue comercial de los modelos (deployment). El foco exclusivo se centra en la aplicación práctica de conocimientos de Inteligencia Artificial, prescindiendo del desarrollo de componentes de ingeniería de software ajenos a las competencias del máster.

3.4. Límites

La evaluación de un modelo de Inteligencia Artificial en entornos reales traslada la teoría a un escenario complejo donde surgen restricciones inevitables. Con el objetivo de mantener el rigor científico de este trabajo, a continuación se exponen de manera crítica las limitaciones y dificultades metodológicas que afectaron al proceso de experimentación, las cuales sientan las bases para futuras líneas de investigación:

- **Dependencia de Tendencias Temporales y Estacionales:** El rendimiento práctico del modelo se encuentra supeditado a la estacionalidad y vigencia del conjunto de datos de entrenamiento. Debido a la naturaleza cambiante del sector de la moda, la diversidad de estilos y la adaptación a tendencias efímeras pueden resultar limitadas si el catálogo no es actualizado de forma continua, restringiendo su viabilidad a largo plazo sin un flujo de datos dinámico.

- **Margen de Replicabilidad:** El modelo seleccionado a partir de la literatura científica debe alcanzar un rendimiento comparable en las tareas de Compatibility Prediction y Fill-In-The-Blank respecto a las métricas publicadas por los autores originales. Se asume de antemano la existencia de un margen de desviación aceptable entre ambos resultados, el cual se atribuye a factores críticos de la réplica exógena como la falta de reproducibilidad en el preprocesamiento, la optimización extrema (cherry-picking) de los estudios originales, las disparidades en la arquitectura de hardware y la aleatoriedad intrínseca en la inicialización de los modelos.

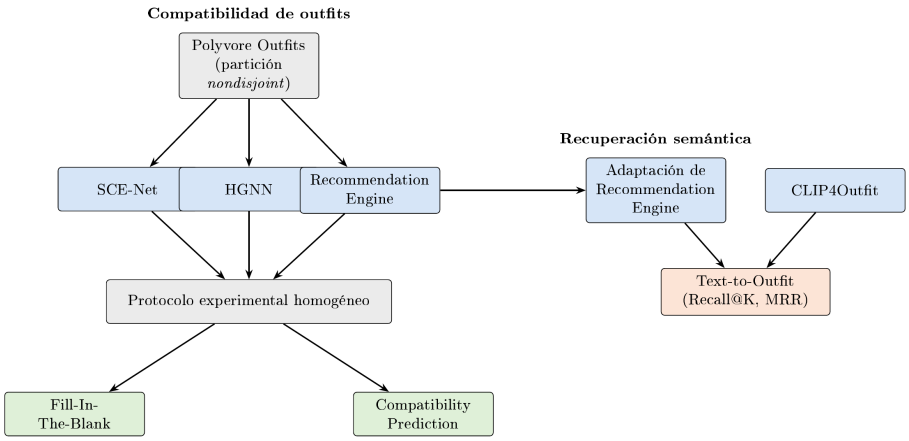


Figura 3.1: Esquema general del sistema desarrollado: las tres arquitecturas de compatibilidad (SCE-Net, HGNN y Recommendation Engine) se entrenan y evalúan bajo un protocolo experimental homogéneo sobre el benchmark Polyvore Outfits, en las tareas Fill-In-The-Blank y Compatibility Prediction. En paralelo, el módulo Text-to-Outfit contrasta una adaptación del espacio latente de Recommendation Engine frente a la arquitectura contrastiva CLIP4Outfit.

Marco Teórico 4

En este capítulo se concretan los diferentes aspectos esenciales, definiciones y formalismos matemáticos que constituyen el núcleo teórico de esta investigación. Su propósito es establecer un marco de referencia unificado sobre el problema de la compatibilidad estético-textil, el aprendizaje de métricas (metric learning) y las estructuras de grafos y Transformers. Esta base conceptual resulta indispensable para justificar la idoneidad técnica del modelo seleccionado y contextualizar el desarrollo metodológico que se expondrá en los capítulos posteriores.

4.1. El problema de la compatibilidad de moda

4.1.1. Delimitación conceptual: Similitud vs. Compatibilidad

Los enfoques iniciales en el aprendizaje de representaciones visuales aplicados al ámbito de la moda adoptaban metodologías heredadas de la búsqueda de imágenes general. Estos sistemas proyectaban todos los artículos, independientemente de su categoría, en un único espacio métrico compartido; si bien esta estrategia facilitaba el modelado de la similitud, resultaba ineficaz para capturar la noción de compatibilidad.

- **Similitud**: Mide el grado en que dos artículos de moda pertenecen a la misma categoría y comparten atributos visuales o estilísticos tan cercanos que los hacen intercambiables. Su principio subyacente es la equivalencia funcional y estética dentro del mismo tipo de prenda.
- **Compatibilidad**: Evalúa si prendas de diferentes categorías temáticas o tipologías (p. ej., un calzado, un pantalón y una chaqueta) se complementan adecuadamente para formar un conjunto armónico y cohesivo según las reglas de la creatividad o las tendencias de moda.

Esta distinción fundamental constituye el núcleo argumental del trabajo de Vasileva et al. (2018). Para resolver la problemática de los espacios compartidos, los autores introducen una propuesta en la que todos los artículos se representan inicialmente en un espacio general de características, para posteriormente ser proyectados sobre subespacios específicos en función del tipo de interacción (top-bottom, bottom-shoes, etc.)

Posteriormente, el enfoque desarrollado por Tan et al. (2019) retoma exactamente esta misma diferenciación, llevando la propuesta un paso más allá en términos de flexibilidad. Aunque el paradigma *type-aware* de Vasileva et al. (2018) demuestra su eficacia, presenta la limitación de requerir el conocimiento explícito de la condición o categoría que se está evaluando (por ejemplo, especificar de antemano que se realiza una comparación entre un *top* y un *bottom*). Esta dependencia restringe la capacidad de generalización del modelo ante categorías nuevas o no observadas durante el entrenamiento. Como solución, se propone aprender dichos subespacios de compatibilidad de manera latente y condicional.

4.1.2. No transitividad en la combinación de prendas

La razón principal por la que la compatibilidad resulta compleja de representar en un espacio latente convencional radica en la naturaleza no transitiva de las relaciones de moda. En el trabajo de Vasileva et al. (2018), se acuña el término *"triángulos rotos"* (*broken triangles*) para describir aquellos escenarios en los que, debido a la propiedad transitiva intrínseca de los espacios métricos tradicionales, el modelo induce erróneamente que un artículo A es cercano a un artículo C simplemente porque ambos comparten una alta proximidad con un artículo intermedio B (es decir, $A \approx B$ y $B \approx C \implies A \approx C$). En el contexto del estilismo, esta asunción geométrica es falsa, lo que demuestra que la compatibilidad no puede capturarse adecuadamente basándose únicamente en funciones de distancia. A modo de ilustración: un gorro de lana puede ser compatible con una chaqueta de invierno, y ese mismo gorro puede combinar con unas botas de nieve; sin embargo, esto no implica que dicha chaqueta y esas botas sean compatibles entre sí o adecuadas para el mismo conjunto.

Más allá de esta limitación de carácter estructural y geométrico, el modelado de la compatibilidad se enfrenta a una importante barrera semántica: la subjetividad inherente al criterio estético. Este fenómeno queda reflejado en el estudio de Prato et al. (2020), donde, tras la validación cuantitativa del modelo mediante métricas estándar, se procedió a realizar una evaluación cualitativa de las recomendaciones con la participación de tres estilistas expertos. Los resultados del estudio demostraron una elevada variabilidad en las opiniones de los expertos, registrando niveles de consenso con el modelo que oscilaban desde un notable 91 % hasta un escaso 67 %.

4.2. Representación matemática de ítems de moda

4.2.1. Modelos de extracción visual

1. Previo al uso de CNNs, los sistemas de recomendación tradicionales se basaban exclusivamente en filtrado colaborativo (datos de navegación, compras de usuarios o clics) o metadatos de texto. Si un artículo había sido introducido recientemente a la base de datos, el sistema tenía dificultades para recomendarlo (*cold-start*).

La primera publicación en utilizar una Red Neuronal Convolucional preentrenada para extraer características visuales de los productos de un católogo fue McAuley et al. (2015). Su solución innovadora aplicaba **transfer-learning** en AlexNet preentrenada con ImageNet, usaban la capa densa del top model antes de la clasificación como embeddings visuales para posteriormente entrenar una transformación linear para aprender un espacio métrico de similitudes.

2. La adopción del **fine-tuning** se convierte en un estándar con el artículo de Veit et al. (2015), su solución propone utilizar una red siamesa para ajustar los pesos de la red específicamente para aprender la compatibilidad de moda.

3. Vasileva et al. (2018) y Tan et al. (2019) para la tarea de extracción de características visuales, realizan una ligera mejora empleando **ResNet-18**. ResNet, al ser una red residual, introduce las *skip connections*: en lugar de que cada bloque aprenda una transformación completa $H(x)$, aprende un residuo $F(x) = H(x) - x$, de modo que la salida del bloque es $F(x) + x$. Esto permite entrenar redes mucho más profundas sin sufrir el problema del desvanecimiento del gradiente, porque el gradiente puede fluir directamente a través de las conexiones de salto (tal y como se ilustra en la figura 4.1).

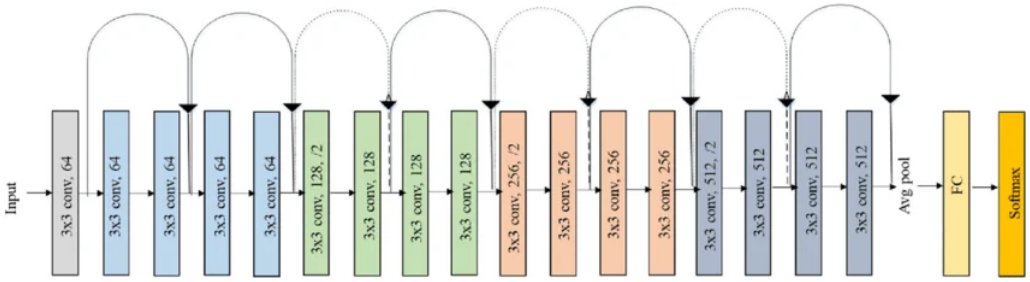


Figura 4.1: Arquitectura ResNet-18 original. Figura adaptada de Ramzan et al. (2019)

4. **InceptionV3** es una arquitectura de la familia GoogLeNet/Inception que introduce los módulos Inception: en lugar de elegir un único tamaño de filtro convolucional por capa (3×3, 5×5, etc.), cada módulo aplica varios tamaños de filtro en paralelo sobre la misma entrada y concatena sus salidas, permitiendo que la red capture patrones a múltiples escalas simultáneamente sin explotar el número de parámetros. Han et al. (2017b) introduce este modelo como estándar demostrando la superioridad frente a ResNet en este subcampo.

5. El **Vision Transformer** rompe con el paradigma convolucional: en lugar de procesar la imagen con filtros locales que se deslizan espacialmente, divide la imagen en parches de tamaño fijo (por ejemplo, 16×16 píxeles), aplana cada parche en un vector, y lo proyecta linealmente a un embedding, tratando la secuencia de parches exactamente como una secuencia de tokens de texto en un Transformer estándar. Sobre esta secuencia de tokens se aplican bloques de *self-attention* multi-cabeza: cada parche puede atender directamente a cualquier otro parche de la imagen, sin importar la distancia

espacial entre ellos.

Gulati (2024) incorpora explícitamente el ViT como mejora sobre el *pipeline* InceptionV3 original de Han et al. (2017b), usando Google Inception y el modelo Visual-Transformer (ViT) de Hugging Face para crear los embeddings de imagen, reportando resultados donde ViT muestra una ligera ventaja (esquema de la figura 4.2).

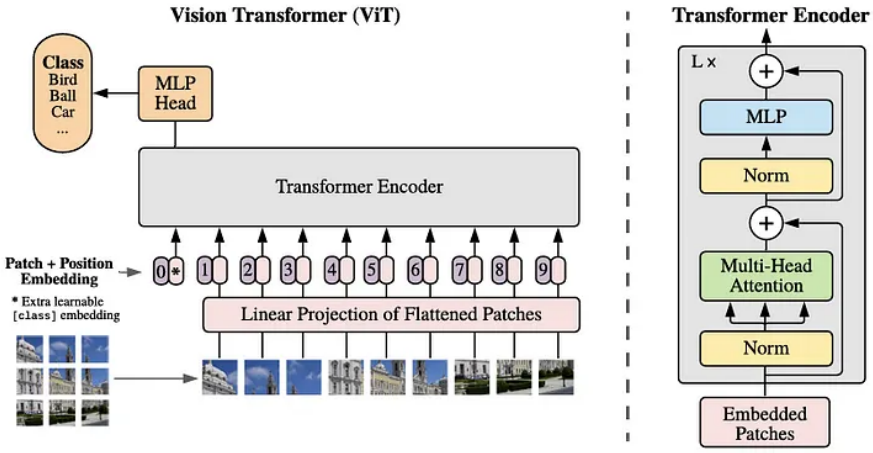


Figura 4.2: Arquitectura del Vision Transformer (ViT). Figura tomada de Li (2025).

4.2.2. Modelos de representación semántica y textual

One-hot encoding: La forma más simple de representar una descripción textual es el one-hot encoding. No obstante, esta representacion tiene dos grandes limitaciones:

1. **Alta dimensionalidad y dispersión**: el tamaño del vector es equivalente al tamaño del vocabulario, y cada vector termina formando por ceros principalmente, esto desperdicia el tamaño y dificulta el aprendizaje.
2. **Ausencia de semántica**: bajo esta representacion, toda palabra se encuentra a la misma distancia, el espacio no codifica de ninguna manera el significado de las palabras si no su existencia o ausencia dada la prenda.

Han et al. (2017b) y Gulati (2024) emplea un vector de one-hot de tamaño 2757 (tamaño del vocabulario tras un preprocesamiento), que luego proyectan mediante una Fully Dense Network hacia un espacio visual-semantico de 512 dimensiones.

Word2vec introduce la idea de embeddings distribucionales: en lugar de asignar a cada palabra un vector arbitrario y disperso, se entrena una red superficial (usando arquitecturas skip-gram o CBOW) sobre un corpus muy grande para predecir palabras a partir de su contexto. El resultado es que palabras que aparecen en contextos similares terminan con vectores cercanos en un espacio denso de baja dimensión (típicamente 100-300 dimensiones), capturando relaciones semánticas y hasta analógicas de forma

emergente. Para el dominio de moda, esto significa que palabras como *vestido* o *falda* terminan próximas entre sí en el espacio.

El artículo de Klein et al. (2015) se emplea como fuente para la representación semántica. Vasileva et al. (2018) no emplea Word2vec de forma directa sino que se emplea como un recurso intermedio para aplicar HGLMM Fisher vectors.

Ya que Word2vec obtiene un solo vector por palabra, y las descripciones de los productos contienen múltiples palabras. **HGLMM Fisher vectors** funciona como un método consistente en el que agregar los vectores manteniendo su representación.

HGLMM representa un documento mediante un vector de Fisher obtenido a partir de un modelo de mezcla de distribuciones Gaussianas y Laplacianas ajustado sobre los embeddings de palabras. A diferencia del promedio de embeddings, esta representación captura cómo la distribución de palabras de un documento difiere de la distribución global del corpus, preservando información estadística aunque de dimensionalidad mucho mayor.

Es por eso que, en los trabajos Vasileva et al. (2018) y Tan et al. (2019), se propone emplear una PCA sobre los fisher vectors, reduciendo un espacio computacionalmente costoso y de dimensiones redundantes en una representacion más manejable.

En otras aplicaciones como Bettaney et al. (2019) sugieren **GloVe** como alternativa a Word2Vec. GloVe persigue el mismo objetivo que Word2Vec, obtener embeddings densos con información semántica, pero emplea un enfoque basado en las coocurrencias globales de palabras en el corpus, en lugar de predecir el contexto local. En la práctica, ambos métodos generan representaciones semánticas de calidad similar, diferenciándose principalmente en su estrategia de entrenamiento.

4.2.3. Fusión multimodal y representación categórica

Los apartados anteriores 4.2.1 y 4.2.2 trataron por separado la extracción de características visuales y textuales. Sin embargo, ningún modelo de compatibilidad de moda opera sobre una única modalidad de forma aislada: el problema real requiere combinar imagen, texto y, frecuentemente, metadatos categóricos, en una única representación por ítem sobre la cual pueda operar el módulo de compatibilidad.

- **Proyección a un espacio común**: Vasileva et al. (2018) no realiza una fusión explícita a nivel arquitectónico, sino una fusión indirecta, inducida por la función de pérdida. La imagen y la descripción textual de cada ítem se proyectan mediante capas de proyección independientes hacia un mismo espacio vectorial compartido, y se entrena mediante una pérdida de ranking de tipo **Visual-Semantic Embedding (VSE)**, que fuerza a que el embedding visual de un ítem quede próximo al embedding de su propia descripción textual, y alejado de las descripciones de ítems no relacionados.

- **Combinacion explicita mediante el producto Hadamard**: Vasileva et al. (2018) propone una variante adicional denominada *Metric*, en la que, se calcula el producto de Hadamard ($\mathbf{x} \odot \mathbf{y}$) entre ambos embeddings, y el resultado se pasa por una capa totalmente conectada cuyos pesos aprendidos actúan como una función de distancia generalizada.

  Tan et al. (2019) emplea un mecanismo relacionado, allí, el producto elemento a elemento se emplea para aplicar máscaras de condición de similitud aprendidas sobre un único embedding visual, generando múltiples sub-embeddings que capturan distintas nociones de similitud (color, categoría, forma) sin necesitar supervisión explícita del tipo de ítem en tiempo de inferencia.

- **Módulos paralelos por modalidad**: La tercera estrategia, la ofrece Prato et al. (2020). Su arquitectura consta de una primera etapa modular en la que cada tipo de entrada imagen, texto y valores categóricos, se procesa mediante un módulo dedicado (una versión reducida de ResNet-18 para la imagen, un diccionario GloVe para la descripción textual, y una codificación categórica para los metadatos), y las tres representaciones se concatenan para formar un espacio común. A diferencia de las estrategias previas, aquí la fusión involucra tres modalidades simultáneamente, y el propio diseño se plantea explícitamente como modular y extensible a nuevos tipos de entrada.

- **Fusión multimodal mediante preentrenamiento contrastivo: CLIP**. Una alternativa más reciente a las estrategias anteriores es la propuesta por Radford et al. (2021) con *Contrastive Language-Image Pretraining*. A diferencia de los enfoques descritos, CLIP no aprende la fusión multimodal de forma supervisada sobre el dominio de moda, sino que preentrena dos *encoders* independientes. Uno visual (Vision Transformer o ResNet) y uno textual (Transformer), sobre 400 millones de pares imagen-texto recopilados de internet, mediante una función de pérdida contrastiva que fuerza a que el embedding visual de una imagen quede próximo al embedding textual de su descripción y alejado del resto.

  El resultado es un espacio vectorial compartido de 512 dimensiones en el que imagen y texto son directamente comparables mediante similitud coseno, sin necesidad de proyecciones adicionales ni pérdidas VSE durante el *fine-tuning*.

4.2.4. La etiqueta de categoría

1. **La categoría como característica de entrada**: En Prato et al. (2020), la categoría se codifica como una variable categórica tradicional y se procesa mediante un módulo dedicado en la etapa inicial, situándose exactamente al mismo nivel jerárquico que la imagen o el texto. Por su parte, Tan et al. (2019) emplea esta condición como un atributo adicional para determinar dinámicamente el peso de cada una de las M máscaras al evaluar un par de elementos.

2. **La categoría como elemento estructural**: En Gulati (2024), la categoría deja de ser un mero dato de entrada o un criterio de selección para influir directamente en la topología del modelo; en este enfoque, cada categoría se integra como un nodo dentro de una estructura de grafo. De manera similar, Vasileva et al. (2018) utiliza la categoría estructuralmente para definir el subespacio latente específico en el cual se debe realizar la comparación entre dos elementos.

4.3. Aprendizaje métrico

4.3.1. Arquitecturas siamesas y funciones de pérdida

La arquitectura siamesa fue propuesta originalmente por Bromley et al. (1993) para el problema de verificación de firmas manuscritas (esquematizado en la figura 4.3), y popularizada posteriormente en el ámbito de la visión por computador por Chopra et al. (2005) para verificación facial. Consiste en dos o más subredes idénticas que comparten pesos, cada una procesando una entrada distinta y proyectándola a un vector de características en un mismo espacio. Compartir los pesos entre ramas es la condición que garantiza que ambas entradas se transformen mediante exactamente la misma función, de modo que la distancia entre sus proyecciones sea comparable y tenga significado geométrico.

Veit et al. (2015) son los primeros en aplicar explícitamente esta arquitectura al dominio de moda, usan CNNs siamesas para aprender una transformación desde el espacio de imagen hacia un espacio de similitud latente, entrenada sobre pares de ítems heterogéneos (de distintas categorías) que co-ocurren frecuentemente. Posteriormente, en Vasileva et al. (2018) y Tan et al. (2019) emplean triplet loss como señal de entrenamiento.

- **Contrastive loss**: Esta función de pérdida opera sobre pares de entradas etiquetados como similares o diferentes. Minimiza la distancia entre pares similares, y penaliza pares diferentes solo si su distancia cae por debajo de un margen prefijado (si ya están suficientemente separados, no se penalizan más). Formalmente:

$$\mathcal{L}_{\text{contrastive}} = y\, d(x_1, x_2)^2 + (1 - y)\, \max\left(0,\, m - d(x_1, x_2)\right)^2 \qquad \text{Hadsell et al. (2006)}$$

- **Triplet loss**: Como su nombre indica esta función de pérdida opera sobre tripletes donde cada uno contiene un ítem ancla, ítem compatible y un ítem no compatible. En el contexto de recomendación de prendas, se emplea un conjunto como ejemplo del que se extrae el ancla y el ítem compatible, el ítem no compatible se extrae aleatoriamente de otro atuendo y debe coincidir en categoría al ítem compatible. Se busca minimizar la distancia del ítem compatible respecto al ancla, y maximizar la del ítem no compatible respecto al ancla, dentro de un margen fijo.La formulación es:

$$\mathcal{L}_{\text{triplet}} = \max\left(0,\; d(a, p) - d(a, n) + m\right) \qquad \text{Schroff et al. (2015)}$$

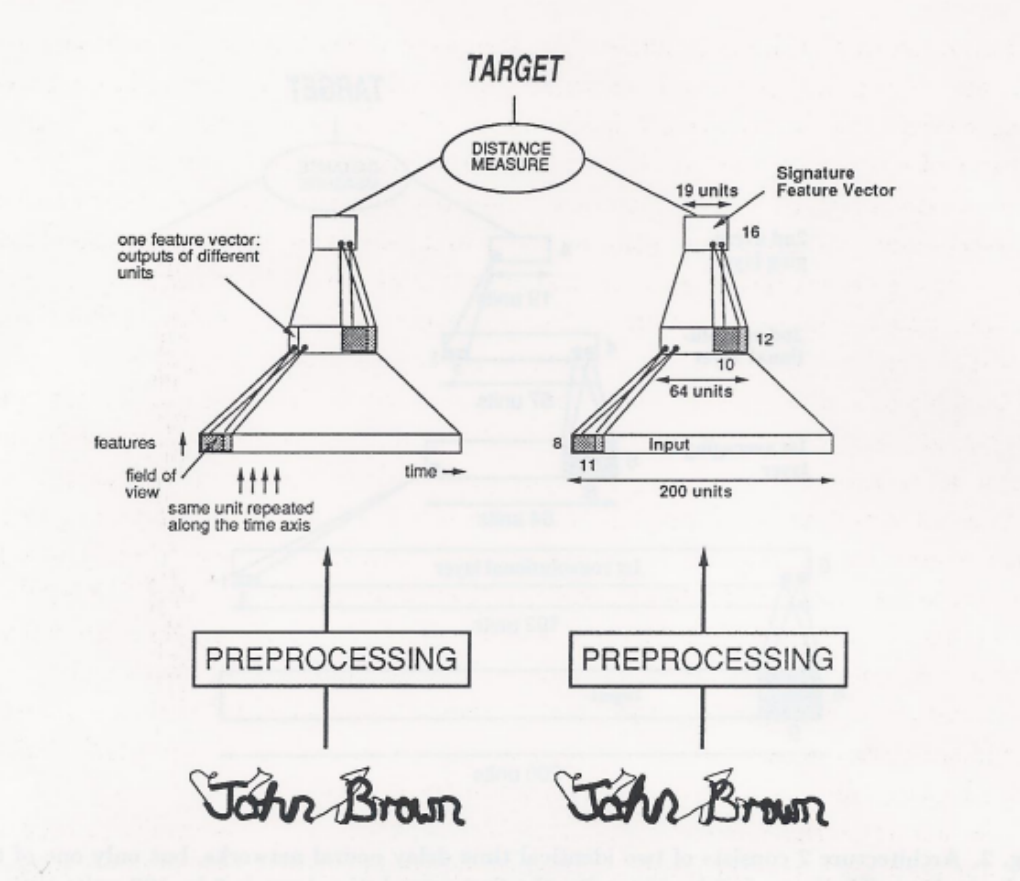


Figura 4.3: Arquitectura siamesa original basada en redes con retardo temporal para la comparación de características. Figura tomada de Bromley et al. (1993).

Vale la pena cerrar el apartado retomando lo comentado en el apartado 4.1.2. Tanto la arquitectura siamesa como la triplet loss constituyen el paradigma de *metric loss* que se critica por representar solamente la simetría y favorecer la transitividad de los ítems. Esto motiva, mas adelante en la literatura, a la creación de arquitecturas menos rígidas que emplean subespacios o máscaras de condición no métricas.

4.3.2. Nociones de distancia geométrica en espacios latentes

En este apartado se repasan y comparan algunas de las funciones de distancias se emplean en las referencias para calcular diferencias entre ítems dentro del espacio latente. Al ser este un tema estudiado en el máster, se trata de forma superficial.

- **Distancia euclídea**: Su ventaja es la simplicidad y la interpretación geométrica directa. Su limitación, que se vuelve relevante en espacios de alta dimensión, es el fenómeno de concentración de distancias: a medida que aumenta la dimensionalidad, las distancias euclidianas entre puntos aleatorios tienden a volverse relativamente uniformes, lo que dificulta distinguir vecinos realmente cercanos de los lejanos.

- **Similitud coseno**: Es la métrica estándar en los espacios de embeddings de palabras (word2vec, GloVe, introducidos en 4.2.2). Al depender únicamente de la orientación de los vectores y no de su magnitud, es menos sensible a diferencias de escala entre las distintas ramas de la red (visual vs. textual), lo cual resulta conveniente precisamente cuando se comparan representaciones que provienen de arquitecturas y modalidades distintas.

- **Distancia de Mahalanobis**: A diferencia de la rigidez de la distancia euclídea, esta métrica actúa deformando el espacio latente, permitiendo estirar o encoger ciertas direcciones. Su propósito es ponderar las dimensiones según su relevancia para la tarea, logrando que aquellas características críticas (como el color o el estilo en la moda) tengan un mayor peso a la hora de determinar la similitud.

4.4. Tareas y métricas de evaluación estándar

Con el objetivo de crear una base sobre la que poder comparar los distintos *pipelines* y arquitecturas de forma objetiva, en la literatura se han aceptado una serie de tareas concretas para evaluar el rendimiento de los modelos; En concreto para este trabajo, las dos primeras tareas. El dataset de Polyvore Outfits, que se ha empleado como base de datos en la mayoría de publicaciones de referencia (y también este trabajo), incluye los tests de ambas tareas para facilitar la fase de prueba de en los proyectos.

1. **Fill-In-The-Blank**: La tarea de completar un conjunto o FITB, consiste en: dado un conjunto al que se le ha retirado un ítem, y un conjunto de cuatro candidatos, el modelo

Tabla 4.1: Comparativa de las métricas de distancia y espacios de representación en los artículos de referencia

Artículo	**Distancia empleada**	**Espacio en que opera**
McAuley et al. (2015)	Mahalanobis (bajo rango)	Único espacio embebido compartido
He et al. (2016), Monomer	Euclidiana sobre proyecciones distintas	Espacio ancla vs. N pseudoespacios
Veit et al. (2015)	Euclidiana (siamesa)	Espacio de estilo compartido
Vasileva et al. (2018), base	Euclidiana	Subespacios *type-specific*
Vasileva et al. (2018), variante "Metric"	Producto de Hadamard + capa totalmente conectada (FC)	Subespacios *type-specific*
Vasileva et al. (2018), VSE	Coseno (marco de Kiros et al. (2014))	Espacio visual-semántico compartido
Tan et al. (2019), SCE-Net	Euclidiana sobre embeddings enmascarados	M subespacios de condición

debe identificar cuál de los candidatos completa correctamente la combinación de prendas de forma más compatible con el resto de piezas. La métrica empleada es accuracy: la proporción de elecciones respondidas correctamente

Se introduce mediante el conjunto de datos Maryland Polyvore en el artículo Han et al. (2017b), donde los negativos se muestran aleatoriamente sin restriccion de categoría, lo que se volvia volver en una tarea de buscar la tarea equivocada. Vasileva et al. (2018) corrige el dataset y los candidatos incorrectos pertenecen a la misma categoría que el correcto.

2. **Predicción de compatibilidad**: Este problema se formula como clasificación binaria sobre el *outfit* completo. El modelo debe emitir una puntuación de compatibilidad o coherencia del conjunto. Para evaluar la tarea, se emplea un conjunto de atuendos negativos o incompatibles. Vasileva et al. (2018) generan estos negativos de forma sistemática: por cada *outfit* real, generan una combinación incompatible reemplazando cada prenda por otra de la misma categoría.

   La métrica estándar es el Área Bajo la Curva ROC (AUC), que mide la capacidad del modelo para ordenar correctamente conjuntos compatibles por encima de los incompatibles, con independencia del umbral de decisión concreto que se elija.

3. **Completa el look** introducida por Kang et al. (2019) es una variante interesante: en lugar de partir de imágenes de producto aisladas (fondo blanco, como en Polyvore), parte de una imagen de escena real (una persona vestida en un contexto natural) y

recomienda un producto compatible con ese contexto visual completo.

4. **Composed Image Retrieval (CIR)**: Dada una imagen de consulta de un producto y un texto modificador que describe un cambio deseado (*p. ej*, «el mismo vestido pero en color rojo y con manga larga»), el sistema debe recuperar del catálogo la prenda que integre ambas señales. A diferencia de la búsqueda por texto o imagen de forma aislada, CIR exige fusionar la representación visual con la instrucción textual en un *embedding* compuesto, empleando técnicas de interacción grano fino como WTI (*Weighted Token-wise Interaction*)

5. **Text-to-Outfit**: Esta tarea de recuperación utiliza únicamente lenguaje natural como entrada para buscar y seleccionar un conjunto completo de prendas en un catálogo. Dada una descripción textual de estilo o contexto (*p. ej*, «*outfit* formal para una entrevista de trabajo en tonos neutros»), el modelo debe evaluar el espacio multimodal para recuperar una combinación de ítems que satisfaga simultáneamente la semántica del texto y la coherencia visual interna del conjunto. Fue formalizada por Jang et al. (2023), eliminando la necesidad de aportar prendas de partida o historiales de usuario.

6. **Generative Outfit Recommendation (GOR)**: Introducida por Xu et al. (2024) con el modelo *DiFashion*, esta tarea traslada el problema de recomendación desde el paradigma de recuperación sobre catálogo hacia la generación sintética mediante modelos de difusión. El objetivo es sintetizar visualmente imágenes totalmente inéditas de prendas compatibles con un estilo, texto o conjunto parcial dado. Esta tarea combina los retos de la compatibilidad de moda con la generación de imágenes condicionada, y se evalúa mediante métricas de calidad y fidelidad de imagen (*FID*, *IS*).

Como se puede observar, existen multitud de tareas creadas expresamente para la finalidad que se busca expresamente en cada proyecto.

4.5. Paradigmas de recomendación: enfoques basados en producto y enfoques personalizados

La razón de la existencia de ambos paradigmas en la literatura no es una cuestión de que una haya sustituido a la otra, sino que responden a preguntas distintas sobre el mismo problema. Dos combinaciones igual de compatibles según cualquiera de estos modelos pueden no gustarle por igual a dos personas distintas, porque el gusto personal introduce un factor que la compatibilidad objetiva no captura por definición.

4.5.1. Fundamentos teóricos de la recomendación personalizada

La evolución de la recomendación personalizada se inició en los años 90 con el filtrado colaborativo basado en memoria, como GroupLens (Resnick et al. (1994)), el cual calculaba la similitud entre perfiles de usuarios sin analizar el contenido de los productos; un enfoque

que Amazon transformó hacia una variante ítem-a-ítem más escalable y que hoy fundamenta la recomendación por compatibilidad. Posteriormente, el paradigma de factores latentes introducido por Koren et al. (2009) mediante la factorización de matrices demostró que proyectar usuarios e ítems en un espacio latente de baja dimensión gestionaba mejor la dispersión de los datos, compartiendo el mismo principio matemático subyacente que la transformación de Mahalanobis de bajo rango.

Finalmente, la llegada del aprendizaje profundo con modelos como Neural Collaborative Filtering He et al. (2017) reemplazó el producto interno clásico por redes neuronales densas capaces de aprender interacciones no lineales, sentando la base metodológica de la que heredan directamente las arquitecturas modernas de moda Lin et al. (2019), las cuales simplemente sustituyen el ítem genérico por representaciones complejas de *outfits* completos procesados mediante redes convolucionales.

4.6. Ecosistema de Datos, Viabilidad Práctica y Sesgos

En esta sección vamos a introducir diferentes bases de datos empleadas en la literatura. Cada fuente incluye como referencia árticulos donde han sido empleados.

Maryland Polyvore Es la versión original extraída de la red social Polyvore. Se utiliza principalmente para evaluar los primeros modelos secuenciales de compatibilidad. Su característica clave es que el muestreo de prendas incorrectas para la tarea de *Fill-In-The-Blank* (FITB) se realiza de forma totalmente aleatoria sin importar la categoría. Esto sirve para validar la capacidad básica del modelo, aunque en la literatura se considera un entorno de evaluación artificialmente sencillo (ya que es muy fácil descartar un elemento incorrecto si es un cinturón en lugar de un pantalón).(Han et al. (2017b) y Vasileva et al. (2018))

Polyvore Outfits Diseñado específicamente para corregir los fallos metodológicos de la versión de Maryland, se ha convertido en el *benchmark* de referencia actual del campo. Se utiliza para evaluar la compatibilidad global bajo condiciones rigurosas: introduce divisiones estrictas que aseguran que ninguna prenda de test haya sido vista en el entrenamiento y obliga a que los candidatos incorrectos de la tarea FITB pertenezcan a la misma categoría que la prenda correcta, forzando al modelo a aprender matices de estilo finos.(Tan et al. (2019))

Polyvore-U Variante de Polyvore que incorpora información de usuario, recogiendo tanto los conjuntos creados por cada usuario como sus interacciones implícitas con los conjuntos de otros usuarios (visualizaciones, guardados, valoraciones). A diferencia de las variantes puramente orientadas a compatibilidad visual, Polyvore-U permite abordar tareas de recomendación personalizada, en las que el objetivo no es únicamente determinar si un *outfit* es compatible, sino si es adecuado para el estilo y las preferencias concretas de un usuario específico.(Xu et al. (2024))

La proliferación de variantes de Polyvore refleja la ausencia de un benchmark unificado en el campo de la compatibilidad de moda. Cada trabajo adapta el conjunto de datos original según sus necesidades específicas, otras variantes empleadas en la literatura son: **Polyvore-T** (Wang et al. (2019)), **OCIR sobre Polyvore Outfits** (Lin et al. (2020))

UT-Zappos50k Un catálogo enfocado exclusivamente en calzado que incluye anotaciones de diseño (tipo de cierre, altura del tacón, género). No se utiliza para entrenar los modelos de recomendación, sino como un banco de pruebas cualitativo: sirve para verificar si un modelo es capaz de descubrir por sí mismo estas características visuales y agrupar los zapatos con criterios coherentes sin haber recibido ayuda de etiquetas durante su entrenamiento.(Gulati (2024))

Fashion200K Colección de 200.000 imágenes de moda extraídas de catálogos en línea. A diferencia de los conjuntos enfocados en *outfits* completos, se centra en la recuperación de prendas a nivel individual utilizando descripciones textuales y atributos detallados (color, corte, estampado). Su principal valor radica en ser uno de los primeros *benchmarks* para evaluar la recuperación composicional imagen-texto: dada una imagen de partida y un modificador textual (*p. ej.*, «el mismo vestido pero en azul»), el sistema debe identificar el elemento que integra ambas modalidades. (Han et al. (2017a))

FashionIQ Conjunto de datos diseñado para la tarea de recuperación composicional imagen-texto (CIR). Incluye imágenes de tres categorías (camisas, vestidos y tops) anotadas con pares de modificaciones en lenguaje natural libre generadas por humanos (*p. ej.*, «más corto y con estampado floral»). A diferencia de Fashion200K, basado en atributos estructurados, las descripciones de FashionIQ capturan la subjetividad y ambigüedad de la interacción conversacional real, consolidándose como el *benchmark* estándar para evaluar la alineación fina entre imagen y lenguaje. (Wu et al. (2020))

Alibaba iFashion Es el conjunto de datos de moda a gran escala más grande del mundo, extraído de la plataforma Taobao. Registra tanto los conjuntos sugeridos por creadores como los clics reales de millones de usuarios. Se utiliza exclusivamente para la tarea de *recomendación personalizada*; es decir, para modelos que no solo buscan si un conjunto es estéticamente compatible, sino si se adapta a los gustos históricos de un cliente concreto.(Chen et al. (2019))

Dataset	Principal información	Uso típico
Maryland Polyvore	*Outfits* + prendas	Compatibilidad, FITB

Continúa en la página siguiente

Dataset	**Principal información**	**Uso típico**
Polyvore Outfits	*Outfits* completos con categorías semánticas	Compatibilidad de *outfits*, FITB con restricción categórica
Polyvore-U	*Outfits* + usuarios + historial de interacciones	Recomendación personalizada
UT-Zappos50k	Catálogo de calzado con anotaciones de diseño	Evaluación cualitativa de embeddings visuales
Fashion200K	Productos + descripciones textuales	Recuperación multimodal, *Composed Image Retrieval* (CIR)
FashionIQ	Imágenes de prendas + modificaciones textuales generadas por humanos	Recuperación multimodal, *Composed Image Retrieval* (CIR)
Alibaba iFashion	*Outfits* + usuarios + interacciones de usuario	Recomendación personalizada a gran escala

Tabla 4.2: Principales datasets empleados en la literatura de compatibilidad y recomendación de moda.

Estado del arte 5

5.1. Evolución cronológica de los sistemas de recomendación de moda

5.1.1. Representación por pares

El paradigma de comparación por pares, iniciado por McAuley et al. (2015), descompone una combinación de prendas en relaciones bilaterales independientes representando el conjunto como un grafo de prendas. Para cada par de ítems (i, j), el sistema proyecta sus vectores visuales $x \in \mathbb{R}^d$ en un espacio latente común y calcula su distancia de Mahalanobis generalizada (subsección 4.3.2). Este valor actúa como un índice de incompatibilidad continua: una menor separación geométrica en el espacio aprendido denota una mayor afinidad estética entre ambas piezas.

Para escalar esta evaluación local a un conjunto completo O de N prendas, la incompatibilidad global se calcula promediando el coste de todas las conexiones posibles en el grafo mediante una normalización combinatoria:

$$D_{\text{global}}(O) = \frac{2}{N(N-1)} \sum_{i=1}^{N} \sum_{j=i+1}^{N} d_M(x_i, x_j) \tag{5.1}$$

Este promedio permite evaluar equitativamente la coherencia de conjuntos con diferente número de elementos. Finalmente, la distancia global se transforma en una probabilidad binaria de compatibilidad mediante una función sigmoide ($P = \sigma(-D_{\text{global}})$). Sin embargo, al sustentarse sobre una métrica tradicional, el modelo impone restricciones estrictas de simetría y reflexividad (subsección 4.1.2). Esta rigidez matemática ignora la direccionalidad y asimetría intrínsecas en las reglas de combinación de la moda, constituyendo la limitación teórica que impulsa las arquitecturas posteriores en la literatura.

5.1.2. Modelos secuenciales (Bi-LSTM)

Para capturar la estructura jerárquica y secuencial de la vestimenta, Han et al. (2017b) propusieron equiparar las prendas a palabras y el conjunto a una oración procesada mediante una red Bi-LSTM. El modelo requiere ordenar los ítems en una secuencia fija según su disposición anatómica, $O = \{x_1, x_2, \ldots, x_T\}$, mapeando sus vectores visuales $x_t \in \mathbb{R}^d$ a un

embedding conjunto $e_t = W_e x_t + b_e$. A continuación, la red computa simultáneamente los estados ocultos recurrentes hacia adelante y hacia atrás:

$$\overrightarrow{h}_t = \text{LSTM}(\overrightarrow{h}_{t-1}, e_t; \Theta_{\text{fwd}}) \tag{5.2}$$

$$\overleftarrow{h}_t = \text{LSTM}(\overleftarrow{h}_{t+1}, e_t; \Theta_{\text{bwd}}) \tag{5.3}$$

La representación contextualizada de cada prenda se construye mediante la concatenación de ambos vectores: $h_t = [\overrightarrow{h}_t ; \overleftarrow{h}_t]$. La optimización se formula mediante un enfoque multitarea que combina la consistencia local, prediciendo la probabilidad de transición al siguiente ítem mediante softmax, y la compatibilidad global del conjunto. Para esto último, los estados ocultos se agregan mediante average pooling ($h_{\text{outfit}} = \frac{1}{T}\sum_{t=1}^{T} h_t$) y se proyectan mediante una función sigmoide: $P(\text{compatible} \mid O) = \sigma(W_c h_{\text{outfit}} + b_c)$. Todo el sistema se entrena de forma *end-to-end* minimizando una función de pérdida compuesta:

$$\mathcal{L} = \mathcal{L}_{\text{global}} + \lambda \mathcal{L}_{\text{local}} \tag{5.4}$$

Donde $\mathcal{L}_{\text{global}}$ evalúa la clasificación binaria del conjunto y $\mathcal{L}_{\text{local}}$ la predicción secuencial del próximo ítem. Aunque este enfoque flexibiliza la evaluación frente a métricas rígidas y captura dependencias de largo alcance (como la coherencia entre prendas no adyacentes), introduce una alta sensibilidad al criterio de ordenamiento inicial; una limitación teórica que impulsará la transición hacia arquitecturas basadas en mecanismos de atención pura como los *Transformers*.

5.1.3. Embeddings condicionados por tipo

La arquitectura propuesta por Vasileva et al. (2018) (figura 5.1) resuelve las limitaciones discutidas en la subsección **??** mediante un enfoque de dos etapas. En primer lugar, un *backbone* ResNet-18 preentrenado en ImageNet y ajustado mediante *fine-tuning* produce, para cada ítem, un **embedding general** $x \in \mathbb{R}^d$, compartido por todas las categorías —idéntico en espíritu a la arquitectura siamesa de Veit et al. (2015) vista en la subsección **??**. En segundo lugar, dado un par de categorías (t_i, t_j) (por ejemplo, *top-bottom* o *zapato-bolso*), se aprende una **proyección diagonal** $\mathbf{P}^{(t_i,t_j)}$ —una máscara de pesos aplicada elemento a elemento sobre el embedding general en lugar de una matriz densa completa— que transforma x en un embedding específico para ese par de tipos: $x^{(t_i,t_j)} = \mathbf{P}^{(t_i,t_j)} \odot x$. De este modo, la compatibilidad entre dos ítems de categorías t_i y t_j se mide únicamente dentro del subespacio (t_i, t_j) correspondiente, mientras que en el espacio de embedding general ambos ítems pueden permanecer arbitrariamente distintos.

El modelo se entrena mediante *triplet loss* (x, x^+, x^-)(subsección 4.3.1) sobre estos embeddings específicos de tipo. Los autores exploran además dos variantes de esta función de distancia mediante ablación: la distancia coseno (apartado 4.3.2), que sustituye la distancia euclidiana, y *Metric*, que, en lugar de emplear cualquier distancia geométrica fija, calcula el

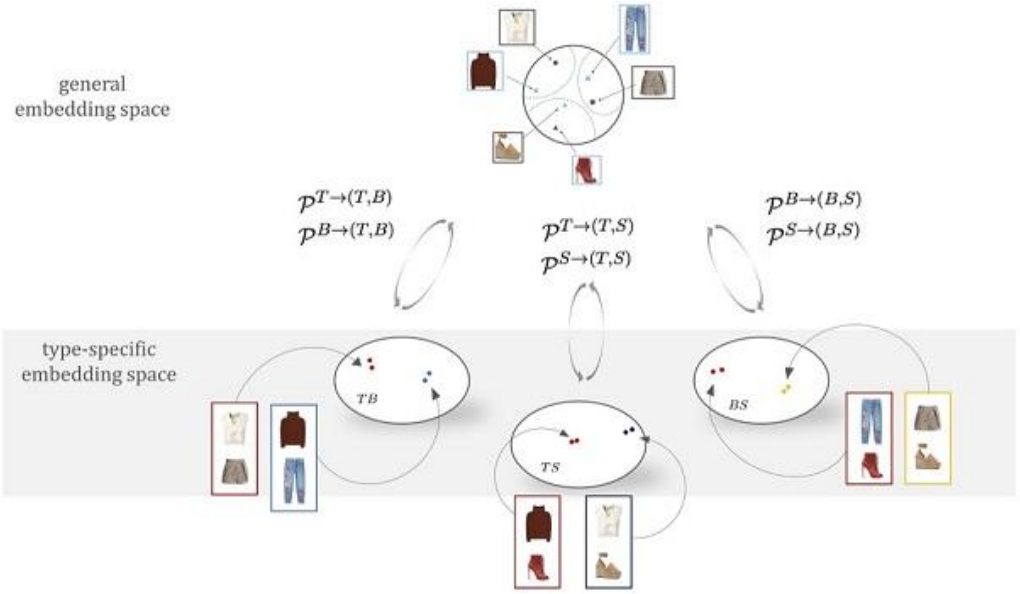


Figura 5.1: Arquitectura de embeddings condicionados por tipo (*type-aware*). A partir de un embedding general extraído por la red base, se aplican máscaras diagonales específicas para proyectar las prendas a subespacios de compatibilidad según el par de categorías. Figura adaptada de Vasileva et al. (2018).

producto de Hadamard entre los dos embeddings de tipo y lo pasa por una capa totalmente conectada cuyos pesos aprendidos actúan como una función de distancia generalizada aprendida.

Sobre esta base, el modelo incorpora dos pérdidas auxiliares entrenadas conjuntamente (*multi-task learning*). La pérdida **VSE** (*Visual-Semantic Embedding*) proyecta también la descripción textual (s) de cada ítem al embedding general, regularizándolo semánticamente mediante una pérdida de *ranking* que acerca la imagen (v) de un ítem a su propia descripción (s)y la aleja de descripciones de ítems no relacionados($\hat{v}, \hat{s}$). La pérdida **Sim** reutiliza estos mismos embeddings generales para una tarea adicional de similitud entre ítems de la misma categoría (por ejemplo, distinguir dos camisas parecidas entre sí), de modo que el embedding general no solo sirva de base para la compatibilidad entre tipos distintos, sino que también capture una noción de similitud intra-categoría útil por derecho propio.

$$\mathcal{L}_{\text{VSE}} = \sum_{\hat{s}} \max\left(0,\ m - f(v,s) + f(v,\hat{s})\right) + \sum_{\hat{v}} \max\left(0,\ m - f(v,s) + f(\hat{v},s)\right) \tag{5.5}$$

donde:

- f representa la función de similitud.
- m representa el margen.

- max funcion tipo ReLu que asegura que la perdida sea 0 si la distancia es menor a 0.

$$\mathcal{L}_{\text{Sim}} = \sum_{(x,x^+,x^-)_{t_i}} \max\left(0,\ D(x,x^+) - D(x,x^-) + m\right), \tag{5.6}$$

donde:

- D representa la funcion de distancia.

Finalmente, para mitigar el crecimiento cuadrático de los pares de categorías y asegurar la escalabilidad del método, los autores evalúan el compromiso entre especificidad y eficiencia paramétrica comparando dos variantes de asignación: CSN, T1:1, donde cada par de tipos dispone de una proyección diagonal exclusiva aprendida, y CSN, T4:1, donde una misma proyección se comparte aleatoriamente entre cuatro combinaciones de tipos distintas, reduciendo drásticamente la carga computacional a costa de cierta precisión semántica.

5.1.4. Aprendizaje de condiciones sin supervisión: SCE-Net

Para superar la dependencia de conocer las etiquetas de categoría en tiempo de test del paradigma *type-aware* Vasileva et al. (2018), lo cual limita la generalización a categorías no vistas, Tan et al. (2019) proponen **SCE-Net** (*Similarity Condition Embedding Network*), cuya arquitectura se ilustra en la figura 5.2.

La arquitectura se compone de dos módulos entrenados conjuntamente *end-to-end*. En primer lugar, un *backbone* ResNet-18 preentrenado en ImageNet proyecta cada imagen al **espacio de embedding general** $q(\mathbf{x};\theta)$, de dimensión D, punto de partida compartido con la subsección 5.1.3, aunque aquí sin ninguna proyección condicionada por tipo aplicada a continuación. En segundo lugar, el modelo mantiene un conjunto de M máscaras de condición de similitud $C_1, \ldots, C_M$, cada una de dimensión D (igual que el embedding general), cuyo valor se aprende durante el entrenamiento. El valor de M se determina experimentalmente mediante un conjunto de validación, no se fija de antemano por conocimiento del dominio.

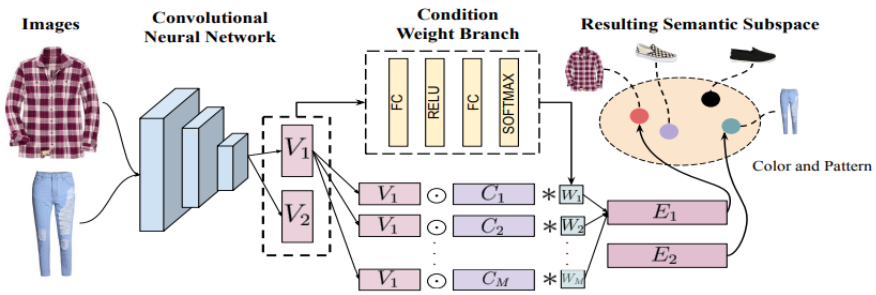


Figura 5.2: Arquitectura de SCE-Net (*Similarity Condition Embedding Network*). Figura adaptada de Tan et al. (2019).

Dada la característica visual general V_i de una imagen y una máscara C_j, la operación de enmascarado se define mediante producto de Hadamard:

$$E_{ij} = C_j \odot V_i, \tag{5.7}$$

proyectando así la imagen a M subespacios semánticos secundarios distintos — cada uno de los cuales, en principio, captura una noción de similitud diferente (por ejemplo, color, silueta, o tipo de cierre), aunque ninguna de estas nociones se especifica ni supervisa explícitamente durante el entrenamiento; emergen directamente de los datos. Se emplea la siguiente función de similitud:

$$d(x_1, x_2) = \sum_{k=1}^{M} W_k \cdot D\big(E_{1k}, E_{2k}\big), \tag{5.8}$$

donde:

- D representa la funcion de distancia euclideana.
- W_k representa los pesos de condición

El segundo componente central es la rama de peso de condición (*condition weight branch*), una red neuronal ligera que recibe como entrada las características visuales generales V_1 y V_2 del par de imágenes a comparar, y produce un vector de pesos $W_1, \ldots, W_M$ de dimensión M. Este vector realiza una asignación dinámica de la relevancia de cada máscara de condición para el par concreto que se está evaluando, donde una función de compuerta determinaba cuánto pesaba cada uno de los N "pseudo-espacios" para una consulta dada; la diferencia clave es que aquí el peso se calcula a partir de las propias características visuales del par, en lugar de depender de una categoría explícita.

La función de pérdida empleada en la architectura es:

$$\mathcal{L}_{\text{SCE}} = \sum_{(x, x^+, x^-)} \max\Big(0,\ d(x, x^+) - d(x, x^-) + m\Big), \tag{5.9}$$

El modelo se entrena de forma débilmente supervisada: durante el entrenamiento solo se sabe si un par de imágenes es similar bajo *alguna* condición no especificada, nunca bajo cuál en concreto. Los autores exploran además, en una sección de variantes, distintas modalidades de entrada para la rama de peso de condición (únicamente visual, o combinada con texto). Evaluado sobre *Polyvore Outfits*, *Maryland Polyvore* y *UT-Zappos50k*, SCE-Net supera al estado del arte en las tareas de *Fill-In-The-Blank*, predicción de compatibilidad de *outfit* y predicción de tripletes.

5.1.5. Modelado de subgrafos de outfits

Cui et al. (2019) plantean una limitación compartida por todos los enfoques anteriores de

este capítulo: tanto la comparación por pares como la representación en secuencia son incapaces de capturar de forma nativa relaciones de orden superior entre tres o más ítems del conjunto. Su propuesta, **NGNN** (*Node-wise Graph Neural Network*), representa el problema explícitamente como un grafo, donde cada nodo corresponde a una categoría de prenda y cada arista representa la interacción observada entre dos categorías que coexisten habitualmente en una combinación.

El modelo opera en tres etapas. En la primera, el estado inicial de cada nodo se obtiene proyectando la característica del ítem f_i (visual o textual) a un espacio de estilo latente de dimensión d, mediante una matriz de proyección lineal específica de categoría W_h^i:

$$r_i = W_h^i f_i. \tag{5.10}$$

En la segunda etapa (la aportación central del artículo), se modela la interacción entre nodos mediante paso de mensajes. Los autores descartan dos alternativas extremas: compartir una única matriz de transformación para todas las aristas (insuficiente para capturar interacciones específicas entre categorías concretas) y aprender una matriz de transformación distinta por cada arista (computacionalmente inviable, dado el gran número de aristas posibles en el grafo completo). La solución intermedia es asociar a cada nodo n_i dos matrices propias, una de salida W_{out}^i y otra de entrada W_{in}^i, de modo que la transformación de una arista concreta $n_i \to n_j$ se genera dinámicamente como el producto de ambas:

$$W_p^{n_i \to n_j} = W_{out}^i W_{in}^j, \tag{5.11}$$

lo que permite una interacción específica para cada par de nodos sin necesitar un parámetro independiente por arista. El mensaje agregado que recibe el nodo n_i en el paso temporal t se calcula sumando las contribuciones de todos sus vecinos, ponderadas por la matriz de adyacencia $\mathbf{A}$:

$$a_i^t = \sum_{n_j \to n_i \in \mathcal{E}} \mathbf{A}[n_j, n_i]\, W_{out}^j W_{in}^i\, h_j^{t-1} + b_p. \tag{5.12}$$

El estado del nodo se actualiza después mediante una puerta de tipo GRU:

$$z_i^t = \sigma\big(W_z a_i^t + U_z h_i^{t-1} + b_z\big), \tag{5.13}$$

donde z_i^t actúa como puerta de actualización que regula cuánta información nueva (a_i^t) se incorpora frente al estado anterior del nodo (h_i^{t-1}).

En la tercera etapa, tras un número fijo T de iteraciones de paso de mensajes, se dispone de un estado final h_i^T para cada nodo de categoría que participa en el subgrafo del *outfit*. Estos estados finales se agregan mediante un **mecanismo de atención** que pondera la contribución de cada ítem a la puntuación de compatibilidad global del conjunto, produciendo así una única puntuación escalar para el conjunto completo, en lugar de mantener una eva-

luación separada por cada par de nodos, resolviendo de este modo el mismo problema de agregación que en la representación por pares exigía calcular y combinar $\binom{k}{2}$ comparaciones independientes para un outfit de k ítems.

El modelo consta de un entrenamiento *end-to-end* en una única pasada siguiendo el principio de optimización aclarado por Gulati (2024): dado un *outfit* real positivo $s = \{v_1, v_2, \ldots, v_p, \ldots\}$, se genera una contrapartida negativa $s^- = \{v_1, v_2, \ldots, v_q, \ldots\}$ reemplazando aleatoriamente un ítem v_p por otro ítem $v_q \in \mathcal{V}$ del corpus. Ambos conjuntos se propagan de forma independiente a través de las etapas de la arquitectura para obtener sus respectivas puntuaciones de compatibilidad, x_s y x_{s^-}. Con el fin de forzar una puntuación consistentemente mayor para el conjunto real, adoptan una pérdida basada en *Bayesian Personalized Ranking* Rendle et al. (2012):

$$\mathcal{L}_{\text{bpr}} = \sum_{(s,s^-)\in\mathcal{P}_{\text{train}}} -\ln\sigma\left(x_s - x_{s^-}\right) \tag{5.14}$$

El objetivo final incorpora la retropropagación conjunta de gradientes sobre todos los parámetros del GNN (Θ, incluyendo las matrices de pesos W^i_{in}, W^i_{out} de cada nodo). Para la variante unimodal, la optimización añade una regularización L_2:

$$\mathcal{L}_1 = \mathcal{L}_{\text{bpr}} + \frac{\lambda}{2}\|\Theta\|^2 \tag{5.15}$$

Mientras que para la versión multimodal (visual y textual), la función de coste final se extiende integrando un término de consistencia entre modalidades ($\mathcal{L}_{\text{con}}$):

$$\mathcal{L}_2 = \mathcal{L}_{\text{bpr}} + \mathcal{L}_{\text{con}} + \frac{\lambda}{2}\|\Theta\|^2 \tag{5.16}$$

HGNN (*Hypergraph Neural Network*), propuesto por Li et al. (2022), extiende esta representación sustituyendo las aristas binarias del grafo simple por **hiperaristas** (figura 5.3): en lugar de descomponer un outfit de k ítems en $\binom{k}{2}$ interacciones por pares entre nodos de categoría, cada conjunto completo se representa como una única hiperarista que conecta simultáneamente todos los nodos de categoría involucrados, capturando de forma nativa la relación de conjunto sin necesidad de descomponerla. Gulati (2024) reimplementa ambas arquitecturas bajo idénticas condiciones experimentales, mostrando que HGNN supera de forma consistente a NGNN, a costa de un tiempo de entrenamiento sustancialmente mayor.

5.1.6. Fusión modular y atención sobre conjuntos: Recommendation Engine

Prato et al. (2020) nos introduce una arquitectura de dos etapas, entrenable *end-to-end*, diseñada explícitamente para escalar a un catálogo industrial completo.

Primera etapa: extracción y fusión multimodal por ítem. Para cada ítem del catálogo, la primera etapa extrae una representación tensorial considerando una entrada multimodal, ima-

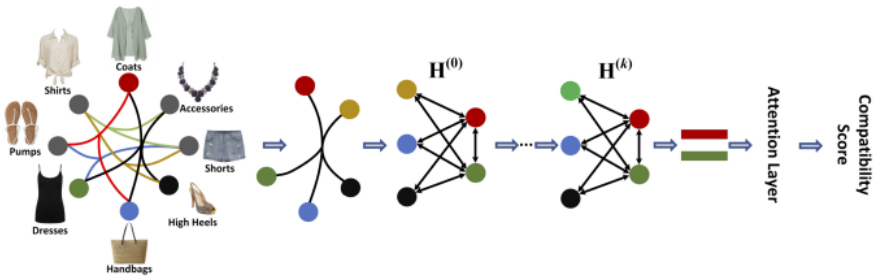


Figura 5.3: Arquitectura de HGNN (*Hypergraph Neural Network*). Figura adaptada de Gulati (2024).

gen, texto y valores categóricos, embebiendo la representación de cada modalidad en un mismo espacio común. El diseño es explícitamente **modular**: existe un módulo dedicado a cada tipo de entrada, de forma que la arquitectura puede expandirse fácilmente para acomodar nuevos tipos de entrada sin rediseñar el resto del sistema. Los tres módulos empleados son:

- una versión reducida de **ResNet-18** (sección 4.2.1), para procesar la imagen del ítem.
- un diccionario **GloVe** (sección 4.2.2) para procesar la descripción textual.
- una codificación categórica para la categoría del ítem.

La publicación, desgraciadamente, no especifica la operación matemática exacta mediante la cual se combinan las representaciones.

Segunda etapa: modelado de compatibilidad mediante Transformer. Sobre las representaciones fusionadas de todos los ítems de un conjunto, la segunda etapa emplea la arquitectura **Transformer** (Vaswani et al., 2023), y en concreto su mecanismo de *self-attention*, para aprender la relación de compatibilidad entre ellos.

Dado una combinación de ítems representada como una matriz $X \in \mathbb{R}^{n \times d}$ (con n el número de ítems del outfit y d la dimensión de la representación fusionada de cada ítem obtenida en la primera etapa), el mecanismo de atención proyecta X en tres matrices, consultas (Q), claves (K) y valores (V), mediante matrices de pesos aprendidas $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V. \tag{5.17}$$

La salida de la capa de atención se calcula entonces como:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \tag{5.18}$$

donde el producto $QK^\top \in \mathbb{R}^{n \times n}$ codifica, para cada par de ítems del *outfit*, un coeficiente de

afinidad que determina cuánto debe influir la representación de un ítem sobre la actualización de otro; el término $\sqrt{d_k}$ normaliza la magnitud de este producto para estabilizar el gradiente durante el entrenamiento, y la función *softmax* convierte estos coeficientes en pesos que suman uno por fila. En la práctica, esta operación se realiza en paralelo con h cabezas de atención (*multi-head attention*), cada una operando sobre una subproyección distinta de Q, K y V, permitiendo que el modelo capture simultáneamente varios tipos de relación entre ítems (por ejemplo, una cabeza podría especializarse en afinidad de color y otra en afinidad de silueta) antes de concatenar y proyectar sus salidas.

La elección de esta arquitectura se justifica al no incorporar el término de positional. Esto trata a los elementos de entrada como miembros de un conjunto no ordenado, en lugar de una secuencia con un orden fijo, evitando así el sesgo de orden artificial que discutido en la subsección 5.1.2. En segundo lugar, esto permite aprender la relación entre las características de todos los ítems simultáneamente, mediante una única operación matricial.

La trascendencia de este enfoque radica en que los autores no se limitan a aplicar un mecanismo de atención, sino que reformulan metodológicamente el problema de la recomendación de moda al establecer una equivalencia exacta con los dos pilares del aprendizaje auto-supervisado que consagraron a las arquitecturas *Transformers* en el procesamiento del lenguaje natural (NLP). En lugar de tratar las tareas de evaluación como heurísticas aisladas, demuestran que el modelado de un conjunto puede unificarse bajo el mismo marco teórico que el entendimiento sintáctico y semántico de un idioma:

- **De *Masked Language Modeling* a *Fill-In-The-Blank*:** En NLP, MLM fuerza al modelo a inferir la representación de una palabra oculta basándose en el contexto bidireccional de la oración. Los autores trasladan esta mecánica de manera idéntica a FITB, donde el sistema debe predecir la prenda faltante evaluando las interacciones atencionales cruzadas con el resto de los componentes del conjunto. Esto transforma la recomendación en una tarea de completado de contexto de alta dimensionalidad sobre todo el catálogo.

- **De *Next Sentence Prediction* a *Compatibility Estimation*:** Mientras que NSP entrena a la red para entender la coherencia y continuidad entre dos estructuras lingüísticas completas, la estimación de compatibilidad se reformula aquí como la validación de la consistencia semántica global de una secuencia visual. El modelo aprende a discriminar si el conjunto de prendas comparte un *concepto estilístico latente* subyacente, emulando la evaluación del flujo lógico de un texto.

5.1.7. Síntesis evolutiva y comparación de arquitecturas

El recorrido cronológico de las subsecciones anteriores puede leerse, en su conjunto, como una sucesión de soluciones parciales: cada arquitectura resuelve una limitación concreta

de la anterior, pero introduce a cambio una nueva restricción que la siguiente generación de trabajos se encarga de abordar. La tabla 5.1 sintetiza los cinco paradigmas revisados y compara la evolución de los resultados alcanzados.

Método	**AUC (Compatibility)**	**Accuracy (FITB)**
Polyvore Outfits / Polyvore-68k		
Bi-LSTM	0.65	37.9 %
Type-Aware Embedding Network	0.86	56.2 %
SCE-Net	0.91	61.6 %
Recommendation Engine	0.91	62.5 %
Subconjunto Polyvore (15 %)		
Aleatorio	0.51	24 %
NGNN (InceptionV3)	0.65	38 %
HGNN (InceptionV3)	0.76	39 %
NGNN (ViT)	0.68	40 %
HGNN (ViT)	0.77	40 %

Tabla 5.1: Cifras de AUC (Compatibility Prediction) y accuracy (Fill-In-The-Blank) reportadas por cada arquitectura, agrupadas por benchmark de evaluación. Las cifras solo son directamente comparables *dentro* de cada bloque, dado que cada uno emplea un conjunto de datos o split diferente.

5.2. Integración de lenguaje natural en la recomendación de moda: *Text-to-Outfit*

La evolución cronológica descrita en el apartado anterior revela una constante en todas las arquitecturas analizadas: desde la representación por pares hasta el modelado de subgrafos, todas ellas asumen un sesgo de entrada puramente visual, donde el usuario debe aportar necesariamente al menos una prenda de partida sobre la que el sistema completa, sustituye o evalúa la compatibilidad del resto del conjunto. Esta restricción condiciona de forma severa la naturaleza de la interacción, limitándola a un escenario de *cold-start* inverso en el que el usuario ya posee una idea visual concreta.

5.2.1. Búsqueda de prenda individual guiada por texto

TIRG: composición de imagen y texto mediante *gating* residual

Los primeros trabajos que incorporaron lenguaje natural en la recuperación de moda se limitaron a la prenda individual. El trabajo seminal que formalizó esta tarea, denominada *Composed Image Retrieval* (sección 4.4), fue el modelo **TIRG** (*Text Image Residual Gating*) de Vo et al. (2018). Dada una imagen de referencia $\phi_x \in \mathbb{R}^{W\times H\times C}$ extraída mediante una CNN convolucional y una instrucción textual de modificación $\phi_t \in \mathbb{R}^C$ codificada mediante una LSTM y posteriormente difundida espacialmente para igualar las dimensiones de ϕ_x, TIRG combina ambas modalidades a través de dos mecanismos complementarios:

$$f_{\text{gate}} = \sigma\left(\phi_x * W_{g1} + \phi_t * W_{g2}\right) \odot \phi_x, \qquad f_{\text{res}} = \text{ReLU}\left(\phi_x * W_{r1} + \phi_t\right) \tag{5.19}$$

donde:

1. f_{gate} actúa como un **selector de características**, preservando o suprimiendo dimensiones visuales de la imagen de referencia en función del contenido semántico del texto.

2. f_{res} inyecta el término **residual**, incorporando explícitamente la modificación solicitada que no está presente en ϕ_x.

Ambos términos se combinan mediante una suma ponderada por un parámetro aprendido, lo que permite al modelo equilibrar de forma adaptativa cuánta información retener de la imagen original frente a cuánta incorporar del texto. Su validación en Fashion200k (sección 4.6) mostró que la combinación multimodal (R@1 $= 14{,}1\,\%$) supera ampliamente a las variantes unimodales (imagen: $3{,}5\,\%$; texto: $1{,}0\,\%$), evidenciando que la interacción entre modalidades concentra el grueso de la señal discriminativa.

CLIP y FashionCLIP: del *gating* explícito al espacio compartido

De forma paralela, y con una filosofía radicalmente distinta, CLIP (subsección 4.2.3) reemplazó las funciones de composición explícitas, como el *gating* de TIRG, por el aprendizaje de un espacio de representación conjunto en el que imagen y texto son directamente comparables mediante similitud coseno, sin necesidad de un módulo de fusión intermedio. Dado un lote de N pares imagen-texto (I_i, T_i), CLIP optimiza simultáneamente un codificador visual y uno textual mediante la función de pérdida simétrica InfoNCE:

$$\mathcal{L} = -\frac{1}{2N}\sum_{i=1}^{N}\left(\log\frac{\exp(\text{sim}(I_i,T_i)/\tau)}{\sum_{j=1}^{N}\exp(\text{sim}(I_i,T_j)/\tau)} + \log\frac{\exp(\text{sim}(I_i,T_i)/\tau)}{\sum_{j=1}^{N}\exp(\text{sim}(I_j,T_i)/\tau)}\right) \tag{5.20}$$

donde $\text{sim}(\cdot,\cdot)$ representa la similitud coseno entre los *embeddings* proyectados y τ es un parámetro de temperatura aprendido que controla la concentración de la distribución de similitudes. El primer término de la suma fuerza a que, para cada imagen, su texto correspondiente sea más similar que el resto de textos del lote (dirección imagen→texto); el segundo término impone la condición simétrica inversa (texto→imagen). Entrenado sobre cientos de millones de pares imagen-texto recolectados de la web, este objetivo dota a CLIP de una notable capacidad de generalización *zero-shot*, sin necesidad de reentrenamiento para dominios específicos.

Esta capacidad general fue posteriormente especializada para el comercio electrónico de moda mediante **FashionCLIP** (Chia et al. (2022)), que realiza un *fine-tuning* de CLIP sobre catálogos de moda con el objetivo de mitigar la brecha de dominio entre las imágenes genéricas de internet y las fotografías de producto (fondos neutros, poses estandarizadas, terminología especializada). El modelo proyecta las características visuales (E_v) y textuales (E_t) en vectores unitarios normalizados, formalizando la puntuación de afinidad semántica $S(I,T)$ como:

$$S(I,T) = \left\langle \frac{E_v(I)}{\|E_v(I)\|_2}, \frac{E_t(T)}{\|E_t(T)\|_2} \right\rangle = \frac{E_v(I)^T E_t(T)}{\|E_v(I)\|_2 \|E_t(T)\|_2} \tag{5.21}$$

5.2.2. Búsqueda de outfits completos guiada por texto: CLIP4Outfit

El modelo **CLIP4Outfit** (Jang et al. (2023)) constituye la extensión natural de estas capacidades al conjunto completo, y representa el trabajo más directamente alineado con el enfoque de este TFM. A diferencia de los modelos revisados en la subsección 5.2.1, adopta el lenguaje natural libre como **única** modalidad de entrada y lo aplica directamente sobre el conjunto de prendas como una estructura no ordenada, eliminando por completo la necesidad de que el usuario aporte una prenda de partida Jang et al. (2023) formalizan este problema bajo la tarea de recuperación *Text-to-Outfit* (sección 4.4)

Como se ilustra en el esquema general de la figura 5.4, CLIP4Outfit estructura la relación entre el texto $T = \{W_j\}_{j=1}^{N_t}$ y el *outfit* $O = \{I_i\}_{i=1}^{N_o}$ en tres niveles semánticos progresivos, inspirados en la idea de que la afinidad entre una descripción y un conjunto de prendas no puede evaluarse de forma satisfactoria en un único nivel de granularidad (sección 4.1).

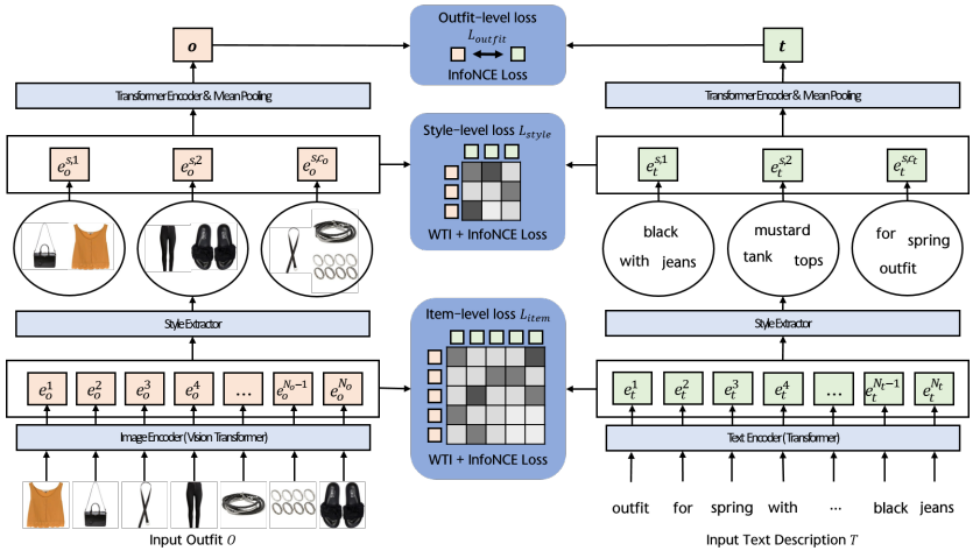


Figura 5.4: Ilustración general de la arquitectura de CLIP4Outfit. Figura adaptada de Jang et al. (2023).

1. Nivel Ítem. Se extraen los *embeddings* del texto mediante BERT y de las prendas mediante un *Vision Transformer*. Sobre ambos se aplica el módulo *Weighted Tokenwise Interaction* (WTI), que calcula la similitud cruzada ponderando la contribución de cada token y cada imagen mediante módulos de importancia aprendidos: la pérdida asociada, $\mathcal{L}_{\text{item}}$, se obtiene aplicando la función contrastiva InfoNCE de forms análoga a la empleada en CLIP.

$$S_{O,T} = \frac{1}{1+p}\left(\sum_{i=1}^{N_o} f_o(e_o)_i \max_{j=1}^{N_t} \frac{(e_t^j)^T e_o^i}{\|e_t^j\|_2 \|e_o^i\|_2} + p \sum_{j=1}^{N_t} f_t(e_t)_j \max_{i=1}^{N_o} \frac{(e_t^j)^T e_o^i}{\|e_t^j\|_2 \|e_o^i\|_2}\right) \tag{5.22}$$

2. Nivel Estilo. Dado que el estilo es una propiedad emergente del conjunto y no se corresponde con elementos aislados, e_o y e_t se procesan mediante dos *Transformer Encoders* independientes y se agrupan mediante K-means en centroides de estilo (e_o^s y e_t^s). Para evitar la inestabilidad asociada a la inicialización aleatoria de K-means en conjuntos reducidos problema especialmente relevante dado que una combinación típica contiene un número reducido de prendas, se emplea una selección *greedy* previa que maximiza la distancia mínima entre centroides, favoreciendo una cobertura más representativa del espacio semántico. Sobre estos centroides se calcula nuevamente la similitud WTI, dando lugar a la pérdida $\mathcal{L}_{\text{style}}$.

$$e'_o = E^o_{trans}(e_o), \qquad e'_t = E^t_{trans}(e_t) \tag{5.23}$$

3. Nivel Conjunto. Finalmente, los tokens de estilo se procesan mediante capas *Transformer* adicionales y se agregan mediante *mean pooling* para obtener representaciones globales y unificadas del *outfit* (o) y del texto (t). Su afinidad se calcula directamente mediante similitud coseno, derivando en la pérdida $\mathcal{L}_{\text{outfit}}$. Este último nivel actúa como una señal de supervisión de grano grueso que complementa, sin sustituir la granularidad fina aportada por los niveles *ítem* y *estilo*.

$$\mathbf{o} = \text{MeanPool}\left(F^o_{trans}(e^s_o)\right), \qquad \mathbf{t} = \text{MeanPool}\left(F^t_{trans}(e^s_t)\right) \tag{5.24}$$

$$S^o_{\mathbf{o},\mathbf{t}} = \frac{\mathbf{o}^T\mathbf{t}}{\|\mathbf{o}\|_2\|\mathbf{t}\|_2} \tag{5.25}$$

Función de pérdida total. Las tres componentes se combinan en un único objetivo de entrenamiento multitarea:

$$\mathcal{L} = \mathcal{L}_{\text{item}} + \alpha\mathcal{L}_{\text{style}} + \beta\mathcal{L}_{\text{outfit}} \tag{5.26}$$

donde α y β son hiperparámetros de escala, que evitan que las señales de nivel superior (*estilo* y *conjunto*) de gradiente potencialmente más ruidoso al derivar de representaciones agregadas, dominen sobre la señal más directa del nivel ítem.

El modelo se evaluó sobre las variantes del conjunto de datos Polyvore (*Maryland Polyvore* Han et al. (2017b) y *Polyvore Outfits* Vasileva et al. (2018)), el mismo ecosistema de datos que articula transversalmente buena parte de los trabajos revisados en la sección 5.1. CLIP4Outfit superó a todos los *baselines* en las métricas de precisión (Recall@k) consideradas. Además, la evaluación cuantitativa se respaldó mediante un estudio de usuarios, en el que la mayoría de los evaluadores respaldó la relevancia semántica de las combinaciones recuperadas, aportando así una validación cualitativa complementaria a las métricas automáticas.

5.2.3. Perspectiva futura: de la recuperación a la generación mediante difusión

Todos los enfoques de recuperación analizados en los apartados anteriores, se encuentran estrictamente acotados por la disponibilidad del catálogo existente, siendo incapaces de proponer combinaciones o prendas inéditas que respondan a consultas no representadas en los datos de entrenamiento o en el índice de recuperación. Para superar esta barrera, la literatura científica más reciente está evolucionando hacia modelos de **generación condicional** basados en difusión latente.

Generative Outfit Recommendation: el modelo DiFashion

Xu et al. (2024) formalizan este nuevo marco bajo la tarea de *Generative Outfit Recommendation* (GOR) con el modelo **DiFashion**. En lugar de buscar en un catálogo cerrado, el sistema sintetiza en paralelo imágenes de prendas inéditas, garantizando su compatibilidad mutua mediante tres niveles de condicionamiento:

1. ***Category prompt*:** Especifica el tipo o clase de prenda a generar (p. ej., pantalón, chaqueta), actuando como restricción estructural sobre la salida del modelo de difusión.
2. **Condición mutua de coherencia:** Garantiza la armonía visual y de estilo entre las prendas generadas simultáneamente, de forma análoga a como los modelos de la sección 5.1 evaluaban la compatibilidad entre prendas existentes, pero aplicada ahora sobre elementos que aún no existen.
3. **Condición de historial:** Reincorpora las preferencias pasadas del usuario como señal de personalización, conectando así el paradigma generativo con los sistemas de recomendación clásicos basados en historial de interacción.

El proceso de generación se refuerza mediante *Classifier-Free Guidance* (CFG), técnica que permite modular la fidelidad de la síntesis respecto a las condiciones impuestas sin necesidad de un clasificador externo adicional. DiFashion ilustra cómo el paradigma generativo no sustituye necesariamente a las tareas de recuperación revisadas en la subsección 5.2.1 y 5.2.2, sino que puede **reformularlas bajo una aproximación sintética**, abriendo la puerta a sistemas híbridos que combinen ambos paradigmas según la disponibilidad de producto en catálogo.

Alineación con preferencias humanas: FashionDPO

Esta línea de investigación ha evolucionado muy recientemente mediante la integración de técnicas de **alineación con preferencias humanas**, en paralelo a la tendencia observada en el campo de los grandes modelos de lenguaje. Yu et al. (2025) introducen **FashionDPO**, un marco de *fine-tuning* basado en *Direct Preference Optimization* (DPO) que evalúa las generaciones bajo tres dimensiones principales:

- **Calidad visual** de la imagen sintetizada, evitando artefactos propios de los modelos de difusión.
- **Compatibilidad estética** entre las prendas del conjunto generado.
- **Grado de personalización** respecto al perfil e historial del usuario.

Frente al ajuste mediante recompensas explícitas, DPO permite optimizar directamente a partir de comparaciones por pares entre generaciones preferidas y no preferidas, evitando la inestabilidad de entrenamiento asociada a los métodos de aprendizaje por refuerzo clásicos

(como RLHF) y simplificando así el proceso de alineación en un dominio, el de la moda, donde recopilar juicios humanos comparativos resulta más viable que definir una función de recompensa explícita y universal. Ambos trabajos (DiFashion y FashionDPO) son evaluados sobre los *datasets iFashion* y *Polyvore-U*.

Metodología

6

6.1. Marco General de la Investigación y Formulación de Hipótesis

La presente investigación adopta un diseño experimental y comparativo fundamentado en la replicación y contraste de arquitecturas de referencia. Tras analizar la evolución de la recomendación de moda en la última década, se seleccionan los paradigmas más representativos para ser evaluados bajo un marco de experimentación homogéneo. Se prioriza una implementación fiel a los diseños originales con el fin de validar de forma estricta los resultados reportados en la literatura, evitando sesgos derivados de reimplementaciones libres y garantizando la reproducibilidad del análisis.

Con el fin de estructurar el proceso experimental, se formulan las siguientes preguntas de investigación e hipótesis asociadas:

RQ1: ¿Son los modelos implementados capaces de capturar la naturaleza no transitiva de la compatibilidad?

Hipótesis: Aquellas arquitecturas que proyectan las prendas en espacios de representación específicos según su categoría o tipo (frente a un único espacio de *embedding* global) modelarán de forma más precisa esta propiedad, al mitigar la propagación geométrica de la similitud a lo largo del espacio latente.

RQ2: ¿Qué arquitectura ofrece una mayor capacidad de transferencia y generalización ante datos del mundo real?

Hipótesis: Dado que el modelo de Prato et al. (2020) es el único evaluado bajo este escenario, se busca extender dicha validación al resto de propuestas. Se postula que los modelos optimizados mediante funciones de pérdida basadas en tripletas exhibirán una mejor generalización en entornos reales, debido al aprendizaje discriminativo fino inducido durante su entrenamiento.

RQ3: ¿Cuál es el grado de correlación entre las métricas de evaluación automática y la percepción estética humana?

Hipótesis: Existirá una correlación positiva pero moderada entre el rendimiento cuantitativo (exactitud en FITB, AUC) y la valoración cualitativa humanos, evidenciando que las métricas *offline* acercan, pero no siempre garantizan en su totalidad una correspondencia con el criterio estético real.

Adicionalmente, se plantea como línea exploratoria evaluar la versatilidad de los *embeddings* multimodales resultantes en tareas no contempladas en su diseño original, tales como la generación de combinaciones guiada por texto. Esta exploración busca verificar la riqueza semántica de las representaciones aprendidas y su potencial de transferencia hacia paradigmas de recomendación más complejos.

6.2. Estrategia de Selección y Diseño de Arquitecturas

Tras la revisión realizada en el capítulo de estado del arte, se han identificado distintas familias de arquitecturas para abordar el problema de la compatibilidad de moda, desde los primeros enfoques basados en embeddings condicionados por tipo, pasando por representaciones basadas en grafos e hipergrafos, hasta arquitecturas orientadas a su aplicación en entornos de producción. De entre estas familias, se han seleccionado tres modelos para su implementación y comparación en este trabajo: SCE-Net (Tan et al. (2019)), HGNN (Li et al. (2022)) y el Recommendation Engine propuesto por Prato et al. (2020).

La selección de estos tres modelos, y no de otros también revisados en el estado del arte, responde a los siguientes criterios:

- **Compatibilidad con el conjunto de datos seleccionado**: Los tres modelos han sido diseñados o evaluados originalmente sobre el dataset Polyvore Outfits (o sobre subconjuntos derivados del mismo, como en el caso de Gulati), lo que permite una comparación directa entre ellos bajo condiciones homogéneas de datos, sin necesidad de adaptaciones que introduzcan variables adicionales no controladas.
- **Reproducibilidad y fidelidad al artículo original**: Con descripciones suficientemente detalladas en sus respectivos artículos como para permitir una implementación fiel, lo cual resulta imprescindible dado el componente de replicación que caracteriza el enfoque metodológico de este TFM.
- **Relevancia directa para las preguntas de investigación planteadas**: La elección de estos tres modelos concretos no es solo representativa a nivel general, sino que responde específicamente a las preguntas formuladas en el enfoque general de este trabajo. En particular, el Recommendation Engine se selecciona por ser, entre toda la literatura revisada, el único modelo evaluado tras su entrenamiento sobre un conjunto de datos de naturaleza más próxima a un entorno real (el dataset industrial de H&M), lo cual lo convierte en la referencia obligada para la segunda pregunta de investigación relativa al comportamiento de los modelos en este tipo de escenarios.

A continuación se detalla brevemente el rol que cada modelo desempeña dentro del diseño experimental de este trabajo:

SCE-Net Se selecciona por representar la evolución natural de los enfoques basados en embeddings condicionados por tipo, al eliminar la necesidad de supervisión explícita sobre la condición de similitud en tiempo de test. Su inclusión permite valorar si la capacidad de aprender dichas condiciones de forma latente resulta suficiente para capturar relaciones de compatibilidad complejas, en comparación con arquitecturas que modelan estas relaciones de forma estructural.

HGNN Se selecciona como representante del paradigma basado en grafos, y en concreto en hipergrafos, frente al paradigma de embeddings puro de SCE-Net. Su estructura permite modelar de forma explícita las relaciones de orden superior entre las distintas categorías de un outfit, lo que resulta especialmente relevante para contrastar la primera pregunta de investigación sobre la naturaleza no transitiva de la compatibilidad.

Recommendation Engine Se selecciona por su arquitectura basada en mecanismos de atención, que procesa la combinación de ítems de un conjunto de forma simultánea en lugar de mediante comparaciones exhaustivas por pares, así como por ser el único trabajo revisado que aporta una evaluación en condiciones próximas a un entorno de producción real, incluyendo una validación cualitativa mediante estilistas profesionales. Ambos aspectos lo convierten en una pieza clave para responder a la segunda y tercera preguntas de investigación de este trabajo

Quedan fuera del alcance de este trabajo otras arquitecturas revisadas en el estado del arte, como el modelo Type-Aware Embeddings o NGNN, al considerarse que los modelos seleccionados cubren de forma suficiente la diversidad de paradigmas necesaria para responder a las preguntas de investigación planteadas, dentro de los límites de tiempo y recursos computacionales disponibles para este TFM.

6.3. Protocolo de Evaluación y Métricas Estándar

La evaluación cuantitativa de los modelos seleccionados se articula en torno a tres tareas complementarias. Las tareas *Fill-In-The-Blank* (FITB) y *Compatibility Prediction* (CP) utilizan las particiones de prueba estandarizadas del dataset Polyvore Outfits. Por su parte, la tarea *Text-to-Outfit* se formula como un marco de evaluación de recuperación multimodal para medir la capacidad de generalización del espacio latente ante consultas en lenguaje natural.

Fill-In-The-Blank

- **Protocolo:** Dado un *outfit* válido de N prendas al que se le retira una posición p, se obtiene un conjunto parcial $O \setminus p = \{i_1, \ldots, i_{p-1}, i_{p+1}, \ldots, i_N\}$. Se evalúa al modelo mediante un conjunto de $M = 4$ candidatos $C = \{c_1, c_2, \ldots, c_M\}$, donde solo $c \in C$

(la prenda original) es correcta. El modelo debe seleccionar la opción que maximice la compatibilidad con el conjunto parcial.

- **Control de Sesgos en Distractores:** Los tres distractores incorrectos pertenecen obligatoriamente a la misma categoría semántica que c. Esta restricción evita que el modelo utilice la categoría como un atajo trivial (p. ej., descartar un calzado cuando se busca una camiseta) y le obliga a discriminar en función del estilo, color y textura.
- **Formulación de la Métrica:** La evaluación se realiza mediante Exactitud (*Accuracy*):

$$\text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}(\hat{y}_i = y_i) \tag{6.1}$$

donde $\hat{y}_i$ es la elección del modelo, y_i el candidato correcto y $\mathbb{I}(\cdot)$ la función indicadora. Al ser una tarea de elección discreta con respuesta única sobre $M = 4$ opciones, la *Accuracy* permite una comparación directa frente a una línea base aleatoria del $25\,\%$.

Compatibility Prediction

- **Protocolo:** Esta tarea se plantea como una clasificación binaria. Dado un conjunto completo $O = \{i_1, i_2, \ldots, i_k\}$, el modelo emite una puntuación continua de compatibilidad $s(O) \in \mathbb{R}$.
- **Generación de Ejemplos Negativos:** Para cada conjunto real del test, Polyvore Outfits genera un *outfit* negativo sustituyendo sus prendas por otras extraídas aleatoriamente de la misma categoría semántica, evaluando así la coherencia armónica y no la estructura del conjunto.
- **Formulación de la Métrica:** Se emplea el Área Bajo la Curva ROC (AUC-ROC):

$$\text{AUC} = \frac{1}{|P||N|}\sum_{O^+ \in P}\sum_{O^- \in N} \mathbb{I}(s(O^+) > s(O^-)) \tag{6.2}$$

donde P y N son los conjuntos de *outfits* positivos y negativos. El AUC-ROC es la métrica idónea dado que evalúa la capacidad de ordenación discriminativa de forma invariante al umbral. Esto permite comparar equitativamente arquitecturas con salidas heterogéneas (p. ej., distancias en espacios latentes en SCE-Net frente a *logits* de clasificación en el Recommendation Engine) sin necesidad de calibración previa.

Text-to-Outfit Recommendation

- **Protocolo:** Dado un texto de consulta T_q (asociado conceptualmente a un *outfit* O_q), el modelo calcula la similitud entre el *embedding* de T_q y los *embeddings* de los *outfits* candidatos del test, generando una lista ordenada por similitud descendente. O_q actúa como el único elemento relevante, actuando el resto del corpus de prueba como negativos implícitos.

- **Métricas de Recuperación:** Se emplean dos métricas estándar de *ranking* calculadas sobre el conjunto de consultas Q:

    1. **Recall@K:** Mide la proporción de consultas donde el conjunto correcto aparece en las primeras K posiciones:

$$\text{Recall@K} = \frac{1}{|Q|} \sum_{q \in Q} \mathbb{I}(\text{rank}(O_q) \leq K) \tag{6.3}$$

    2. **Mean Reciprocal Rank (MRR):** Evalúa la precisión exacta de la posición devuelta evaluando el inverso del rango del elemento correcto:

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}(O_q)} \tag{6.4}$$

Mientras que Recall@K valida la utilidad del sistema en una ventana de atención acotada para e-commerce, MRR penaliza continuamente la caída de posiciones del elemento relevante en todo el *ranking*.

Marco de Comparabilidad y Reproducibilidad de Datos

Las pruebas cuantitativas descritas se llevan a cabo sobre la partición *nondisjoint* (con solapamiento de prendas entre entrenamiento y test) provista por Polyvore Outfits. Adoptar este protocolo estándar es una decisión metodológica clave para garantizar la comparabilidad directa de los resultados obtenidos con las métricas reportadas en la literatura científica de referencia para los modelos *baseline*.

6.4. Diseño Experimental para la Validez de Transferencia

Motivación: la brecha entre benchmark y entorno real

Los resultados reportados habitualmente en la literatura y los evaluados en este trabajo bajo el protocolo estandarizado, se calculan sobre la partición de prueba de Polyvore Outfits. Este conjunto comparte distribución con el corpus de entrenamiento: mismo origen, fotografía de producto sobre fondo neutro con poses estandarizadas y composiciones generadas bajo hábitos de anotación homogéneos. Un rendimiento elevado en este entorno controlado no garantiza una capacidad de generalización equivalente en escenarios no controlados.

Esta discrepancia responde al fenómeno de *domain shift* o *distribution shift*: la degradación del desempeño del modelo al evaluarse sobre datos que provienen de una distribución distinta a la de entrenamiento. En recomendación de moda, este fenómeno resulta severo debido a las variaciones en iluminación, la política de categorización de cada plataforma e-commerce, la calidad de imagen y la heterogeneidad de los catálogos comerciales. Por ello,

el marco metodológico contempla no solo la replicación en el *benchmark* estándar, sino la estimación de la transferencia *out-of-domain*.

Planteamiento: evaluación de transferencia zero-shot

Para responder a la pregunta de investigación sobre la validez de transferencia (RQ2), se diseña un protocolo de validación cruzada entre dominios articulado en dos escenarios:

- **Escenario A (Evaluación en dominio estándar):** Los modelos se entrenan y evalúan sobre las particiones de Polyvore Outfits, constituyendo la referencia de fidelidad y reproducibilidad frente a las cifras del estado del arte.
- **Escenario B (Evaluación de transferencia en entorno real):** Los modelos, entrenados sobre Polyvore Outfits y sin ningún ajuste o *fine-tuning* adicional (evaluación *zero-shot*), se aplican sobre un corpus independiente construido a partir de catálogos comerciales reales. Al mantener los pesos congelados, cualquier degradación en el rendimiento respecto al Escenario A se atribuye de forma aislada a la capacidad de generalización de la arquitectura ante distribuciones no vistas.

Hipótesis asociada: Se postula que las arquitecturas optimizadas mediante funciones de pérdida basadas en tripletas exhiben una mejor generalización en el Escenario B, al fundamentarse en un aprendizaje discriminativo fino entre pares de ítems, a diferencia de modelos cuya señal de compatibilidad depende fuertemente de la topología categórica observada durante el entrenamiento (como HGNN).

Protocolo de medición

El Escenario B reutiliza las mismas tareas y métricas definidas para el entorno estándar (FITB y *Compatibility Prediction*), adaptando el origen de los datos. Esto requiere replicar sobre el corpus comercial la generación de preguntas FITB (con distractores restringidos a la misma categoría semántica) y la construcción de *outfits* negativos. La comparación directa entre las métricas obtenidas en ambos escenarios para una misma arquitectura permite cuantificar formalmente la magnitud de la caída de rendimiento atribuible al *domain shift*.

Desarrollo del Proyecto

7

En este capítulo se detalla el proceso de ingeniería de software y ciencia de datos llevado a cabo para operacionalizar, entrenar y validar los modelos de recomendación y compatibilidad de moda seleccionados. Tras haber establecido el marco teórico en el capítulo previo, el presente bloque traslada las formulaciones abstractas a un pipeline de ejecución práctico, documentando tanto las decisiones de implementación como las barreras técnicas encontradas durante el ciclo de vida del proyecto.

El capítulo se articula de la siguiente manera: en primer lugar, la sección 7.1 describe la infraestructura hardware, el entorno de ejecución y el stack tecnológico utilizado para soportar el cómputo. A continuación, la sección 7.2 aborda el procesamiento del benchmark Polyvore Outfits y analiza los desafíos técnicos vinculados a la adquisición de catálogos comerciales. Posteriormente, la sección 7.3 expone la implementación modular de las arquitecturas baseline (SCE-Net, HGNN y Recommendation Engine), seguida de la sección 7.4, donde se detalla el desarrollo del motor de recomendación Text-to-Outfit. Finalmente, la sección 7.5 profundiza en la estrategia de entrenamiento, el ajuste de hiperparámetros y las curvas de convergencia de cada modelo.

7.1. Arquitectura del Sistema y Entorno de Ejecución

Para garantizar la reproducibilidad técnica y la trazabilidad de los experimentos, la fase de desarrollo e implementación de las tres arquitecturas evaluadas se soporta sobre una infraestructura de soporte dedicada y un conjunto de herramientas de software de estado del arte en aprendizaje profundo.

7.1.1. Infraestructura Hardware y Estrategias de Optimización de Memoria

El entrenamiento y la evaluación de los modelos se han llevado a cabo en un entorno local utilizando una GPU NVIDIA GeForce RTX 3050 Laptop con 6 GB de VRAM. Esta capacidad de cómputo contrasta significativamente con la literatura de referencia; por ejemplo, Gulati (2024) emplea GPUs de gama profesional NVIDIA RTX 6000. Esta restricción de hardware, que se profundiza en el 7.3, imposibilitó realizar una búsqueda sistemática de hiperparámetros, adoptando en su lugar las configuraciones predeterminadas sobre los trabajos originales.

Para mitigar esta limitación de memoria sin comprometer la fidelidad de los experimentos, se aplicaron dos técnicas de optimización de forma selectiva según los requerimientos de cada arquitectura:

- **Acumulación de Gradientes (*Gradient Accumulation*):** Permite simular un tamaño de lote efectivo $B_{ef} = n \times b$ equivalente al de la literatura original sin sobrecargar la VRAM. En lugar de actualizar los pesos tras cada micro-lote de tamaño b, los gradientes calculados sobre n pasadas consecutivas se acumulan en memoria, ejecutando la optimización únicamente en el n-ésimo paso. Esta técnica resulta clave en modelos con tamaños de lote nativos superiores a la capacidad del hardware local.

- **Entrenamiento en Precisión Mixta Automática (*Automatic Mixed Precision*, AMP):** Reducir la precisión de 32 a 16 bits en operaciones no críticas disminuye notablemente la huella de memoria y acelera el cómputo. Para garantizar la estabilidad numérica, las operaciones sensibles; en particular, el cálculo de la función de pérdida, se ejecutan explícitamente en precisión completa. Esta técnica beneficia especialmente a las arquitecturas que realizan un *fine-tuning* directo sobre el *backbone* visual convolucional.

Ambas estrategias se ajustaron al cuello de botella específico de cada modelo (tamaño de lote, propagación a través de extractores visuales o la combinación de ambos). Los detalles de implementación concretos para cada arquitectura se especifican en el apartado 7.3.

7.1.2. Pipeline Tecnológico y Stack de Software

La implementación del sistema se ha unificado bajo el *framework* de aprendizaje profundo PyTorch, empleando su extensión `torchvision` para las transformaciones de preprocesamiento y la carga de extractores de características visuales preentrenados (*ResNet18* en SCE-Net y como núcleo de *SmallResNet18* en el Recommendation Engine). Esta centralización responde a la compatibilidad con el entorno de aceleración por GPU y a la necesidad de mantener un código fuente homogéneo. Dicha arquitectura modular facilita la reutilización de componentes transversales (bucles de entrenamiento con AMP y acumulación de gradientes, *DataLoaders* de carga perezosa y rutinas de *checkpointing*), armonizando las distintas implementaciones independientemente del *framework* empleado en la literatura original.

Sobre este núcleo común, se incorporan librerías especializadas según los requerimientos sintácticos y funcionales de cada modelo:

- **Cómputo Científico y Estructuras Dispersas:** HGNN utiliza SciPy (`scipy.sparse`) para la construcción de la matriz de incidencia del hipergrafo y su posterior laplaciana normalizada (Θ). Dado que esta matriz se computa una única vez sobre la totalidad de categorías del catálogo, su representación dispersa optimiza drásticamente la memoria RAM sin necesidad de densificación previa al entrenamiento.

- **Gestión de Datos y Métricas de Evaluación:** La manipulación de los metadatos tabulares y el manejo de *embeddings* precomputados se basan en Pandas y NumPy. El cálculo estandarizado del AUC-ROC en la fase de prueba se instrumenta mediante Scikit-learn (`roc_auc_score`).

- **I/O de Imágenes y Telemetría Básica:** La lectura de imágenes en disco se articula mediante la librería Pillow (`PIL`), integrada directamente en los objetos `Dataset` de PyTorch, mientras que el monitoreo de bucles de ejecución por época y micro-lote se instrumenta con la librería `tqdm`.

7.1.2.1. Control de Versiones y Monitorización de Experimentos

El control de código fuente se ha articulado mediante Git, centralizado en un repositorio remoto privado. La adopción de este flujo garantiza la integridad del código y la trazabilidad del historial de cambios entre las distintas iteraciones de las arquitecturas. Este aspecto resultó crítico durante la implementación de HGNN, cuyo ajuste requirió correcciones específicas en la propagación sobre el hipergrafo para mitigar los efectos de sobre-suavizado (*over-smoothing*), tal como se detalla en el apartado 7.3.2.

Por otra parte, la monitorización de las sesiones de entrenamiento prescindió deliberadamente de plataformas de seguimiento complejo como TensorBoard o *Weights & Biases*. Esta decisión se alinea con la estrategia metodológica fijada en el apartado 6.4, donde se descartó la búsqueda sistemática de hiperparámetros en favor del cumplimiento estricto de las configuraciones de la literatura de referencia. El control del proceso se basó en la verificación de la convergencia de la función de pérdida frente a los valores reportados en los trabajos originales, apoyado puntualmente con Matplotlib para la representación de curvas de convergencia. La persistencia de los estados óptimos de los modelos se ejecutó mediante la serialización del `state_dict()` de PyTorch (`torch.save` / `torch.load`), garantizando la reproducibilidad en la fase de evaluación posterior (capítulo 8).

7.2. Pipeline de Procesamiento de Datos y Preparación de Corpora

El tratamiento eficaz de los datos de entrada constituye un requisito crítico para garantizar la estabilidad durante el entrenamiento y la validez cuantitativa de las evaluaciones. En esta sección se documenta el pipeline de ingesta, preprocesamiento y estructuración del benchmark de referencia, así como las limitaciones técnicas encontradas en el intento de construcción de un corpus comercial independiente.

7.2.1. Benchmark de Referencia: Polyvore Outfits

El conjunto de datos utilizado como *benchmark* de referencia es la partición *nondisjoint* de Polyvore Outfits, propuesta por Vasileva et al. (2018). Conforme a lo expuesto en el apartado 6.3, esta partición garantiza la comparabilidad directa con las cifras reportadas en la literatura científica al ser la adoptada de forma mayoritaria por las arquitecturas evaluadas.

Obtención, Descarga y Trazabilidad de los Datos.

El proceso de adquisición del conjunto de datos atravesó dos fases secuenciales hasta consolidar el *corpus* definitivo:

1. **Recolección inicial (GitHub y Kaggle):** Siguiendo las instrucciones de trazabilidad del repositorio oficial de Han et al. (2017b)[1], se procedió inicialmente a la descarga de los archivos de definición de *outfits* y metadatos textuales desde dicho repositorio de GitHub. De manera complementaria, los ficheros fotográficos de los productos se recolectaron a partir del repositorio indexado en Kaggle.

2. **Identificación del conflicto de partición y resolución vía Hugging Face:** Durante la fase de entrenamiento e integración del modelo SCE-Net, se detectó una discrepancia crítica en el volumen de datos procesados (incidencia documentada en detalle en el apartado 7.5.2). Para subsanar esta limitación, se reestructuró el *pipeline* de ingesta migrando hacia el otro repositorio oficial publicado por Tan et al. (2019); Vasileva et al. (2018) en la plataforma *Hugging Face*[2], garantizando la descarga verificada de la partición *nondisjoint* completa.

   Nota de reproducibilidad: En el momento de la redacción de esta memoria, la réplica de imágenes en Kaggle empleada en la fase inicial ha sido discontinuada. Para futuros trabajos de replicación, se recomienda canalizar la ingesta íntegra de metadatos e imágenes mediante solicitud directa al equipo de investigación original.

Estructura Jerárquica y Volumetría del Dataset

El *corpus* se estructura internamente mediante un esquema relacional distribuido en archivos JSON y CSV. Las particiones experimentales (`train.json`, `valid.json` y `test.json`) definen los *outfits* como secuencias ordenadas de ítems. Por su parte, `polyvore_item_metadata.json` compila las fichas de 251.008 artículos indexados en siete atributos primarios (`url_name`, `title`, `description`, `related`, `category_id`, `semantic_category` y `categories`), mientras que `polyvore_outfit_titles.json` asocia descriptores a nivel de conjunto. Finalmente, `categories.csv` codifica la taxonomía del dominio, compuesta por 158 identificadores distribuidos en 141 subcategorías y 13 categorías principales.

[1] https://github.com/xthan/polyvore-dataset
[2] https://huggingface.co/datasets/mvasil/polyvore-outfits

Tabla 7.1: Distribución volumétrica de la partición *nondisjoint* de Polyvore Outfits.

Partición	**Nº de Outfits**	**Artículos Únicos**	**Distribución (%)**
Entrenamiento	53.306	204.679	78,03 %
Validación	5.000	25.132	7,32 %
Prueba (Test)	10.000	47.854	14,65 %
Total	**68.306**	**251.008**	**100,00 %**

Adicionalmente, el *dataset* incorpora los conjuntos de evaluación predefinidos para las tareas estándar descritas en el apartado 6.3: las tuplas FITB con sus correspondientes candidatos y la colección de conjuntos negativos para *Compatibility Prediction*.

Implementación del Pipeline de Ingesta y Carga Perezosa

La ingesta de datos se articula mediante clases personalizadas derivadas de `torch.utils.data.Dataset`. El procesamiento se ejecuta en dos fases diferenciadas:

1. **Parsing de Metadatos y Alineamiento Categórico:** Extracción e indexación mediante la estructura item_id $\rightarrow$ (imagen, texto, categoría). La normalización de `semantic_category` resulta crítica tanto para la delimitación de distractores en la tarea FITB (apartado 6.3) como para la posterior construcción de la matriz de incidencia del hipergrafo en HGNN (apartado 8.3).
2. **Carga Perezosa e I/O de Imágenes:** Para optimizar el consumo de memoria RAM, la lectura de las imágenes desde disco se pospone hasta la invocación del método `__getitem__`.

La implementación de este pipeline presenta matices de optimización específicos entre las distintas arquitecturas:

- **Transferencia a Memoria de Vídeo (`pin_memory`):** El modelo Recommendation Engine activa explícitamente `pin_memory=True` condicionado a la presencia de GPU. SCE-Net únicamente lo implementa en el bucle de validación, mientras que HGNN prescinde de esta propiedad.
- **Paralelismo de Carga (`num_workers`):** El grado de paralelismo se fijó en `num_workers` ≤ 1 en las tres arquitecturas. En el caso particular de HGNN, esta restricción responde a incompatibilidades técnicas del módulo de multiprocessing y la serialización por *pickle* en entornos Windows, derivando en un proceso de I/O de carácter eminentemente secuencial.

Respecto al preprocesamiento visual, las transformaciones deterministas integradas mediante `torchvision.transforms` aplican el escalado espacial específico requerido por cada

backbone (224×224 píxeles para ResNet18 en SCE-Net y Recommendation Engine; 299×299 píxeles para InceptionV3 en HGNN). En todos los casos se aplica la normalización estadística sobre ImageNet:

$$\mu = [0{,}485, 0{,}456, 0{,}406], \quad \sigma = [0{,}229, 0{,}224, 0{,}225] \tag{7.1}$$

Auditoría de Calidad de Datos

El análisis exploratorio de los metadatos evidencia una clara asimetría entre las distintas modalidades sensoriales:

- **Integridad del Canal Visual:** Presenta un 100 % de disponibilidad. No se registraron punteros nulos ni ficheros corruptos. Las imágenes en estado crudo exhiben una homogeneidad dimensional estricta de 300×300×3 píxeles (RGB).
- **Incompletitud del Canal Textual:** Se identificó un elevado volumen de valores ausentes. El 72,0 % del catálogo global (180.725 artículos) carece de al menos un atributo descriptivo primario. En contraposición, los atributos de clasificación taxonómica (`category_id` y `semantic_category`) presentan una completitud del 100,0 %.

Tabla 7.2: Porcentaje de valores nulos o ausentes por atributo descriptivo en Polyvore Outfits.

Atributo de Metadato	Registros Incompletos	Tasa de Ausencia (%)
`description`	178.968	71,3 %
`related`	178.466	71,1 %
`title`	176.207	70,2 %
`categories`	176.207	70,2 %
`outfit_title`	10.792 (outfits)	15,8 %

Esta distribución de nulos responde a la ausencia nativa de fichas descriptivas durante el proceso de extracción original en la plataforma Polyvore.com.

Estrategia Homogénea de Concatenación y Fallback Textual

Para mitigar la pérdida masiva de datos (que habría alcanzado el 71,3 % de emplear únicamente el campo `description`), se diseñó una estrategia común de agregación de texto. Los tres pipelines construyen una secuencia sintética concatenando los campos disponibles (`title`, `description`, `url_name` y `sub_category`).

Para las instancias residuales carentes de cualquier atributo descriptivo, cada arquitectura aplica un mecanismo de *fallback* (detallado en el apartado 8.3): imputación mediante el vector promedio de la categoría semántica (Recommendation Engine y HGNN) o enriquecimiento mediante el campo de ítems relacionados `related` (SCE-Net).

7.2.2. Desafíos Técnicos, Legales y Metodológicos en la Adquisición de Corpora Comerciales

Conforme al diseño experimental fijado en el apartado 6.4, la construcción de un *corpus* de catálogo comercial real para la evaluación del Escenario B (*Out-of-Domain*) requería resolver cuatro fases secuenciales, articuladas en estricto cumplimiento del marco ético y legal:

1. **Verificación jurídica y directivas de acceso:** Inspección previa del archivo `robots.txt` y de los Términos de Servicio (ToS) de los dominios candidatos, evaluando explícitamente las restricciones sobre la extracción y redistribución del material fotográfico de producto.

2. **Infraestructura de ingesta:** Desarrollo de un bot de recolección estructurado en dos componentes principales: un *spider* de rastreo y un *parser* de extracción de metadatos.

3. **Alineamiento taxonómico:** Normalización de las categorías locales de cada tienda hacia la taxonomía de categorías principales de Polyvore Outfits (apartado 7.2.1), condición necesaria para reutilizar el pipeline de preprocesamiento y las arquitecturas sin introducir modificaciones sintácticas.

4. **Validación de combinaciones (*outfits*):** Identificación de fuentes que aportaran combinaciones explícitas predefinidas o, en su defecto, aplicación de una composición manual validada mediante revisión humana sobre un filtrado previo por puntuación de compatibilidad.

Prospección y Evaluación Sistemática de Fuentes de Datos La búsqueda de fuentes de datos comerciales se ejecutó abarcando múltiples alternativas tecnológicas, las cuales se desestimaron tras identificar barreras técnicas, legales o presupuestarias insalvables:

- **Grandes Cadenas de Moda Minorista (Grupo Inditex, H&M):** Se evaluaron en primera instancia por la calidad de su fotografía de producto y el volumen de sus catálogos. Inicialmente se descartaron debido a que la estructura de sus datos no contempla la agrupación explícita en *outfits*, sino únicamente productos individuales navegables por categoría. Tras reconsiderar la vía de composición manual previa revisión humana, las plataformas actualizaron sus mecanismos de protección, impidiendo la descarga de imágenes de producto.

- **Plataformas con Recomendación de Conjuntos (*StylingBooks*):** Se exploraron plataformas que ofrecen combinaciones prediseñadas, tales como *Uniqlo StylingBook* y *The Pulse Boutique* (*Shop-the-Look*). Aunque las directivas `robots.txt` permitían la navegación automatizada de las secciones correspondientes, los Términos de Servicio de ambas plataformas prohíben explícitamente la descarga y reutilización de su material fotográfico, incluso con fines académicos o de investigación. En cumplimiento de un marco ético, se descartó su extracción a pesar de contar con viabilidad técnica.

- **Comercio Minorista sobre Infraestructura Shopify:** Se contempló un conjunto de tiendas de menor escala construidas sobre Shopify para reutilizar un *parser* común aprovechando su plantilla estandarizada. Sin embargo, la revisión de sus licencias individuales reveló las mismas restricciones de redistribución de imágenes identificadas en la categoría anterior, descartándose por el mismo motivo legal.

- **APIs Comerciales de Moda:** Se exploró el acceso estructurado a catálogos mediante interfaces API comerciales (tales como Channel3 APIs o Rakuten Advertising APIs), lo que habría evitado la incertidumbre técnica y legal del *scraping* HTML. No obstante, los servicios con cobertura de catálogo suficiente operan bajo modelos de suscripción de pago cuyo coste supera el presupuesto asignado a este trabajo.

- **Corpora Existentes en la Literatura Científica:** Se auditó la disponibilidad de conjuntos de datos publicados en trabajos previos para escenarios de transferencia similares. *ASOS Outfit Dataset*, introducido por Bettaney et al. (2019), no es de acceso público y las solicitudes de acceso enviadas a los autores no obtuvieron respuesta. Otros conjuntos estándar en recuperación multimodal, como *Fashion200k* o *FashionIQ* (apartado 4.6), solo distribuyen los metadatos y anotaciones textuales, estando la descarga directa de imágenes bloqueada por titulares de derechos. Finalmente, alternativas citadas en la literatura relacionada (*FACAD* e *IQON3000*, publicados por Yang et al. (2022) y Song et al. (2019)) mostraron un estado de obsolescencia en sus enlaces de origen que impidió su recuperación efectiva.

Veredicto Metodológico y Repercusión en la Pregunta de Investigación RQ2 Agotadas las vías descritas sin haber obtenido autorización legal para la redistribución ni acceso efectivo a las fuentes que cumplían los requisitos de composición, se tomó la decisión de no ejecutar empíricamente el Escenario B en la presente memoria.

Las implicaciones de esta acotación sobre la respuesta a la pregunta de investigación **RQ2** se abordan en la discusión del capítulo 9, definiendo como línea de trabajo futuro tanto la negociación de convenios académicos de acceso a datos como la anotación manual de un *corpus* reducido.

7.3. Implementación Módulo a Módulo de las Arquitecturas

Una vez fijada la arquitectura general del *pipeline* y el flujo unificado de ingesta de datos, esquematizados en la figura 7.1, en este apartado se aborda la instanciación técnica y las decisiones de diseño concretas adoptadas para cada una de las tres arquitecturas objeto de estudio: *SCE-Net*, *HGNN* y *Recommendation Engine*. Conforme a los criterios de selección justificados en el apartado 6.2.

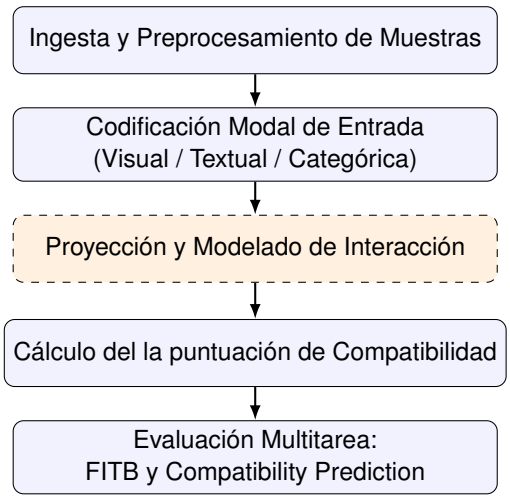


Figura 7.1: Flujo general del *pipeline* común a las tres arquitecturas de compatibilidad implementadas en este trabajo, desde la ingesta de datos (apartado 7.2) hasta la evaluación final (capítulo 8). La etapa de proyección y modelado arquitectónico (recuadro discontinuo) es el punto de divergencia entre modelos, detallado de forma individual para cada arquitectura en los apartados 7.3.1, 7.3.2 y 7.3.3.

Para garantizar un análisis riguroso, transparente y simétrico, la descripción de cada modelo se articula conforme a una plantilla metodológica homogénea estructurada en cinco bloques de ingeniería:

1. **Representación por modalidad:** Detalle de los *backbones*, dimensiones y naturaleza del entrenamiento (en línea o precomputado *offline*) para los componentes visuales, textuales y categóricos.
2. **Mecanismo de fusión y cálculo de compatibilidad:** Integración de las distintas fuentes de información y su traducción operativa en una puntuación de compatibilidad.
3. **Fidelidad al artículo original y vacíos de especificación:** Auditoría de los aspectos documentados en las fuentes primarias frente a las omisiones o inconsistencias identificadas en la literatura.
4. **Decisiones propias y extensiones sobre el diseño original:** Justificación técnica y diagnóstica de las modificaciones, términos de regularización, ajustes de hiperparámetros o módulos adicionales introducidos en esta implementación.
5. **Configuración final de hiperparámetros:** Tabla síntesis que resume la parametrización exacta empleada en las sesiones de entrenamiento.

7.3.1. SCE-Net: Extracción e Integración de Máscaras Latentes

Representación por modalidad

La representación visual se obtiene mediante un *backbone* ResNet18 preentrenado sobre ImageNet (`IMAGENET1K_V1`), del cual se conserva la totalidad de bloques convolucionales

hasta la capa de *pooling* global, descartando la cabeza de clasificación original. El vector de 512 dimensiones resultante se proyecta linealmente, sin término de sesgo, a un espacio de $D = 64$ dimensiones. Este *backbone* no se congela: se entrena de forma conjunta con el resto de la red mediante un *fine-tuning* completo, en estricta consonancia con el diseño original de Tan et al. (2019). La implementación mantiene simultáneamente dos versiones de este *embedding* visual; una primera normalizada mediante norma L_2; una segunda sin normalizar, dado que ambas se emplean de forma diferenciada en los distintos términos de la función de pérdida final (7.3.1).

La representación textual se genera mediante una tubería de procesamiento de Fisher Vectors implementada íntegramente dentro del entorno de entrenamiento del modelo, sin reutilizar el *pipeline* textual común descrito en el apartado 7.2.1: el texto combinado de cada ítem se proyecta a un espacio Word2Vec de 300 dimensiones (modelo preentrenado sobre el corpus Google News), sobre el cual se ajusta un modelo de mezcla de Gaussianas (GMM) de covarianza diagonal; el Fisher Vector resultante se reduce mediante PCA incremental a TEXT_DIM $= 2048$ dimensiones. A diferencia del canal visual, esta representación permanece estática durante toda la fase de optimización, únicamente se transforma mediante una proyección lineal entrenable de tres capas (con activación ReLU y *dropout*) integrada en la rama de ponderación de condición.

Conforme al diseño original de Tan et al. (2019), SCE-Net no recibe la categoría semántica como entrada explícita en ninguna capa de la red: la noción de tipo o condición de comparación se aprende de forma completamente latente a través del mecanismo de fusión descrito a continuación.

Mecanismo de fusión y cálculo de compatibilidad

El modelo mantiene un conjunto de $M = 5$ máscaras de condición aprendibles, de dimensión $D = 64$ cada una, aplicadas mediante producto de Hadamard sobre el *embedding* visual sin normalizar de cada ítem, generando M proyecciones candidatas en distintos subespacios semánticos. La relevancia de cada máscara para un par concreto de ítems se determina mediante la rama de ponderación de condición: los *embeddings* visuales y textuales proyectados de ambos ítems se combinan mediante producto elemento a elemento, se concatenan, y se procesan a través de un perceptrón multicapa (MLP) de tres capas hasta obtener M *logits*, que se convierten en pesos mediante una función *softmax* con temperatura $\tau = 0,1$.

Listing 7.1: Máscaras de condición y rama de ponderación dinámica de SCE-Net.

```
class ConditionMasks(nn.Module):
    def __init__(self, m_conditions, embed_dim):
        super().__init__()
        self.masks = nn.Parameter(torch.rand(m_conditions,
```

```
                                                    embed_dim))

    def forward(self, v):
        return v.unsqueeze(1) * self.masks.unsqueeze(0)  # (B, M, D)

    def l1_loss(self):
        return self.masks.abs().sum()


class ConditionWeightBranch(nn.Module):
    def __init__(self, m_conditions, embed_dim, text_dim,
                 hidden_dim=None, temperature=1.0):
        super().__init__()
        self.text_proj = nn.Sequential(
            nn.Linear(text_dim, 512), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(512, embed_dim), nn.ReLU(),
        )
        hidden = hidden_dim or max(2 * embed_dim, 64)
        self.mlp = nn.Sequential(
            nn.Linear(2 * embed_dim, hidden), nn.ReLU(),
                                              nn.Dropout(0.3),
            nn.Linear(hidden, hidden),        nn.ReLU(),
                                              nn.Dropout(0.3),
            nn.Linear(hidden, m_conditions),
        )
        self.temperature = temperature

    def forward(self, v_i, v_j, t_i, t_j):
        vt_i   = v_i * self.text_proj(t_i)
        vt_j   = v_j * self.text_proj(t_j)
        logits = self.mlp(torch.cat([vt_i, vt_j], dim=-1))
        return F.softmax(logits / self.temperature, dim=-1)
```

Una particularidad no especificada en el artículo original, y necesaria para resolver la implementación operativa, surge de la naturaleza inherentemente pareada de este mecanismo: la rama de ponderación requiere como entrada un par de ítems, mientras que la dinámica de entrenamiento se organiza en tripletas (ancla, positivo, negativo). En consecuencia, el *embedding* final del ítem ancla se calcula en dos instancias: una vez ponderado en relación con el ítem positivo, y otra en relación con el negativo, promediándose ambos resultados para obtener una representación única del ancla sobre la que evaluar la pérdida de tripletas (4.3.1). Esta resolución constituye una decisión de implementación propia adoptada en este trabajo

para garantizar el flujo de gradientes.

La puntuación de compatibilidad entre dos ítems se obtiene, en todos los casos, como la distancia euclídea entre sus *embeddings* finales normalizados. La resolución de las dos tareas de inferencia evaluadas se operacionaliza de la siguiente manera:

- **Predicción de compatibilidad del *outfit*:** Se evalúa la distancia euclídea media ponderada entre todos los pares posibles de prendas que componen el conjunto.
- **Resolución de FITB (*Fill-in-the-Blank*):** Dada una prenda faltante en un conjunto incompleto, se computa la distancia euclídea media de cada una de las opciones candidatas (correcta y distractores) frente a los ítems fijos del conjunto, seleccionando como respuesta la opción que minimiza dicha distancia métrica en el espacio latente.

Fidelidad al artículo original y vacíos de especificación

Tan et al. (2019) no publican el código fuente de la arquitectura ni especifican los hiperparámetros críticos de entrenamiento (optimizador, tasa de aprendizaje, tamaño de lote o margen de la *triplet loss*). Esta omisión ha exigido adoptar en este trabajo el esquema de tasas de aprendizaje diferenciadas propuesto por Vasileva et al. (2018), dada la coincidencia en el conjunto de datos y el *backbone* visual utilizado para completar los parámetros no documentados.

Asimismo, si bien el trabajo original argumenta que el número óptimo de máscaras de condición M debe determinarse experimentalmente mediante búsqueda sobre el conjunto de validación, la presente implementación no reproduce dicho barrido: el valor $M = 5$ se fija directamente por configuración. Se trata, por tanto, de un parámetro heredado de la práctica habitual reportada en la literatura de referencia, lo cual se asume formalmente como una simplificación metodológica respecto a la especificación original. Se evaluó también la perdida VSE original, descartada finalmente por razones diagnosticadas en el apartado 7.5.2.

Decisiones propias y extensiones sobre el diseño original

Las fases preliminares de ajuste evidenciaron un colapso de las M máscaras hacia patrones de ponderación redundantes, anulando la capacidad del modelo para capturar relaciones de similitud independientes. Para mitigar esta patología, se incorporaron tres extensiones no presentes en la publicación original de Tan et al. (2019):

1. **Temperatura en la selección de condición ($\tau = 0,1$):** Modulación que agudiza la distribución de pesos w entre las M condiciones, evitando asignaciones excesivamente planas o uniformes (código 7.1).
2. **Pérdida de diversidad de condiciones ($\lambda_{div} = 0,5$):** Término basado en la entropía de Shannon sobre la distribución w, el cual aplica una penalización únicamente cuan-

do dicha entropía cae por debajo de un umbral mínimo tolerable, sin forzar de forma artificial una distribución estrictamente uniforme.

3. **Pérdida de similitud global ($\lambda_{sim} = 5 \times 10^{-4}$):** Una *triplet loss* adicional evaluada sobre los *embeddings* visuales normalizados sin enmascarar (previos a la proyección por condiciones), obligando al espacio latente base a estructurar también una noción de compatibilidad general, acorde con la justificación de Plummer et al. (2019).

La función de pérdida global integra estos términos junto con una regularización L_1 sobre las máscaras ($\lambda_{L1} = 5 \times 10^{-4}$, aplicada como suma acumulada sin normalizar por el tamaño de lote) y una regularización L_2 explícita sobre el *embedding* visual crudo ($\lambda_{L2} = 5 \times 10^{-4}$) (implementado mediante el método `l1_loss()` del código 7.1):

$$\mathcal{L} = \mathcal{L}_{triplet} + \lambda_{L1}\mathcal{L}_{L1} + \lambda_{L2}\mathcal{L}_{L2} + \lambda_{sim}\mathcal{L}_{sim} + \lambda_{div}\mathcal{L}_{div} \tag{7.2}$$

El código 7.2 muestra la implementación conjunta de los cinco términos, reutilizando los módulos `cond_masks` y `weight_branch` ya introducidos en el código 7.1.

Listing 7.2: Función de pérdida total de SCE-Net, combinando la *triplet loss* principal con los cuatro términos de regularización.

```
def total_loss(model, E_a, E_p, E_n, v_a, v_p, v_n, t_a, t_p, t_n):
    l_t   = triplet_loss(E_a, E_p, E_n)
    l_l1  = model.cond_masks.l1_loss()
    l_l2  = (l2_reg(v_a) + l2_reg(v_p) + l2_reg(v_n)) / 3.0
    l_sim = sim_loss(v_a, v_p, v_n)

    w_ap  = model.weight_branch(v_a, v_p, t_a, t_p)
    w_an  = model.weight_branch(v_a, v_n, t_a, t_n)
    l_div = (diversity_loss(w_ap) + diversity_loss(w_an)) / 2.0

    loss = (l_t + LAMBDA_L1 * l_l1 + LAMBDA_L2 * l_l2
                + LAMBDA_SIM * l_sim + LAMBDA_DIV * l_div)
    return loss, l_t, l_l1, l_l2, l_sim, l_div
```

Es relevante destacar que la regularización L_2 se aplica de forma explícita mediante este término en la función de pérdida y no a través del mecanismo habitual de *weight decay* del optimizador, el cual se fija deliberadamente en 0 para evitar una penalización redundante sobre los mismos tensores de parámetros.

Adicionalmente, y con el objetivo de garantizar la estabilidad numérica y la eficiencia en el uso de VRAM sin alterar el comportamiento teórico de la arquitectura, se aplican de forma conjunta dos técnicas de optimización de cómputo (apartado 7.1.1): entrenamiento en precisión mixta automática (*Automatic Mixed Precision*, evaluando la pérdida explícitamente

en precisión completa de 32 bits fuera del bloque `autocast` para prevenir inestabilidades) y recortado de gradiente (*gradient clipping*) con norma máxima de $1,0$.

Configuración final de hiperparámetros

La tabla 7.3 resume la parametrización técnica y los hiperparámetros finales adoptados para el entrenamiento de la arquitectura SCE-Net.

Tabla 7.3: Configuración final de hiperparámetros de implementación para SCE-Net.

Hiperparámetro / Configuración	**Valor**
Dimensión del *embedding* (D)	64
Número de máscaras de condición (M)	5
Dimensión del vector textual (Fisher Vector)	2048
Temperatura (τ)	$0,1$
Margen de la *triplet loss* (m)	$0,2$
Coeficiente λ_{L1} (regularización máscaras)	5×10^{-4}
Coeficiente λ_{L2} (regularización *embeddings*)	5×10^{-4}
Coeficiente λ_{sim} (similitud global)	5×10^{-4}
Coeficiente λ_{div} (diversidad de condiciones)	$0,5$
Optimizador	Adam (`weight_decay=0`)
Tasa de aprendizaje (*backbone* visual)	$2,5 \times 10^{-5}$
Tasa de aprendizaje (máscaras + *weight branch*)	$2,5 \times 10^{-4}$
Planificador de tasa de aprendizaje	`ReduceLROnPlateau` (factor $0,1$, paciencia 3)
Tamaño de lote (B_{ef})	128
Épocas máximas / Paciencia (*early stopping*)	50 / 5

7.3.2. HGNN: Construcción del Hipergrafo y Paso de Mensajes

Representación por modalidad

La representación visual se obtiene mediante la arquitectura InceptionV3 preentrenada sobre ImageNet, de la cual se extrae el vector de 2048 dimensiones previo a la capa de clasificación final, en estricta consonancia con la especificación de Li et al. (2022). Este *backbone* permanece congelado: los vectores de características se precomputan de forma *offline* en la

tubería de preprocesamiento (apartado 7.2.1) y no propagan gradientes durante el entrenamiento del modelo. Dado que el trabajo original no explicita si la red convolucional se ajusta de extremo a extremo (*end-to-end*).

La representación textual se genera mediante una codificación Bag-of-Words (BoW) booleana, también precomputada de forma *offline*, construida sobre un vocabulario filtrado por frecuencia documental con un umbral mínimo de 100 apariciones y un filtro de longitud de término de al menos tres caracteres. Conviene destacar que esta modalidad no forma parte del diseño original de la arquitectura OCPHN Li et al. (2022), donde únicamente se emplea la información de imagen de los artículos.

La representación categórica, en cambio, constituye un elemento estructural de primer nivel en la arquitectura base: conforme a la formulación original, cada categoría dispone de un vector de características aprendible e independiente del contenido visual ($\mathbf{c}_i$). Este vector se concatena con la proyección de las características visuales ($\mathbf{r}_i$) mediante un perceptrón multicapa (MLP) de tres capas, dando lugar a un estado de nodo inicial de dimensión $2d$.

Mecanismo de fusión y resolución de tareas

A diferencia del mecanismo específico descrito por Li et al. (2022), el cual selecciona únicamente los dos nodos más divergentes de la hiperarista por cada conjunto para formar un grafo simple con nodos mediadores, propagado mediante matrices de entrada/salida específicas por nodo, esta implementación construye una única estructura de hipergrafo global, calculada de forma única sobre la totalidad del conjunto de entrenamiento: las categorías actúan como hipernodos y cada *outfit* funciona como una hiperarista que los conecta.

La propagación no emplea las matrices específicas de OCPHN, sino la matriz Laplaciana normalizada de hipergrafo estándar propuesta por Feng et al. (2019), trabajo que los propios Li et al. (2022) citan como antecedente general de la técnica, sin adoptar directamente su formulación:

$$\mathbf{\Theta} = \mathbf{D}_v^{-1/2}\,\mathbf{H}\,\mathbf{W}\,\mathbf{D}_e^{-1}\,\mathbf{H}^{\top}\,\mathbf{D}_v^{-1/2} \tag{7.3}$$

donde $\mathbf{H}$ representa la matriz de incidencia categoría-*outfit*, $\mathbf{W}$ pondera cada atuendo en función de su número de categorías distintas, y $\mathbf{D}_v, \mathbf{D}_e$ corresponden a las matrices diagonales de grado de nodo e hiperarista, respectivamente. El código 7.3 muestra su cómputo directo mediante operaciones matriciales dispersas.

Listing 7.3: Cálculo de la laplaciana normalizada de hipergrafo.

```
def compute_hypergraph_laplacian(H, W, D_v, D_e):
    D_v_inv_sqrt = sp.diags(np.where(D_v > 0, D_v ** -0.5, 0))
    D_e_inv      = sp.diags(np.where(D_e > 0, D_e ** -1.0, 0))
    Theta = D_v_inv_sqrt @ H @ W @ D_e_inv @ H.T @ D_v_inv_sqrt
```

```
    return Theta
```

Sobre esta matriz global y fija, cada capa de propagación aplica la siguiente transformación:

$$\mathbf{X}' = \text{Dropout}\big(\text{ReLU}(\text{Linear}(\mathbf{\Theta X}))\big) \tag{7.4}$$

implementada mediante el módulo del código 7.4.

Listing 7.4: Capa de propagación sobre el hipergrafo.

```
class HGNNPropagation(nn.Module):
    def __init__(self, in_dim, hidden_dim, dropout_rate):
        super().__init__()
        self.linear  = nn.Linear(in_dim, hidden_dim)
        self.norm    = nn.LayerNorm(hidden_dim)
        self.dropout = nn.Dropout(dropout_rate)

    def forward(self, X, Theta):
        # X:     (batch, n_categorias, dim_in)
        # Theta: (n_categorias, n_categorias)
        X = torch.einsum('nm,bmd->bnd', Theta, X)
        X = self.linear(X)
        X = self.norm(X)
        X = F.relu(X)
        return self.dropout(X)
```

La fusión multimodal se implementa mediante una combinación tardía:

$$\mathbf{x}_s = \beta \cdot \mathbf{x}_{vis} + (1-\beta) \cdot \mathbf{x}_{txt} \tag{7.5}$$

integrada junto a una función de pérdida de consistencia $\mathcal{L}_{con}$ entre las representaciones iniciales de ambos canales. Esta extensión deliberada permite evaluar el impacto de la multimodalidad de forma homogénea frente al resto de modelos, adoptando la lógica de combinación de la red NGNN Cui et al. (2019).

Por su parte, la puntuación de compatibilidad del conjunto de prendas, $S(O)$, tampoco reproduce la agregación de OCPHN sobre únicamente dos nodos clave: se calcula mediante un mecanismo de atención (importancia ponderada mediante sigmoide y compatibilidad vía activación LeakyReLU) aplicado y sumado sobre la totalidad de los ítems del *outfit* tras la fase de propagación. Esta misma función resuelve ambas tareas de evaluación:

- **Predicción de compatibilidad del *outfit*:** Se obtiene la puntuación $S(O)$ de la coleccion de prendas ejecutando un *forward pass* completo del modelo sobre cada uno de sus ítems.

- **Resolución de FITB (*Fill-in-the-Blank*):** El modelo no aísla la contribución de un único nodo, sino que reconstruye el conjunto completo incorporando secuencialmente cada uno de los M candidatos. El *forward pass* se ejecuta íntegramente de forma independiente para cada opción, recalculando $S(O)$ desde cero y seleccionando aquel candidato cuyo atuendo resultante maximiza la puntuación global.

Fidelidad al artículo original y vacíos de especificación

La auditoría del trabajo de Li et al. (2022) frente a su código fuente distribuido permitió identificar diferencias clave entre la especificación teórica y su implementación práctica:

- **Dimensión oculta ($d = 16$) y pasos de propagación ($T = 1$):** Aunque la elección de $d = 16$ y $T = 1$ coincide numéricamente con los valores óptimos reportados por Li et al. (2022), conviene precisar que en esta implementación dichos hiperparámetros se aplican sobre el operador de propagación global de Feng et al. (2019). En las fases preliminares de este trabajo con $T = 3$, la función de pérdida BPR permaneció estancada en $\ln(2)$ (equivalente a una predicción aleatoria) incluso al forzar el sobreajuste sobre un subconjunto reducido de control. Este comportamiento se diagnosticó como un fenómeno de sobre-suavizado, derivado de la elevadísima densidad ($\approx 90\,\%$) de la matriz Laplaciana normalizada $\mathbf{\Theta}$ sobre la estructura global. Reducir T a 1 corrigió dicho bloqueo, evidenciando de forma experimental la necesidad de restringir la profundidad de propagación en este operador para preservar la capacidad discriminativa de las representaciones.

- **Formulación de la pérdida BPR:** Se detectó una discrepancia entre la ecuación publicada y el código oficial. La fórmula matemática del artículo define la pérdida como $-\ln \sigma(S_{pos} - S_{neg})$, que es la opción adoptada por defecto en esta memoria. El repositorio distribuido, sin embargo, empleaba la variante sin logaritmo $-\sigma(S_{pos} - S_{neg})$, cuyo gradiente sufre de desvanecimiento en diferencias de puntuación elevadas.

- **Alcance de la regularización L_2:** La formulación original aplica la penalización de peso $\lambda\|\Theta\|_2^2$ sobre todos los parámetros sin distinción. En esta implementación, el *weight decay* se restringe a los tensores de peso de dimensión ≥ 2, excluyendo sesgos y parámetros de normalización, lo que evita que la red colapse hacia una solución trivial constante para minimizar $\mathcal{L}_{con}$.

Decisiones propias y extensiones sobre el diseño original

Además de la extensión multimodal, se incorporaron dos componentes de estabilización numérica no presentes en la formulación de Li et al. (2022):

1. **Normalización mediante LayerNorm:** Aplicada antes de la proyección inicial de características crudas (debido a las marcadas diferencias de escala entre InceptionV3 y

el vector BoW) e inmediatamente antes de la capa de atención final. Su omisión provocaba la saturación de la activación $\tanh$ en ± 1, degradando la pérdida a la cota aleatoria de $\ln(2)$.

2. **Regularización por Dropout:** Se añadió una tasa de *dropout* intermedio ($p = 0,1$) desactivable mediante configuración.

Por su parte, el coeficiente del término de consistencia entre canales se fijó en $\lambda_{con} = 0,01$. Se constató experimentalmente que valores superiores ($\lambda_{con} \geq 0,1$) inducían un colapso de las representaciones hacia vectores nulos como mecanismo trivial para minimizar dicho término.

Configuración final de hiperparámetros

La tabla 7.4 sintetiza la parametrización técnica y los hiperparámetros finales utilizados en la arquitectura HGNN.

Tabla 7.4: Configuración final de hiperparámetros de implementación para HGNN.

Hiperparámetro / Configuración	**Valor**
Frecuencia mínima de categoría (*train*)	100
Mínimo de ítems válidos por *outfit*	3
Dimensión oculta (d)	16
Pasos de propagación (T)	1
Peso del canal visual en la fusión tardía (β)	0,2
Regularización L_2 (`weight_decay`, solo pesos)	0,001
Tasa de *dropout*	0,1
Función de pérdida	BPR ($-\ln \sigma(S_{pos} - S_{neg})$)
Coeficiente λ_{con} (consistencia visual-textual)	0,01
Optimizador	Adam
Tasa de aprendizaje (η)	0,0001
Tamaño de lote (B_{ef})	16
Épocas máximas / Paciencia (*early stopping*)	30 / 5

7.3.3. Recommendation Engine: Redes de Atención Transformer Multimodal

Representación por modalidad

La representación visual se obtiene mediante `SmallResNet18`, una variante reducida de la arquitectura ResNet18 que omite el último grupo de bloques residuales (`layer4`), conservando únicamente los tres primeros grupos (64, 128 y 256 canales, con dos bloques básicos cada uno) sobre imágenes de 224×224 píxeles. Como se justificó en el apartado 7.1.2, esta

reducción responde a una decisión propia adoptada ante la ausencia de especificación arquitectónica en el trabajo de Prato et al. (2020), tomando como referencia la convención de Vasileva et al. (2018). El *backbone* completo se optimiza de extremo a extremo (*end-to-end*), sin capas congeladas.

La representación textual parte de vectores GloVe promediados de 100 dimensiones, precomputados de forma *offline* sin reducción por PCA (apartado 7.1.2). Sobre este vector estático actúa un módulo `TextEncoder` entrenable, una proyección lineal seguida de activación ReLU, que sí recibe gradientes de las funciones de pérdida del modelo. De este modo, aunque la incrustación GloVe inicial permanece fija, su transformación hacia el espacio latente conjunto se optimiza dinámicamente durante el entrenamiento.

La representación categórica se genera mediante un módulo de incrustación aprendible por categoría (`CategoricalEncoder`), implementado como una capa de incrustación de parámetros (`nn.Embedding`). A diferencia de las modalidades visual y textual, que parten de características precalculadas y se proyectan linealmente a $d_{\text{model}} = 128$, este módulo mapea directamente cada identificador de categoría entero a un vector denso entrenable de dimensión $d_{\text{model}} = 128$ (e_{cat} en la figura 7.2), inicializado aleatoriamente y optimizado conjuntamente con el resto de la red. Esta parametrización directa permite que el vector categórico se sume elemento a elemento de forma nativa con los vectores proyectados de imagen y texto, actuando además como la condición de entrada para el módulo `TypeAwareGate`.

Las tres modalidades se proyectan a una dimensión común $d_{\text{model}} = 128$ y se combinan mediante suma elemento a elemento, seguida de una capa de normalización (`LayerNorm`). El resultado de esta combinación constituye la representación fusionada del ítem (x en la figura 7.2), la cual es posteriormente modulada por el módulo `TypeAwareGate` mediante el esquema FiLM detallado en dicha figura. Dado que Prato et al. (2020) no detallan el operador matemático empleado para integrar imagen, texto y categoría, este mecanismo de fusión constituye una extensión propia de esta implementación.

Mecanismo de fusión y cálculo de compatibilidad

Sobre la representación fusionada de cada ítem actúa el módulo `TypeAwareGate`, la extensión arquitectónica central de este modelo: una modulación de tipo FiLM condicionada por la categoría semántica:

$$\mathbf{x}' = \mathbf{x} \odot \boldsymbol{\gamma} + \boldsymbol{\beta} \tag{7.6}$$

con $\boldsymbol{\gamma} = \sigma(\mathbf{W}_{\gamma}\mathbf{e}_{cat})$ y $\boldsymbol{\beta} = \mathbf{W}_{\beta}\mathbf{e}_{cat}$, donde $\mathbf{e}_{cat}$ representa el vector de incrustación de la categoría del ítem.

La secuencia de ítems modulados se antepone con un token aprendible `[CLS]` y se procesa mediante un codificador Transformer de 2 capas, 4 cabezas de atención y una red de

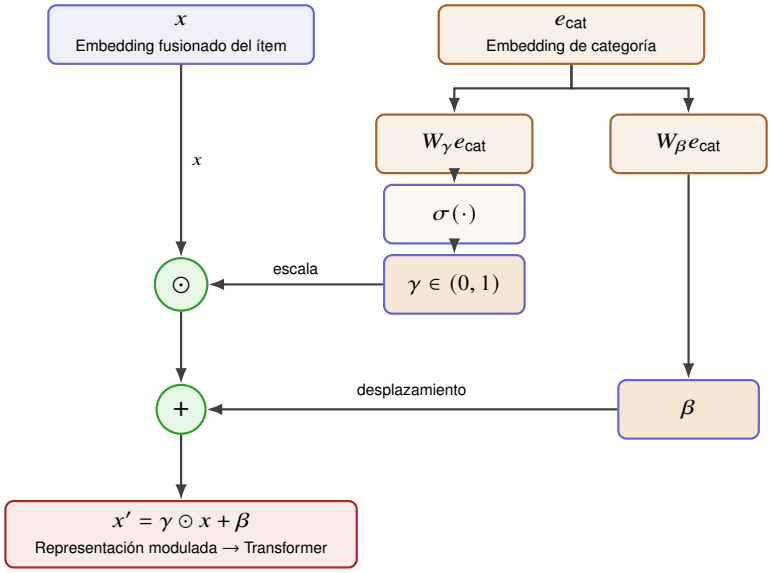


Figura 7.2: Esquema del módulo `TypeAwareGate` basado en modulación de características FiLM (*Feature-wise Linear Modulation*) sigue a Perez et al. (2017), aplicado sobre espacios condicionados por categoría en la linea de Vasileva et al. (2018). El *embedding* de categoría e_{cat} proyecta de forma independiente un factor multiplicativo de escala γ y un sesgo aditivo de desplazamiento β, condicionando el *embedding* del ítem x antes de ser procesado por el Transformer del *Recommendation Engine*.

avance (*feed-forward*) de dimensión 256 (`nn.MultiheadAttention`, con máscara de relleno para conjuntos de longitud variable). Conforme a la justificación ya expuesta en el apartado 5.1.6 —tratar el *outfit* como un conjunto no ordenado, evitando el sesgo de orden artificial propio de una secuencia—, este codificador no incorpora ningún término de *positional encoding*, tal como refleja el código 7.5.

Listing 7.5: Bloque de codificador Transformer del *Recommendation Engine*, sin *positional encoding*.

```
class TransformerEncoderBlock(nn.Module):
    def __init__(self, d_model, n_heads, ffn_dim, dropout=0.1):
        super().__init__()
        self.attention = nn.MultiheadAttention(embed_dim=d_model,
            num_heads=n_heads, dropout=dropout, batch_first=True)
        self.ffn = nn.Sequential(
            nn.Linear(d_model, ffn_dim), nn.ReLU(inplace=True),
            nn.Dropout(dropout), nn.Linear(ffn_dim, d_model),
        )
        self.norm1 = nn.LayerNorm(d_model, eps=1e-6)
        self.norm2 = nn.LayerNorm(d_model, eps=1e-6)

    def forward(self, x, key_padding_mask=None):
        attn_out, _ = self.attention(x, x, x,
```

```
        key_padding_mask=key_padding_mask, need_weights=False)
        x = self.norm1(x + attn_out)
        return self.norm2(x + self.ffn(x))
```

La representación final del token `[CLS]`, normalizada mediante norma L_2, resuelve ambas tareas de evaluación a través de dos cabezas de proyección independientes:

- **Predicción de compatibilidad del *outfit*:** La representación `[CLS]` se proyecta mediante un perceptrón multicapa (MLP) con activación Sigmoide final, produciendo directamente una probabilidad de compatibilidad $\hat{p}(O) \in [0, 1]$.

- **Resolución de FITB (*Fill-in-the-Blank*):** Para cada uno de los M candidatos, se concatena la salida `[CLS]` del contexto (que ha atravesado el Transformer completo) con la incrustación del candidato tras pasar únicamente por `TypeAwareGate`, sin atravesar el Transformer ni interactuar mediante auto-atención con los ítems del contexto. El vector resultante de dimensión $2d_{model}$ se proyecta mediante un segundo MLP hasta obtener un valor (*logit*) no normalizado por candidato.

Esta asimetría en la representación del candidato constituye una decisión de eficiencia computacional deliberada, no discutida por Prato et al. (2020). Evaluar los M candidatos haciéndolos pasar individualmente por el Transformer completo junto con el contexto requeriría $O(M)$ pasadas del codificador por conjunto evaluado, recalculando en cada iteración la auto-atención entre los ítems del contexto (que es idéntica e independiente de cada candidato). Esta implementación optimiza la evaluación: el contexto se codifica una única vez ($O(1)$ pasadas del Transformer), y los M candidatos se procesan en paralelo mediante una única llamada vectorizada sobre el tamaño de lote $B \times M$, evitando M ejecuciones secuenciales. El coste de esta escalabilidad es que la representación del candidato no incorpora atención cruzada explícita con los ítems del contexto ni con otros candidatos; el modelo evalúa la compatibilidad frente al contexto agregado, asumiendo un compromiso deliberado entre capacidad expresiva y viabilidad computacional (apartado 7.1.1).

Fidelidad al artículo original y vacíos de especificación

Prato et al. (2020) no publican código ni especifican hiperparámetros de entrenamiento (optimizador, tasa de aprendizaje, decaimiento de pesos, tamaño de lote o estructura exacta del Transformer), limitándose a la descripción conceptual de la arquitectura de dos etapas expuesta en el capítulo 5. La configuración de este apartado ha sido determinada de forma independiente en este trabajo, tomando como referencia indirecta el esquema de entrenamiento de Vasileva et al. (2018), del cual se adapta tanto el *backbone* visual reducido como la escala de la tasa de aprendizaje y la parametrización de Transformers pequeños Vaswani et al. (2023).

Decisiones propias y extensiones sobre el diseño original

Además de la incorporación del módulo `TypeAwareGate` y la vectorización asimétrica de la tarea FITB, esta implementación introduce las siguientes decisiones de diseño:

- **Optimización multitarea unificada:** Con el fin de entrenar un único modelo que optimice ambas tareas, en lugar de dos modelos independientes, se opta por una optimización conjunta. Se implementa una función de pérdida conjunta mediante suma simple, combinando Entropía Cruzada Binaria ($\mathcal{L}_{BCE}$) para la compatibilidad y Entropía Cruzada Categórica ($\mathcal{L}_{CatCE}$) para FITB:

$$\begin{aligned} \mathcal{L} &= \mathcal{L}_{CE} + \mathcal{L}_{FITB}, \quad \text{donde} \quad \mathcal{L}_{CE} = \mathcal{L}_{BCE}\big(\hat{p}(O), y\big), \\ \mathcal{L}_{FITB} &= \mathcal{L}_{CatCE}\big(\{s_1, \ldots, s_M\}, c^*\big). \end{aligned} \tag{7.7}$$

  donde $\hat{p}(O) \in [0, 1]$ es la estimación sigmoidal de la cabeza de compatibilidad, $y \in \{0, 1\}$ es la etiqueta binaria del conjunto, $\{s_1, \ldots, s_M\}$ son los *logits* producidos por la cabeza FITB para los M candidatos, y c^* es el índice de la opción correcta. Esta formulación es coherente con las dos cabezas de salida: `ce_head` incluye activación Sigmoide interna, mientras que `fitb_head` emite *logits* sobre los que `CrossEntropyLoss` aplica internamente la función *softmax*.

Configuración final de hiperparámetros

La tabla 7.5 sintetiza la parametrización técnica y los hiperparámetros finales adoptados para la arquitectura *Recommendation Engine*.

7.4. Módulo de Búsqueda y Recomendación Text-to-Outfit

El módulo *Text-to-Outfit* extiende las capacidades del sistema de recomendación permitiendo la recuperación de combinaciones de prendas a partir de consultas expresadas libremente en lenguaje natural. Para abordar esta tarea, se exploraron dos vías con fundamentaciones técnicas distintas: por un lado, una aproximación de bajo coste computacional orientada a reutilizar y adaptar el espacio latente multimodal preentrenado en la arquitectura *Recommendation Engine* (apartado 7.3.3); por otro, la implementación de *CLIP4Outfit* Jang et al. (2023), una arquitectura dedicada específicamente al aprendizaje contrastivo entre descripciones globales y conjuntos de prendas basada en modelos visión-lenguaje.

Tabla 7.5: Configuración final de hiperparámetros de implementación para Recommendation Engine.

Hiperparámetro / Configuración	**Valor**
Dimensión del modelo (d_{model})	128
Cabezas de atención	4
Dimensión de la red *feed-forward*	256
Capas del Transformer codificador	2
Longitud máxima del *outfit*	8
Tasa de *dropout*	0,1
Función de pérdida (Compatibilidad)	$\mathcal{L}_{BCE}$ (sobre salida sigmoidal)
Función de pérdida (FITB)	$\mathcal{L}_{CatCE}$ (sobre *logits* sin normalizar)
Optimizador	AdamW
Tasa de aprendizaje (η)	5×10^{-5}
Decaimiento de pesos (`weight_decay`)	1×10^{-4}
Tamaño de lote (micro-lote / efectivo)	8 / 32 (acumulación $\times 4$)
Épocas máximas / Paciencia (*early stopping*)	50 / 5

7.4.1. Recommendation Engine: Adaptación del Espacio Latente Multimodal

Formulación del espacio de representación reutilizado

A diferencia de *CLIP4Outfit* (apartado 7.4.2), que construye un modelo visión-lenguaje dedicado desde cero, esta primera aproximación explota directamente la infraestructura previamente optimizada de *Recommendation Engine* (apartado 7.3.3). El *backbone* completo, constituido por el módulo `ItemEncoder`, la puerta de procesamiento de tipos `TypeAwareGate` y el *Transformer* de 2 capas y 4 cabezas de atención, se carga a partir del mejor punto de control obtenido en la tarea de predicción de compatibilidad (`best_ce.pt`) y se congela en su totalidad. De este modo, se reutiliza la salida del token `[CLS]` como un *embedding* vectorial de 128 dimensiones representativo de atuendo. Esta decisión responde a un criterio deliberado de eficiencia computacional: en lugar de entrenar un espacio latente específico para la tarea de recuperación orientada por texto, se evalúa hasta qué punto la representación aprendida para la evaluación de compatibilidad, desarrollada sin supervisión textual a nivel de conjunto, resulta transferible a este segundo propósito.

Implementación de la estrategia *Zero-Shot*

Sobre este espacio latente estático, se entrena exclusivamente un componente ligero denominado codificador de consultas (`QueryEncoder`). Este módulo proyecta la consulta en lenguaje natural, representada mediante el vector GloVe promediado de 100 dimensiones, a través de un perceptrón multicapa (MLP) dotado de una capa oculta de 256 unidades, función de activación ReLU y una tasa de *dropout* de 0,1, mapeándola hacia el espacio compartido

de 128 dimensiones. La correspondencia entre *outfit* y texto se establece a partir de los atributos `title` y `url_name` del *dataset* (utilizando `url_name` como alternativa de reserva en caso de ausencia de título). La optimización se articula mediante un esquema de aprendizaje contrastivo basado en la pérdida InfoNCE (apartado 5.2.1), con una temperatura aprendible, inicializada en $\tau = 0{,}07$ en consonancia con la literatura convencional de modelos visión-lenguaje.

Diagnóstico de duplicidad y formulación *Multi-Positive*

Un análisis exploratorio del *corpus* reveló una limitación estructural que invalidaba el uso directo de la formulación InfoNCE estándar: el $92{,}4\,\%$ de los atuendos comparten su descripción textual con al menos otra combinación dentro del mismo dominio (por ejemplo, la denominación «*Clog Sandals*» se encuentra asociada a 825 *outfits* distintos). Bajo el esquema diagonal convencional, donde cada instancia del lote asume de forma biyectiva una única etiqueta correcta (labels $= [0, 1, \ldots, B-1]$), cualquier par de conjuntos que compartiese la misma descripción dentro de un lote era categorizado erróneamente como un par negativo. Esta asunción penalizaba de forma indebida al modelo al distanciar representaciones semánticamente equivalentes.

Para subsanar este inconveniente, se asignó a cada muestra un identificador de grupo (`group_id`) determinado por su contenido textual dentro de la partición correspondiente, y se sustituyó la función InfoNCE por una variante enmascarada de carácter *multi-positivo* adaptada del marco SupCon Khosla et al. (2021):

$$\mathcal{L}_i = -\frac{1}{|P(i)|} \sum_{j \in P(i)} \log \frac{\exp(s_{ij}/\tau)}{\sum_{k=1}^{B} \exp(s_{ik}/\tau)} \tag{7.8}$$

donde $P(i)$ denota el conjunto de índices del lote que comparten el mismo identificador de grupo textual con la muestra i (incluyéndose a sí misma), y s_{ij} representa la similitud coseno entre la representación de la consulta i y la del *outfit* j. La métrica de evaluación se ajustó coherentemente a este planteamiento: una consulta se contabiliza como un acierto dentro del Top-K si cualquiera de los conjuntos de prendas pertenecientes a su grupo textual de referencia figura dentro de dicho *ranking*.

Esquema de *Fine-Tuning*

Como extensión al enfoque estático, se contempló el ajuste fino parcial del espacio de representación de *Recommendation Engine*. En esta variante, se descongela de forma selectiva la última capa del *Transformer* (cargada desde `best_ce.pt`), manteniendo congelado el resto del *backbone*. A fin de optimizar el consumo de recursos durante el entrenamiento, se diseñó la rutina `encode_outfit_prelast`, la cual ejecuta y almacena en memoria caché las representaciones intermedias procedentes de las capas fijas. De este modo, la última capa se recalcula en cada iteración de optimización, permitiendo la propagación del gradiente

hacia sus parámetros sin la necesidad de recomputar las etapas anteriores.

El proceso de ajuste aplica tasas de aprendizaje (LR) diferenciadas: 10^{-3} para el módulo `QueryEncoder` (de inicialización aleatoria) y 10^{-5} para la última capa del *Transformer*, previniendo el fenómeno del olvido catastrófico en el espacio preentrenado. Sobre este esquema se evaluaron dos etapas adicionales de depuración textual aplicadas exclusivamente sobre la partición de entrenamiento:

1. **Capa 1 (Filtrado de *spam* y contenido promocional):** Eliminación de hiperenlaces, códigos de descuento y referencias a tallajes. Esta capa se mantuvo en la configuración definitiva dado su impacto favorable en el rendimiento.
2. **Capa 4 (Truncado por frecuencia de etiqueta):** Dirigida a mitigar el sesgo provocado por descripciones excesivamente recurrentes. Fue **descartada** tras comprobar empíricamente que, una vez resuelta la duplicidad mediante la función de pérdida *multi-positivo*, la restricción artificial de frecuencias no aportaba mejoras significativas y reducía la variabilidad del conjunto de entrenamiento.

Inferencia y recuperación por similitud vectorial

Durante la etapa de inferencia, la consulta expresada en lenguaje natural se proyecta mediante el `QueryEncoder` hacia el espacio latente de 128 dimensiones. Posteriormente, se evalúa la similitud coseno entre dicho vector y la matriz de *embeddings* precomputados correspondientes a la totalidad del catálogo de *outfits* (utilizando la representación estática o la ajustada, según la variante considerada). El sistema ordena y devuelve las K combinaciones que presentan los mayores índices de similitud semántica.

7.4.2. CLIP4Outfit: Alineación Multimodal y Recuperación Contrastiva

Motivación y arquitectura

A diferencia de la aproximación descrita en el apartado anterior, la cual reutiliza el espacio de representación previamente optimizado de *Recommendation Engine*, esta segunda vía construye un modelo dedicado a partir de un *backbone* visión-lenguaje preentrenado. Este enfoque reproduce fielmente el diseño jerárquico de tres niveles descrito en el apartado 5.2.2. La arquitectura de partida emplea la variante `openai/clip-vit-base-patch32` (ViT-B/32) de CLIP, esta, no permanece congelada durante la segunda fase de entrenamiento. Sus parámetros se optimizan de forma conjunta (*end-to-end*) con los nuevos módulos de ponderación WTI y los *Transformers* contextuales de estilo y conjunto.

Tabla 7.6: Configuración de hiperparámetros para la adaptación de *Recommendation Engine*.

Hiperparámetro	**Estrategia *Zero-Shot***	**Estrategia *Fine-Tuning***
Dimensión de entrada (GloVe)	100	100
Arquitectura `QueryEncoder`	MLP (Capa oculta: 256, ReLU, Dropout 0,1)	MLP (Capa oculta: 256, ReLU, Dropout 0,1)
Dimensión latente de salida (d)	128	128
Temperatura inicial (τ)	0,07 (aprendible)	0,07 (aprendible)
Componentes optimizables	`QueryEncoder`	`QueryEncoder` + Última capa *Transformer*
Tasa de aprendizaje (LR)	10^{-3}	10^{-3} (`QueryEncoder`) / 10^{-5} (*Transformer*)
Decaimiento de pesos (*Weight decay*)	10^{-4}	10^{-4}
Optimizador	*AdamW*	*AdamW*
Épocas máximas / Paciencia	100 / 10	100 / 10
Criterio de parada temprana	*Recall@5* (Validación)	*Recall@5* (Validación)
Filtros de calidad textual	—	Capa 1 (Filtrado de *spam*)

Para ello se utiliza un único optimizador y una tasa de aprendizaje común (5×10^{-7}), sin distinción entre componentes, lo cual resulta coherente con las especificaciones del artículo original Jang et al. (2023), que no contempla diferenciación en el ajuste de los pesos. Esta etapa intermedia permite mitigar la brecha de dominio entre las imágenes genéricas de pre-entrenamiento y la fotografía de producto, en sintonía con la metodología de *FashionCLIP* Chia et al. (2022), antes de articular sobre ella la arquitectura de recuperación completa.

Como se ilustra en el esquema de la figura 7.3, la implementación integra los tres niveles semánticos progresivos del diseño original:

- **Nivel Ítem:** La similitud entre las prendas del *outfit* y los *tokens* de la consulta textual se evalúa mediante *Weighted Tokenwise Interaction* (WTI). Este mecanismo pondera la contribución de cada prenda y *token* mediante dos módulos de importancia aprendidos (`AdaptiveWeightNet`, constituidos por una proyección lineal con *softmax* enmascarado y un parámetro de escalado $p = 0{,}2$), manteniendo estricta fidelidad con la ecuación 5.23 y con la implementación recogida en el Código 7.6.

- **Nivel Estilo:** Los *embeddings* de ítem y de *token*, contextualizados mediante un *Transformer Encoder* específico por rama (visual y textual), se agrupan en centroides representativos mediante el algoritmo K-means. El número de clústeres se determina de forma proporcional a la dimensión del *outfit* ($K_o = \lceil N/3 \rceil$ para la rama visual y $K_t = \lceil N/6 \rceil$ para la textual, donde N representa el número de prendas). La inicialización de los centroides no se realiza de forma aleatoria, sino mediante una selección

greedy que maximiza la distancia mínima entre ellos; esta estrategia previene la inestabilidad intrínseca del algoritmo K-means en conjuntos con un número reducido de prendas. La similitud sobre estos centroides se calcula reutilizando, sin ninguna modificación, la misma función WTI del nivel Ítem (Código 7.6), sustituyendo únicamente los *embeddings* de entrada por los centroides c_o^k y c_t^l.

- **Nivel Conjunto:** Los centroides de estilo se procesan mediante un segundo *Transformer Encoder* y se agregan a través de un *mean pooling* enmascarado. De este modo, se obtiene una única representación global para el *outfit* y para la consulta textual, las cuales se comparan finalmente mediante similitud coseno directa (S_{outfit}).

Listing 7.6: Cálculo de la similitud *Weighted Tokenwise Interaction* (WTI), reutilizada en los niveles Ítem y Estilo de CLIP4Outfit.

```
class AdaptiveWeightNet(nn.Module):
    def __init__(self, d_model):
        super().__init__()
        self.linear = nn.Linear(d_model, 1)

    def forward(self, x, mask):
        logits = self.linear(x).squeeze(-1)
        return masked_softmax(logits, mask, dim=-1)


def wti_similarity_batch(item_emb, item_mask, text_emb, text_mask,
                         fo_weight, ft_weight, p=P_WEIGHT):
    sim = torch.einsum("nid,mjd->nmij", item_emb, text_emb)
    joint_mask = item_mask.unsqueeze(1).unsqueeze(-1) &
    text_mask.unsqueeze(0).unsqueeze(2)
    sim = sim.masked_fill(~joint_mask, float("-inf"))

    max_over_tokens = torch.nan_to_num(sim.max(dim=3).values,
                                       neginf=0.0)
    term1 = (fo_weight.unsqueeze(1) * max_over_tokens).sum(dim=2)

    max_over_items = torch.nan_to_num(sim.max(dim=2).values,
                                      neginf=0.0)
    term2 = (ft_weight.unsqueeze(0) * max_over_items).sum(dim=2)

    return (term1 + p * term2) / (1 + p)
```

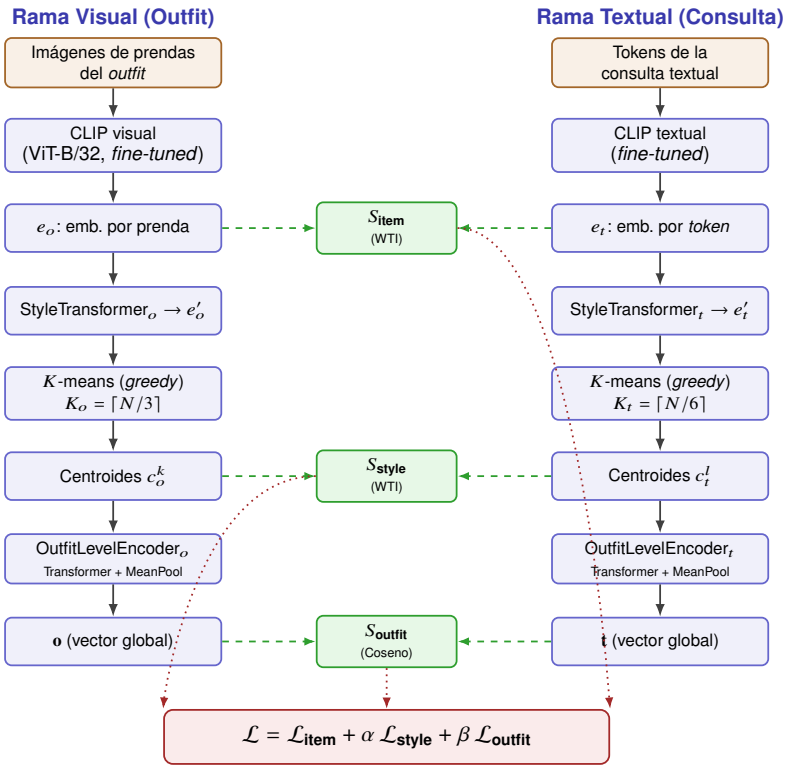


Figura 7.3: Arquitectura jerárquica de tres niveles de CLIP4Outfit, inspirada en el diseño propuesto por Jang et al. (2023). Las dos ramas paralelas (visual y textual) procesan los componentes del *outfit* y la consulta, evaluando la similitud cruzada en tres niveles semánticos progresivos (S_{item}, S_{style} y S_{outfit}). Las flechas punteadas indican la agregación de las tres métricas en la función de pérdida combinada $\mathcal{L}$.

Fidelidad al artículo original y vacíos de especificación

A diferencia de las arquitecturas de compatibilidad evaluadas en el apartado 7.3, donde la falta de hiperparámetros obligaba a adoptar soluciones heurísticas propias, Jang et al. (2023) detallan explícitamente en su trabajo el optimizador (Adam, excluyendo AdamW), la tasa de aprendizaje (5×10^{-7}), el número de épocas (20) y el tamaño de lote efectivo (50); todos ellos adoptados de manera literal en esta implementación. La arquitectura jerárquica y la formulación de la función de pérdida combinada:

$$\mathcal{L} = \mathcal{L}_{\text{item}} + \alpha \mathcal{L}_{\text{style}} + \beta \mathcal{L}_{\text{outfit}} \tag{7.9}$$

se reproducen con estricta fidelidad. Ante la ausencia del valor exacto de los coeficientes α y β en el artículo de referencia, se ha optado por una asignación conservadora y simétrica de $\alpha = 0{,}3$ y $\beta = 0{,}3$.

La principal extensión introducida respecto al diseño original radica en la integración del esquema *multi-positivo* (apartado 7.4.1) en las tres funciones de pérdida de nivel. Esta modificación responde al problema de redundancia textual diagnosticado en el *corpus*, incorporándose en esta arquitectura desde el diseño inicial del *pipeline*. Esta adaptación difiere del protocolo original, el cual asume una correspondencia unívoca y estricta entre cada *outfit* y su descripción.

Mecanismos de agregación e integración del aprendizaje contrastivo

El entrenamiento a lo largo de los tres niveles se ejecuta mediante la técnica *GradCache*, la cual permite optimizar el modelo con un tamaño de lote efectivo sensiblemente mayor al que la memoria física de la GPU sostendría en un único paso de propagación. El proceso se articula en tres fases:

1. **Generación de representaciones sin gradiente:** Se evalúan los *embeddings* de todos los micro-lotes que constituyen el lote efectivo desactivando el cálculo de gradientes, minimizando así el consumo de memoria.

2. **Cálculo de la pérdida global:** Se ensamblan dichas representaciones en un único bloque denso, incorporando la totalidad de las muestras negativas reales del lote efectivo completo y se evalúa la función de pérdida final, retropropagando los gradientes exclusivamente hasta los propios *embeddings* ensamblados.

3. **Acumulación de gradientes por micro-lote:** Se ejecuta nuevamente la pasada hacia adelante (*forward*) de CLIP micro-lote a micro-lote, esta vez con los gradientes activos, inyectando sobre cada uno el gradiente precalculado mediante la función `torch.autograd.backward` empleando tensores de gradiente externos.

Gracias a este flujo, el proceso optimiza el modelo aprovechando la señal contrastiva del lote

efectivo completo sin necesidad de mantener retenido en memoria el grafo computacional global de CLIP.

Inferencia nativa y ranking Top-K.

Durante la fase de recuperación, la simulación de inferencia toma como puntuación final de *ranking* la métrica S_{outfit} (correspondiente al nivel de abstracción más alto), devolviendo los K *outfits* con mayor similitud semántica frente a la consulta de entrada. Coincidiendo con la metodología planteada en el apartado 4.4, la evaluación se articula bajo dos criterios con propósitos complementarios:

- **Criterio multi-positivo:** Considera válida la recuperación de cualquiera de los *outfits* que compartan grupo textual con la consulta. Se adopta como métrica principal para la comparación interna entre las arquitecturas desarrolladas en este trabajo.
- **Criterio estricto:** Exige la coincidencia exacta con el identificador del *outfit* original. Este esquema se aproxima al protocolo de evaluación estándar reportado por Jang et al. (2023), reservándose para la comparación directa frente a sus cifras de *Recall@K*.

Tabla 7.7: Configuración de hiperparámetros para la arquitectura CLIP4Outfit.

Hiperparámetro	**Valor**
Backbone CLIP	`openai/clip-vit-base-patch32` (ViT-B/32)
α (peso $\mathcal{L}_{\text{style}}$)	0,3
β (peso $\mathcal{L}_{\text{outfit}}$)	0,3
p (ponderación WTI)	0,2
K_o / K_t (ratio de centroides de estilo)	$\lceil N/3 \rceil$ / $\lceil N/6 \rceil$
Escala de temperatura (τ_{scale})	100 (fija)
Optimizador	Adam
Tasa de aprendizaje	5×10^{-7}
Planificador de tasa de aprendizaje (*Scheduler*)	Cosine annealing
Tamaño de lote efectivo (vía *GradCache*)	50
Épocas máximas / Paciencia	20 / 5
Criterio de parada temprana (*Early stopping*)	Recall@5 (Validación, multi-positivo)

7.5. Proceso de Entrenamiento, Convergencia y Reproducibilidad

7.5.1. Configuración del Entrenamiento e Hiperparámetros

Sobre la base de las configuraciones de hiperparámetros detalladas de forma individual para cada arquitectura en los apartados 7.3 y 7.4.2, y en coherencia con las condiciones

comunes fijadas en el diseño experimental en el apartado 6.2, esta sección sistematiza las decisiones transversales del proceso de optimización compartidas por SCE-Net, HGNN, Recommendation Engine y CLIP4Outfit. El módulo de Text-to-Outfit basado en Recommendation Engine (apartado 7.4.1), al reutilizar directamente la infraestructura de entrenamiento ya documentada para este último, no constituye una configuración transversal adicional a sistematizar en este apartado; sus resultados de convergencia propios se retoman de forma puntual en el apartado 7.5.3.

Estrategia de optimización y tasas diferenciadas

Los cuatro modelos emplean algoritmos de la familia de optimización adaptativa por momentos: Adam estándar para SCE-Net, HGNN y CLIP4Outfit, y AdamW para *Recommendation Engine*. Esta última elección es coherente con la práctica en arquitecturas basadas en Transformer, donde el desacoplamiento de la regularización L_2 respecto al gradiente adaptativo evita la atenuación prematura de los pesos en capas de atención.

En el caso de SCE-Net, el optimizador se parametriza con dos grupos de parámetros con tasas de aprendizaje diferenciadas:

$$\eta_{\text{backbone}} = 2{,}5 \times 10^{-5}, \quad \eta_{\text{other}} = 2{,}5 \times 10^{-4} \tag{7.10}$$

asignando una tasa un orden de magnitud inferior al extractor visual preentrenado (*backbone*) respecto a los módulos de nueva creación (máscaras de condición y rama de ponderación). Esta estrategia previene que la retropropagación inicial desestabilice las representaciones visuales consolidadas al ajustar pesos inicializados aleatoriamente.

CLIP4Outfit, en cambio, no incorpora ninguna diferenciación de tasas de aprendizaje entre el *backbone* CLIP y los nuevos módulos de ponderación WTI y de agregación por estilo/conjunto (apartado 7.4.2): la totalidad de los parámetros del modelo, tanto los ya preentrenados como los inicializados aleatoriamente, se optimizan bajo una única tasa común, $\eta = 5 \times 10^{-7}$. Esta decisión no responde a un criterio propio de este trabajo, sino a la especificación literal de la sección 4.2 de Jang et al. (2023); su viabilidad práctica se apoya, en cualquier caso, en la magnitud extremadamente reducida de dicha tasa, que por sí misma limita el riesgo de desestabilización de la representación visual preentrenada sin necesidad de una diferenciación explícita por grupo de parámetros.

Planificación de la tasa de aprendizaje (*Learning Rate Scheduling*)

SCE-Net y *CLIP4Outfit* son las únicas arquitecturas que incorporan un planificador explícito de la tasa de aprendizaje, si bien de naturaleza distinta entre ambas. *SCE-Net* emplea `ReduceLROnPlateau`, configurado con un factor de reducción $\gamma = 0{,}1$ y una paciencia de 3 épocas sobre la pérdida de validación, un planificador reactivo que solo interviene ante un

estancamiento observado en la métrica. *CLIP4Outfit*, en cambio, emplea un planificador de tipo *cosine annealing*, el cual reduce la tasa de aprendizaje de forma continua y predecible a lo largo de la totalidad de los pasos de entrenamiento previstos, con independencia de la evolución concreta de la pérdida de validación, un planificador proactivo, en sintonía con la especificación original y coherente con el propio criterio de fidelidad al artículo original señalado previamente.

HGNN y *Recommendation Engine* mantienen una tasa fija ($\eta = 1 \times 10^{-3}$ y $\eta = 5 \times 10^{-5}$, respectivamente) durante todo el ciclo. Esta decisión responde a la rápida dinámica de convergencia observada en estos dos últimos modelos, los cuales alcanzan su óptimo local en un número reducido de épocas (apartado 8.3.1), dejando escaso margen operacional para la intervención útil de un planificador dinámico.

Criterio de parada anticipada (*Early Stopping*)

Los cuatro modelos implementan un mecanismo de parada anticipada con una paciencia de $P = 5$ épocas consecutivas sin mejora sobre un máximo de 50 épocas, a excepción de *CLIP4Outfit*, que acota el entrenamiento a un máximo de 20 épocas. No obstante, difieren de forma justificada en la métrica de supervisión:

- **SCE-Net:** Monitoriza la pérdida total de validación $\mathcal{L}_{\text{total}}$, que integra la pérdida de tripletas junto con los cuatro términos de regularización definidos en el apartado 7.3.1, garantizando que el punto de control seleccionado preserve la estabilidad del espacio latente y no únicamente la separación del margen de entrenamiento.
- **HGNN y Recommendation Engine:** Monitorizan de forma conjunta el AUC de compatibilidad y la exactitud (*accuracy*) en la tarea FITB, incrementando el contador de paciencia únicamente si ninguna de las dos métricas muestra mejora respecto a su óptimo histórico.
- **CLIP4Outfit:** Monitoriza una única métrica, el *Recall@5* bajo el criterio multi-positivo (apartado 7.4.2).

En todos los casos, al activarse la parada anticipada, se restauran automáticamente los pesos correspondientes al mejor punto de control (*best checkpoint*) y no los de la última época ejecutada.

Gestión de memoria y precisión computacional

Como se justificó en el apartado 7.1.1, SCE-Net y *Recommendation Engine* combinan precisión mixta automática (AMP mediante `float16`) con recorte de norma de gradiente (*gradient clipping* fijado en $1{,}0$). Adicionalmente, *Recommendation Engine* aplica acumulación de gradientes con un factor de 4 pasos sobre micro-lotes de $B_{\text{micro}} = 8$ para alcanzar un lote efectivo de $B_{\text{eff}} = 32$. *CLIP4Outfit*, por su parte, prescinde del recorte de gradiente y recurre

en su lugar a *GradCache* para sostener un lote efectivo de $B_{\text{eff}} = 50$ sin incurrir en su coste de memoria íntegro. *HGNN* prescinde de estas técnicas dado que no requiere el ajuste fino de ningún extractor visual denso, manteniendo un consumo de *VRAM* significativamente inferior.

Tolerancia a fallos e infraestructura

De forma complementaria al guardado del mejor modelo, la canalización de entrenamiento de SCE-Net programa la persistencia periódica de un punto de control (*checkpoint*) completo cada 5 épocas de manera incondicional. Esta medida responde al elevado coste computacional por época que presenta SCE-Net frente al resto de arquitecturas de este trabajo, dotando al flujo de trabajo de tolerancia ante interrupciones imprevistas en el entorno de ejecución y permitiendo la reanudación del proceso sin pérdida de estado.

7.5.2. Diagnóstico de Incidencias y Decisiones de Diseño

Los incidentes de mayor calado durante el proceso de entrenamiento se concentraron en SCE-Net, al ser la primera arquitectura desarrollada en este trabajo y servir de base para la consolidación del *pipeline* de datos común descrito en el apartado 7.2. Los diagnósticos específicos de HGNN y *Recommendation Engine*, de naturaleza primariamente arquitectónica, se documentan de forma individual previamente en sus respectivos apartados 7.3.2 y 7.3.3. El módulo de Text-to-Outfit presenta, a su vez, un error de naturaleza análoga a los de esta sección, que afecta también a *CLIP4Outfit* y se documenta en el apartado 7.4.1.

Selección y depuración del conjunto de datos.

Durante la construcción del canal de datos, los ficheros de definición de *outfits* procesados para SCE-Net correspondían inadvertidamente a la partición *disjoint* ($16{,}644$ conjuntos de entrenamiento) en lugar de la partición *nondisjoint* ($53{,}306$ conjuntos) fijada como estándar en este trabajo. El origen del fallo residía en la estructura del respositorio empleado[3]: la carpeta contenedora se encontraba etiquetada como `nondisjoint`, pero sus ficheros internos contenían la versión restringida.

El síntoma que permitió diagnosticar esta discrepancia fue una exactitud (*accuracy*) en la tarea FITB anómalamente reducida ($\approx 38\,\%$), sensiblemente inferior al $61{,}6\,\%$ reportado por Vasileva et al. (2018). Al auditar el volumen real de datos procesados, se confirmó que el modelo se entrenaba sobre apenas el $31\,\%$ del corpus previsto. Para solventar esta limitación, se regeneró la totalidad de los datos consumidos directamente desde el repositorio oficial de Hugging Face, garantizando la trazabilidad y la correspondencia entre la partición asignada y su volumen real. No obstante, al no contener ficheros de imagenes el nuevo repositorio, se mantuvieron los descargados anteriormente de la plataforma de Kaggle ahora descontinuados.

[3] https://github.com/xthan/polyvore-dataset

Estrategia de muestreo de tripletas y restricción categórica

Un segundo incidente afectaba al módulo de construcción de tripletas (`build_triplets`): la implementación inicial seleccionaba el ítem negativo compartiendo categoría semántica con el ítem ancla, cuando la especificación original de Vasileva et al. (2018) exige que el negativo comparta categoría estricta con el ítem positivo. Al evaluar la tasa de coincidencia categórica entre positivo y negativo sobre las tripletas generadas, solo el $4{,}7\,\%$ cumplía dicha restricción.

Con negativos pertenecientes a categorías mayoritariamente distintas a la del positivo, la tarea de discriminación se tornaba trivial: el modelo aprendía a separar las muestras apoyándose en diferencias categóricas de bajo nivel sin extraer nociones reales de compatibilidad estilística. Esto provocaba un descenso prematuro de la pérdida de entrenamiento ($\mathcal{L}_{\text{triplet}} \to 0$ en las primeras épocas) derivado del memorizado de pares de fácil resolución. Tras corregir la lógica de muestreo, se verificó una tasa de coincidencia categórica positivo-negativo del $100\,\%$, mientras que la coincidencia ancla-negativo se redujo a un valor residual del $3{,}8\,\%$, representativo de la coocurrencia natural de categorías dentro de un mismo *outfit*.

Dinámica de pérdidas auxiliares: colapso de condiciones y pérdida VSE

Durante la evaluación de los términos de regularización de la architectura SCE-Net 7.3.1, se analizó la función de alineación multimodal VSE propuesta por Vasileva et al. (2018) para vincular las representaciones visuales y textuales de cada ítem. Empíricamente, la similitud del coseno entre ambos subespacios se mantuvo estancada en valores cercanos a $0{,}06$ (ortogonalidad práctica), evidenciando una nula transferencia de conocimiento entre modalidades.

El diagnóstico analítico reveló que las características textuales extraídas mediante *Fisher Vectors* presentaban una distancia media inter-categoría de tan solo $0{,}031$, resultando insuficientemente discriminativas para guiar el aprendizaje del espacio visual. Ante la imposibilidad de lograr una alineación semántica útil y con el fin de optimizar el tiempo de cómputo por época, se determinó suprimir el término $\mathcal{L}_{\text{VSE}}$ de la función de pérdida global. Esta decisión se justifica metodológicamente como una limitación inherente a los descriptores textuales clásicos frente a arquitecturas modernas de lenguaje, y no como un defecto de implementación.

7.5.3. Dinámica de Convergencia y Estabilidad del Modelo

Esta sección analiza la dinámica de convergencia durante el entrenamiento de los modelos implementados a partir de la evolución de sus curvas de pérdida y métricas de validación, tras la subsanación de las incidencias descritas en el apartado anterior.

HGNN y *Recommendation Engine* exhiben la dinámica característica de una optimización estocástica adecuadamente regularizada: un descenso pronunciado de la función de pérdida durante las épocas iniciales, seguido de una fase de convergencia asintótica, con una brecha

de generalización entre los conjuntos de entrenamiento y validación que se mantiene en márgenes moderados a lo largo del proceso, gracias al conjunto de términos de regularización y mecanismos de control específicos de cada arquitectura (detallados individualmente en los apartados 7.3 y 7.4).

SCE-Net y *CLIP4Outfit* constituyen, en cambio, excepciones relevantes a este patrón general, si bien de gravedad distinta entre sí: la primera presenta un sobreajuste moderado, parcialmente recuperado hacia el final del entrenamiento; la segunda exhibe una divergencia más persistente y sostenida, sin señales de recuperación dentro del número de épocas ejecutadas. Ambos casos se detallan de forma individual más adelante en esta misma sección.

Diagnostico de entrenamiento y sobreajuste en SCE-Net

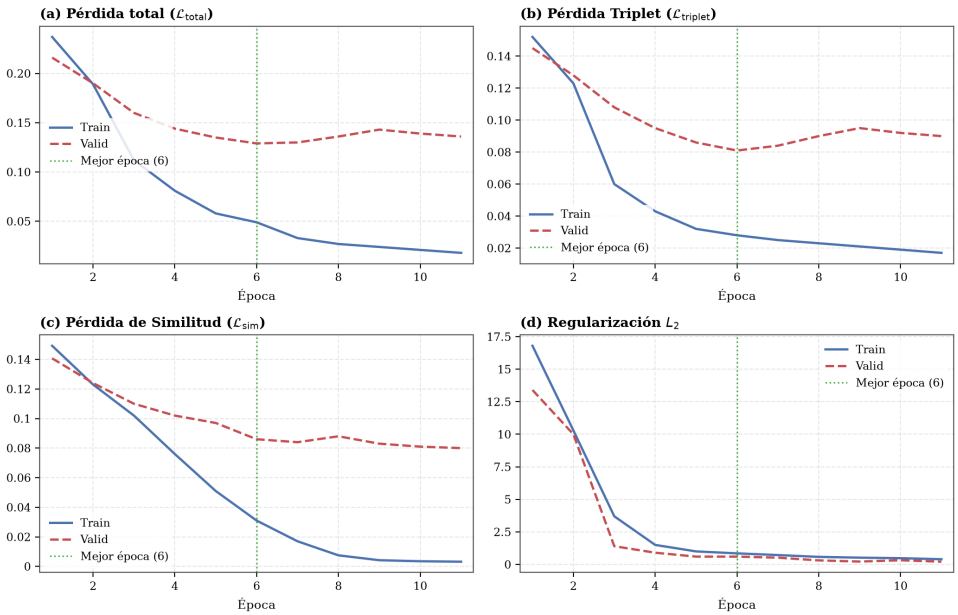


Figura 7.4: Evolución de las componentes de optimización de SCE-Net durante 11 épocas de entrenamiento: (a) pérdida total combinada ($\mathcal{L}_{\text{total}}$); (b) pérdida *Triplet* ($\mathcal{L}_{\text{triplet}}$); (c) término de regularización L_1 sobre las máscaras de atención por subespacio; y (d) término de regularización L_2 sobre los *embeddings*. La línea vertical punteada verde señala la época 6, identificada mediante *early stopping* como el punto de óptima generalización en el conjunto de validación.

El análisis de las curvas de optimización de la arquitectura *SCE-Net* (figura 7.4) revela un patrón de sobreajuste moderado pero estructuralmente informativo a partir del punto medio del entrenamiento, cuya caracterización resulta fundamental para comprender los límites de generalización del modelo.

Las cuatro componentes de la función de pérdida total ($\mathcal{L}_{\text{total}}$, la pérdida *Triplet* $\mathcal{L}_{\text{triplet}}$, la pérdida de similitud $\mathcal{L}_{\text{sim}}$ y la regularización L_2 sobre los *embeddings*) exhiben una dinámica

de convergencia diferenciada pero internamente coherente. Como se aprecia en los paneles (a) y (b), tanto la pérdida total como la componente *Triplet* sobre el conjunto de validación alcanzan su mínimo global en la época 6, identificada por el mecanismo de parada anticipada (*early stopping*). A partir de dicho ciclo, ambas métricas sufren un repunte moderado hasta la época 9, estabilizándose en una meseta en la fase final del proceso. Por el contrario, los términos de regularización L_1 y L_2 experimentan un descenso monótono y asintótico continuado a lo largo de las 11 épocas, lo que confirma que el régimen de penalización sobre la norma de los parámetros opera conforme al diseño estipulado, sin sufrir de estancamiento ni de inestabilidad numérica.

Para determinar si este sobreajuste prematuro fue inducido por un colapso degenerado en la asignación de condiciones, es decir, que la red concentrara toda la capacidad expresiva en una o dos condiciones dominantes de forma no equitativa—, se auditó la distribución de pesos entre las $M = 5$ máscaras de condición en la época óptima (6), cuyos valores se resumen en la tabla 7.8.

Tabla 7.8: Distribución de pesos asignados a las $M = 5$ máscaras de condición en la época óptima (6).

Máscara de Condición (m)	**Peso de Atención Ponderado**
Condición 1	$0{,}1285$
Condición 2	$0{,}1714$
Condición 3	$0{,}1482$
Condición 4	$0{,}2528$
Condición 5	$0{,}2990$

La distribución resultante presenta una entropía de Shannon de $H \approx 1{,}558$ nats, valor extraordinariamente próximo al máximo teórico para una distribución uniforme con cinco dimensiones ($H_{\text{máx}} = \ln(5) \approx 1{,}609$). Esta alta entropía, muy por encima del umbral crítico que desencadenaría la activación de la penalización por diversidad $\mathcal{L}_{\text{div}}$ (apartado 7.3.1), confirma que el mecanismo de enmascaramiento por condición preserva la diversidad funcional para la que fue diseñado, sin colapsar.

En consecuencia, se concluye que el sobreajuste observado no deriva del módulo de atención por subespacios, sino de una saturación en la capacidad de generalización del *backbone* visual compartido bajo ajuste fino completo (*full fine-tuning*). Esta dinámica sugiere un desequilibrio entre la elevada parametrización de la red extractora de características y la variedad efectiva de tripletas complejas disponibles tras la fase de depuración del conjunto de datos (apartado 7.5.2).

Comportamiento y regularización en HGNN

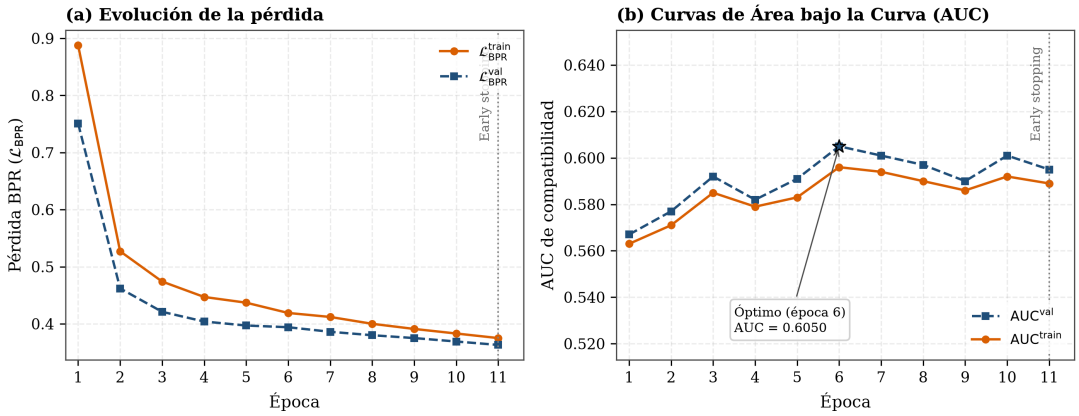


Figura 7.5: Dinámica de entrenamiento y validación de la arquitectura *HGNN*: (a) curvas de pérdida por ordenación ($\mathcal{L}_{\text{BPR}}$) y (b) métrica de compatibilidad AUC. La línea vertical discontinua indica el punto de parada anticipada (*early stopping*).

En la figura 7.5 se ilustra la evolución temporal de las métricas de entrenamiento y validación durante las 11 épocas de optimización de la arquitectura *HGNN*. Como se observa en el panel (a), la función de pérdida BPR sobre el conjunto de entrenamiento ($\mathcal{L}_{\text{BPR}}^{\text{train}}$) experimenta un descenso pronunciado en las tres primeras épocas, pasando de $0{,}890$ a $0{,}475$, para posteriormente estabilizarse en una trayectoria de convergencia asintótica hasta alcanzar un valor de $0{,}376$ en la undécima época.

Sin embargo, la métrica del área bajo la curva ROC (panel b) no refleja esta misma tendencia decreciente e indefinida: mientras que la pérdida BPR sigue reduciéndose, el AUC^{val} alcanza su máximo absoluto $(0{,}6050)$ en la sexta época, a partir de la cual entra en una fase de estancamiento con leves oscilaciones.

El análisis comparativo entre ambas curvas de pérdida revela una propiedad estructural relevante: la pérdida de validación se mantiene sistemáticamente por debajo de la de entrenamiento a lo largo de todo el proceso. Este patrón resulta coherente con dos factores complementarios:

1. **Efecto de la regularización por *dropout*:** La tasa de *dropout* ($p = 0{,}1$, apartado 7.3.2), activa únicamente durante la fase de optimización y desactivada en la evaluación, penaliza artificialmente la pérdida de entrenamiento al introducir ruido estocástico en las representaciones latentes, un obstáculo del que carece el modelo durante la inferencia sobre el conjunto de validación.

2. **Dinámica de cálculo acumulativo de la pérdida:** La pérdida de entrenamiento se calcula como el promedio flotante sobre la totalidad de los *batches* a lo largo de la época, reflejando el estado del modelo en fases previas a su reajuste completo, mientras que la

pérdida de validación se computa de forma estática una sola vez al cierre de la época, sobre los pesos ya íntegramente actualizados.

Paralelamente, el término de regularización para la consistencia visual-textual ($\mathcal{L}_{\text{con}}$) muestra una tasa de decrecimiento sustancialmente más acentuada que la propia pérdida de ordenación (*ranking*), reduciéndose de $26{,}5$ a $1{,}2$ en apenas 8 épocas. Este comportamiento empírico confirma la hipótesis planteada en la sección 7.3.2 sobre el riesgo de colapso hacia soluciones degeneradas, justificando formalmente la adopción de un coeficiente ponderador conservador ($\lambda_{\text{con}} = 0{,}01$).

Evolución multitarea en Recommendation Engine

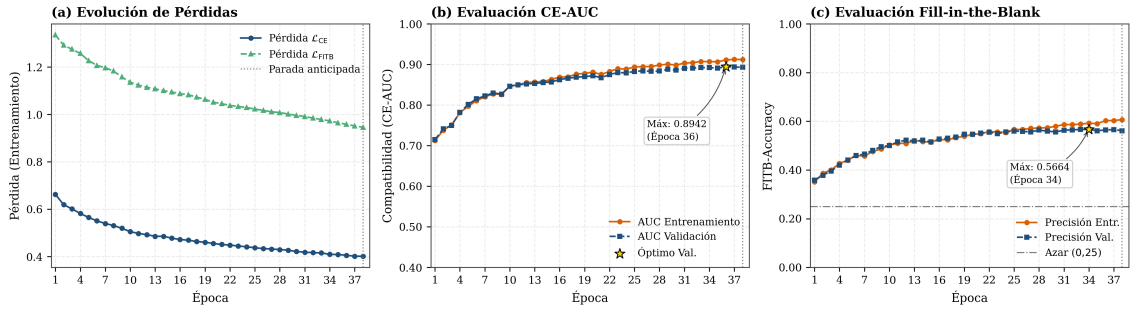


Figura 7.6: Dinámica de entrenamiento y validación de la arquitectura *Recommendation Engine*: (a) evolución de las pérdidas de entrenamiento ($\mathcal{L}_{\text{CE}}$ y $\mathcal{L}_{\text{FITB}}$); (b) métrica de compatibilidad CE-AUC sobre las particiones de entrenamiento y validación; y (c) precisión en la tarea de completado de conjuntos (*Fill-in-the-Blank*), comparada frente al umbral aleatorio de referencia ($0{,}25$). La línea vertical discontinua señala el punto de parada anticipada (*early stopping*).

Respecto a la arquitectura *Recommendation Engine*, tanto el área bajo la curva (CE-AUC) en la predicción de compatibilidad como la exactitud (*accuracy*) en la tarea FITB muestran una progresión ascendente sostenida sin signos de sobreajuste limitante durante la mayor parte del entrenamiento (figura 7.6). El CE-AUC de validación evoluciona desde $0{,}7145$ en la época inicial hasta un máximo de $0{,}8942$ en la época 36. Por su parte, la *accuracy* en FITB se incrementa desde $0{,}3584$ hasta alcanzar un óptimo de $0{,}5664$ en la época 34. Las curvas de entrenamiento y validación discurren con notable proximidad, confirmando que ambas tareas se optimizan de forma conjunta y sinérgica, de acuerdo con el diseño multitarea planteado.

Esta capacidad de generalización se confirma al contrastar estos resultados con la evaluación sobre el conjunto de test (sección 8): el CE-AUC ($0{,}8956$) y la precisión FITB ($0{,}5585$) obtenidos en test se sitúan en un rango equivalente a los máximos de validación, descartando que el *checkpoint* seleccionado haya sobreajustado a la partición de validación.

Únicamente hacia el final del proceso emerge una ligera divergencia en la tarea FITB (figura 7.6c): en la última época (época 38), la precisión de entrenamiento alcanza un $0{,}6064$

frente al óptimo de validación de $0{,}5664$ —una brecha de $4{,}0$ puntos porcentuales—. Este comportamiento es consecuencia directa de la prolongación del entrenamiento debida al criterio de parada estricto, si bien el desfase resulta marginal y no compromete la estabilidad ni la capacidad de generalización final del modelo.

Convergencia en Text-to-Outfit sobre Recommendation Engine

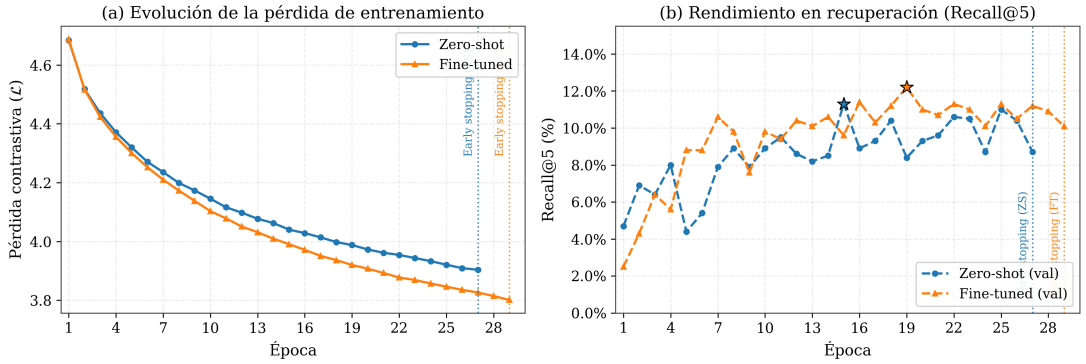


Figura 7.7: Comparativa de la dinámica de entrenamiento y validación de la arquitectura *CLIP4Outfit* bajo la configuración de *backbone* base (*Zero-shot*) frente a la variante adaptada al dominio de moda (*Fine-tuned*): (a) evolución de la pérdida contrastiva de entrenamiento; y (b) rendimiento de recuperación en validación medido mediante Recall@5. La estrella (⋆) señala la posición del óptimo alcanzado por cada modelo y las líneas verticales discontinuas indican el momento de parada anticipada (*early stopping*).

La dinámica de convergencia del módulo de *Text-to-Outfit* implementado sobre el motor de *Recommendation Engine* presenta un patrón cualitativo común en sus dos configuraciones de partida, si bien se observan diferencias sustanciales en la evolución temporal y el rendimiento máximo alcanzado. Como se ilustra en la figura 7.7(a), la función de pérdida contrastiva de entrenamiento describe un descenso monótono y estable en ambas variantes, lo que evidencia una optimización libre de oscilaciones drásticas en los gradientes.

Por su parte, la métrica que rige el criterio de parada anticipada (*early stopping*), el *Recall*@5 sobre el conjunto de validación (figura 7.7(b)), exhibe una oscilación inter-época apreciable. Esta variabilidad es un comportamiento esperado en el problema de recuperación multimodal, motivado por los valores absolutos reducidos de la métrica y el número limitado de *outfits* estrictamente positivos asociados a cada consulta textual en cada lote. La naturaleza ruidosa de esta curva justifica la necesidad de fijar un margen amplio de paciencia (10 épocas) para evitar paradas prematuras.

A pesar de compartir esta dinámica de fondo, la comparación entre ambas variantes revela un comportamiento diferenciado de gran interés metodológico:

- **Variante base (*Zero-shot*):** Muestra un arranque inicial más rápido en las primeras épocas debido a la estabilidad de sus pesos congelados, alcanzando su punto óptimo

de *Recall*@5 ($11,3\,\%$) en la época 17. El entrenamiento se detiene en la época 27 tras agotar las 10 épocas sin mejora (línea discontinuas azul).

- **Variante con adaptación (*Fine-tuned*):** Como resulta habitual en procesos de ajuste fino sobre representaciones complejas, esta variante registra un rendimiento ligeramente inferior en las épocas iniciales. Este desacople temporal responde al reajuste progresivo entre los pesos del *backbone* y las capas superiores. Sin embargo, una vez superada esta fase inicial, la especialización en el dominio de la moda permite al modelo superar sustancialmente a la variante base, alcanzando un valor óptimo de *Recall*@5 de $12,2\,\%$ en la época 19 ($\star$ naranja) y finalizando en la época 29 (línea discontinuas naranja).

En conclusión, los resultados confirman que, aunque la adaptación de dominio previa requiere una fase inicial de reajuste en los espacios de proyección, permite construir un subespacio de compatibilidad semántica más preciso y con mayor capacidad de generalización final.

Sobreajuste y disociación métrica en CLIP4Outfit

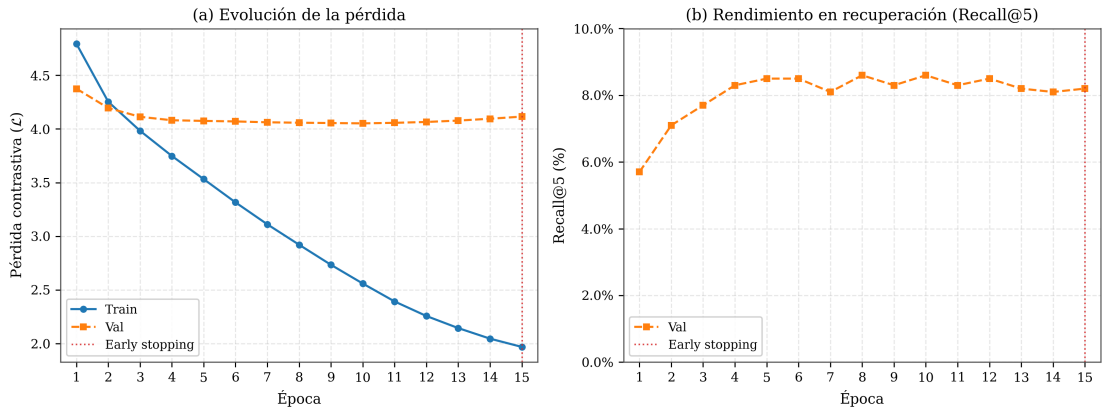


Figura 7.8: Curvas de optimización y evaluación durante el entrenamiento de CLIP4Outfit: (a) evolución de la pérdida contrastiva ($\mathcal{L}$) sobre los conjuntos de entrenamiento (línea azul continua) y validación (línea naranja discontinua); (b) métrica de recuperación *Recall@5* ($\%$) sobre el conjunto de validación. La línea punteada roja vertical indica la parada anticipada (*early stopping*) en la época 15, tras agotar la paciencia de $P = 5$ épocas respecto al máximo alcanzado en *Recall@5*.

El entrenamiento de tres niveles de CLIP4Outfit mediante *GradCache* (apartado 7.4.2) se extendió durante 15 épocas, activándose la parada anticipada (*early stopping*) al agotarse la paciencia de 5 épocas consecutivas sin registrar mejoras en la métrica *Recall@5* de validación (figura 7.8).

La pérdida contrastiva sobre el conjunto de entrenamiento desciende de forma pronunciada y sustained a lo largo de todo el proceso ($4{,}79$ a $1{,}97$), sin mostrar indicios de estan-

camiento hasta la última época. Por el contrario, la pérdida de validación alcanza un mínimo en torno a las épocas 9-10 ($\mathcal{L}_{\text{val}} \approx 4{,}05$) e inicia a partir de ahí un leve repunte hacia el final del entrenamiento. Esta divergencia progresiva entre ambas curvas evidencia un régimen de sobreajuste persistente, fenómeno coherente con la ausencia de recorte de gradiente (*gradient clipping*) comentada en el apartado 7.5.1 y con la elevada capacidad del modelo, que combina el ajuste fino de CLIP con tres módulos adicionales de agregación semántica (apartado 7.4.2).

Por su parte, la métrica *Recall@5* de validación, que gobierna el criterio de parada y guardado de *checkpoints*, refleja esta dinámica solo de forma parcial: tras un crecimiento sostenido hasta la época 5, entra en una meseta oscilante en torno al 8,0 ‰-8,6 ‰, sin exhibir una tendencia de degradación acusada. Sin embargo, el análisis del *MedianRank* de validación, revela una dinámica más crítica: tras alcanzar su mejor posición (257) en la época 4, la mediana del rango empeora de forma prácticamente monótona hasta alcanzar la posición 305 en la época 15 (un deterioro acumulado de casi 50 posiciones). Esta disociación métrica indica que, si bien la probabilidad de situar el conjunto correcto en los primeros cinco puestos permanece acotada, el rango típico del acierto en la totalidad del conjunto de prueba se degrada progresivamente a medida que se prolonga el entrenamiento.

7.5.4. Protocolo de Reproducibilidad y Control de Aleatoriedad

Sin un control estricto sobre las fuentes de aleatoriedad, una misma arquitectura puede registrar variaciones métricas significativas entre ejecuciones independientes. Esta variabilidad dificulta discernir si una diferencia en el rendimiento responde a un avance en el diseño arquitectónico o a un artefacto del proceso de optimización estocástica. La práctica estándar para mitigar este fenómeno consiste en fijar una semilla estática uniforme sobre todos los generadores de números aleatorios (*Pseudo-Random Number Generators*, PRNG) que intervienen en el *pipeline*:

- `random.seed`: Modulo nativo de Python empleado en las operaciones de muestreo de datos, como la construcción dinámica de tripletas y la selección de ítems negativos.
- `numpy.random.seed`: Generador utilizado en las transformaciones vectorizadas y el preprocesamiento numérico de metadatos.
- `torch.cuda.manual_seed_all` y `torch.backends.cudnn.deterministic = True`: Motor de PyTorch de los que dependen la inicialización de los pesos de la red, el orden de mezcla (*shuffling*) en el `DataLoader` y la aplicación de las máscaras de *dropout*, tanto en CPU como en aceleración GPU.

Fijar estos tres componentes de forma conjunta garantiza la replicabilidad exacta de los experimentos bajo condiciones idénticas de código y entorno de ejecución.

En el marco de esta investigación, este protocolo de control se implementó de forma progresiva a lo largo del desarrollo experimental.

SCE-Net, al ser la primera arquitectura prototipada, no incorporó la fijación explícita de semillas aleatorias en su *pipeline* de entrenamiento. Por consiguiente, los resultados reportados para esta red proceden de una ejecución única, lo que se reconoce explícitamente como una limitación metodológica del presente trabajo: la inicialización de los parámetros, el muestreo de tripletas y la regularización por *dropout* exhiben cierta variabilidad estocástica entre ejecuciones.

En contraposición, las arquitecturas desarrolladas con posterioridad (HGNN y *Recommendation Engine*) integraron un protocolo rígido de fijación de semilla común (*seed* $= 42$) sobre la totalidad de los generadores citados. Aunque persistieron pequeñas variaciones no deterministas no controladas inherentes a las operaciones atómicas de la librería CUDA en la GPU (tales como las operaciones `torch.bmm` o las transformaciones convolucionales no deterministas de *cuDNN*), esta medida garantiza una documentación honesta, trazable y altamente reproducible de las métricas obtenidas.

Resultados Experimentales y Discusión

8

El propósito fundamental de este capítulo es evaluar de manera sistemática, empírica y cuantitativa las arquitecturas desarrolladas para la recomendación multimodal de moda y el análisis de compatibilidad indumentaria. Tras la fundamentación teórica (capítulo 4) y la formulación metodológica de los modelos (capítulo 5), esta fase experimental aporta la evidencia empírica necesaria para verificar el grado de consecución de los objetivos generales del trabajo.

Específicamente, se pondera la capacidad de discriminación estilística visual de las redes implementadas y se analiza la efectividad del alineamiento multimodal en tareas de búsqueda semántica guiada por texto libre. El diseño experimental se adscribe al protocolo riguroso articulado en el apartado 6.3, sustentado sobre el *benchmark* estándar de la literatura *Polyvore Outfits* en su partición *nondisjoint*.

La discusión y estructuración del análisis de resultados se articula a lo largo de este capítulo conforme al siguiente esquema:

- La **sección 8.1** formaliza el protocolo experimental de evaluación, precisando las tareas estandarizadas del dominio (*Fill-In-The-Blank* y *Compatibility Prediction*), las métricas de recuperación multimodal (*Recall@K* y *MRR*) y las especificaciones técnicas de la infraestructura informática de cómputo.
- La **sección 8.2** aborda la evaluación cuantitativa y cualitativa de las tres arquitecturas de compatibilidad consideradas (SCE-Net, HGNN y *Recommendation Engine*), contrastando sus métricas frente al estado del arte y examinando analíticamente sus casos límite (*edge cases*).
- La **sección 8.3** profundiza en la evaluación del módulo multimodal *Text-to-Outfit*, comparando el comportamiento cuantitativo y cualitativo de las dos aproximaciones diseñadas: la proyección sobre el espacio latente del *Recommendation Engine* y la arquitectura contrastiva jerárquica de tres niveles basada en CLIP4Outfit.
- La **sección 8.4** sintetiza las decisiones de diseño e ingeniería más determinantes del desarrollo, ofreciendo un diagnóstico de componentes, la justificación de la regularización de máscaras de condición y el análisis de las dinámicas del canal de datos.

- Finalmente, la **sección 8.5** evalúa la eficiencia computacional de las soluciones en términos de volumen de parámetros y latencia de inferencia en despliegue.

8.1. Protocolo Experimental y Métricas de Evaluación

La evaluación empírica de las arquitecturas desarrolladas en este trabajo se adscribe estrictamente al protocolo experimental y a las métricas formalizadas en el apartado 6.3. Por consiguiente, para la formulación teórica de las tareas de *Fill-In-The-Blank* (FITB) y *Compatibility Prediction*, así como de las métricas de recuperación semántica (*Recall*@K y MRR) empleadas en la validación del módulo *Text-to-Outfit*, se remite a dicha sección.

La totalidad de los resultados expuestos en este capítulo corresponde a los modelos definitivos. Las cuatro arquitecturas evaluadas: SCE-Net, HGNN, *Recommendation Engine* y CLIP4Outfit, han completado su ciclo de entrenamiento conforme a la configuración de hiperparámetros detallada en las secciones 7.3 y 7.4, constituyendo la evidencia empírica final del trabajo.

Las pruebas experimentales se ejecutaron en su totalidad sobre la infraestructura de hardware y software especificada en la sección 7.1.1. Del mismo modo, el protocolo de fijación de semillas aleatorias y las consideraciones sobre el control de reproducibilidad asociadas a cada modelo se corresponden con el diagnóstico analizado en el apartado 7.5.4. Sobre este marco homogéneo de evaluación, las siguientes secciones presentan el desempeño cuantitativo y cualitativo alcanzado en las tareas de compatibilidad visual (sección 8.2) y recuperación multimodal (sección 8.3).

8.2. Evaluación de Compatibilidad y Complementariedad

8.2.1. Comparación Cuantitativa y Estado del Arte

La tabla 8.1 sintetiza el desempeño cuantitativo alcanzado por las tres arquitecturas de compatibilidad sobre el conjunto de prueba de *Polyvore Outfits* (partición *nondisjoint*), contrastado directamente con las cifras reportadas en las respectivas publicaciones de referencia.

Tabla 8.1: Evaluación comparativa en el conjunto de prueba frente a los valores reportados en la literatura.

Modelo	**FITB (TFM)**	**FITB (Lit.)**	**AUC (TFM)**	**AUC (Lit.)**
SCE-Net	0,509	0,616	0,793	0,910
HGNN	0,637	0,390	0,602	0,760
Recommendation Engine	0,566	0,625	0,896	0,910

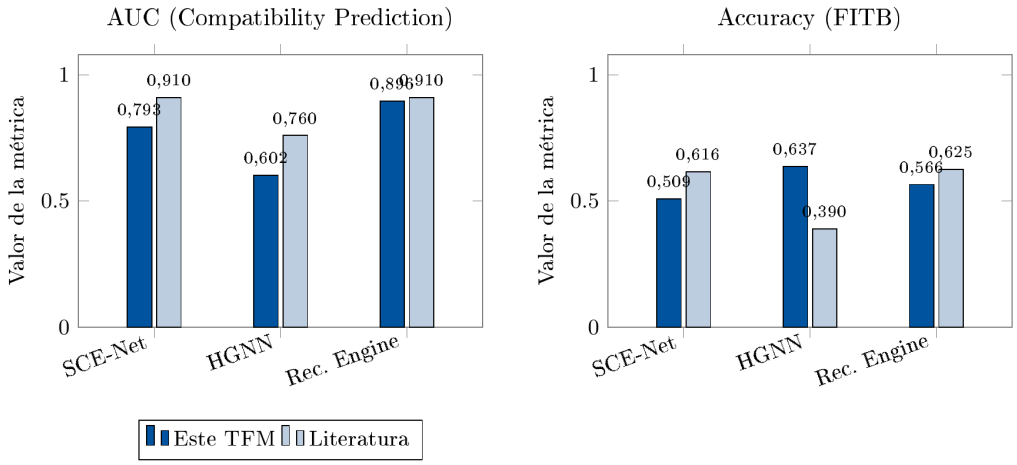


Figura 8.1: Comparativa de AUC (*Compatibility Prediction*) y *accuracy* (Fill-In-The-Blank) obtenidas frente a los valores reportados en la literatura de referencia, para las tres arquitecturas de compatibilidad evaluadas. Datos de la tabla 8.1.

Entre los modelos evaluados, *Recommendation Engine* es el que reproduce con mayor fidelidad los valores del estado del arte: su métrica AUC ($0{,}896$) se sitúa a apenas $1{,}4$ puntos porcentuales del valor de referencia reportado por Prato et al. (2020), mientras que la exactitud en FITB ($0{,}566$) registra un margen de desviación acotado de $5{,}9$ puntos.

Por su parte, SCE-Net presenta la mayor divergencia con respecto a la literatura, registrando una brecha de $10{,}7$ puntos en FITB y de $11{,}7$ puntos en AUC respecto a la propuesta de Tan et al. (2019).

Finalmente, el modelo HGNN exhibe un patrón claramente asimétrico: la exactitud en FITB ($0{,}637$) supera de forma sustancial el valor reportado por Gulati (2024) ($0{,}390$), mientras que el AUC ($0{,}602$) queda sensiblemente por debajo de dicha referencia ($0{,}760$).

Los resultados obtenidos admiten una lectura analítica fundamentada en los principios de diseño de cada red:

- **Fragilidad de las máscaras latentes en SCE-Net:** La brecha de rendimiento en SCE-Net evidencia la vulnerabilidad del mecanismo de máscaras de condición aprendidas de forma explícitamente latente (apartado 7.3.1). Pese a las cuatro intervenciones de estabilización introducidas sobre el diseño original, ausentes en la publicación de referencia, la dinámica de sobreajuste documentada en el apartado 7.5.3 confirma que este mecanismo resulta intrínsecamente más sensible a la configuración de entrenamiento que las aproximaciones de HGNN o *Recommendation Engine*, las cuales incorporan algún grado de supervisión categórica explícita.

  Este sobreajuste constituye, además, un argumento empírico determinante para cuestionar la capacidad de generalización del modelo ante patrones no vistos.

- **Comportamiento asimétrico en HGNN y efecto de escala del conjunto de datos:** Resulta de especial interés metodológico observar que el patrón de asimetria obtenido en esta investigación (FITB elevado y AUC reducido) es el inverso del reportado por Gulati (2024) para esta misma arquitectura (AUC elevado y FITB reducido; Tabla 8.1).

  Dado que dicho trabajo entrena y evalúa sobre un subconjunto sustancialmente más acotado que la partición *nondisjoint* completa utilizada aquí, del orden de 1.600 *outfits* frente a los 68.306 procesados en este TFM (apartado 7.3.2), se formula como hipótesis que la escala masiva y la diversidad categórica del conjunto de entrenamiento condicionan de manera crítica el compromiso (*trade-off*) de optimización entre ambas tareas.

- **Consistencia holística en Recommendation Engine:** A diferencia de HGNN, *Recommendation Engine* no muestra ningún tipo de compromiso asimétrico entre tareas. Este comportamiento es congruente con su mecanismo de atención sobre la totalidad del conjunto de ítems (apartado 7.3.3): al modelar simultáneamente las interdependencias entre todos los elementos del *outfit* sin descomponer el problema en comparaciones por pares ni sobreestructuras de categorías independientes, preserva por igual la precisión local y la coherencia global.

En conclusión, el análisis comparativo consolida a *Recommendation Engine* como la arquitectura más robusta y equilibrada del conjunto evaluado. Mientras que la estructura de hipergrafo en HGNN parece supeditar la coherencia global a la discriminación categórica local, y SCE-Net acusa una acusada dependencia de su hiperparametrización latente, el mecanismo de atención global demuestra una capacidad superior para capturar la compatibilidad multimodal en escenarios de recomendación real.

8.2.2. Análisis Cualitativo y Casos de Éxito

Más allá de las métricas agregadas presentadas en la tabla 8.1, la inspección de casos de estudio concretos permite examinar convergencias y divergencias de comportamiento entre las tres arquitecturas que las métricas globales no logran visibilizar por sí solas.

Patrones de consenso ante señales de alta coherencia.

Un primer escenario representativo corresponde a una instancia de FITB contextualizada en un conjunto de gala (compuesto por bolso de mano bordado, calzado de tacón plateado y pendientes con gema rosa). Ante los cuatro candidatos posibles, los tres modelos identifican de forma unánime el mismo vestido rosa como la respuesta correcta (figuras 8.2, 8.3 y 8.4).

Esta convergencia sugiere que, ante señales de compatibilidad cromática y de formalidad altamente explícitas, los tres paradigmas coinciden en su predicción, independientemente de sus diferencias en la representación latente. Un consenso análogo se observa en la tarea de *Compatibility Prediction*, donde tanto *Recommendation Engine* como SCE-Net clasifican

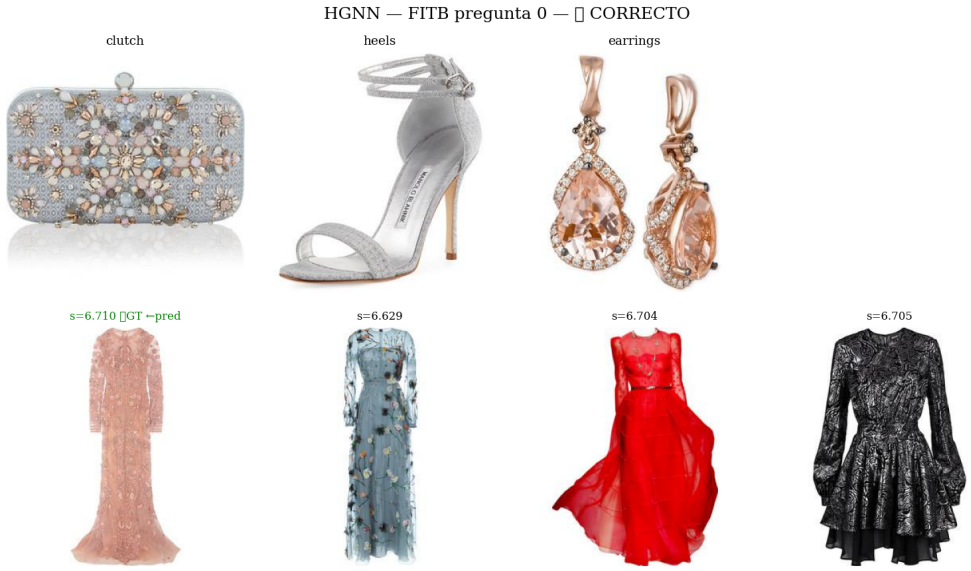


Figura 8.2: Ejemplo de acierto en la tarea *Fill-in-the-Blank* (FITB) mediante el modelo HGNN. Las puntuaciones asignadas a los candidatos (s) muestran una dispersión reducida entre la opción correcta y los distractores.



Figura 8.3: Ejemplo de acierto en la tarea *Fill-in-the-Blank* (FITB) mediante *Recommendation Engine*. El modelo asigna una puntuación significativamente más alta al candidato correcto ($s = 0{,}723$) frente a las alternativas.



Figura 8.4: Ejemplo de acierto en la tarea *Fill-in-the-Blank* (FITB) mediante SCE-Net. La distancia en el espacio latente (d) permite identificar de forma correcta la opción de referencia.

correctamente como verdadero positivo (*True Positive*) un mismo conjunto monocromático en tono amarillo (comprendido por bolso, calzado de cuña, pendientes y conjunto de top y falda; figuras 8.5 y 8.6).



Figura 8.5: Clasificación correcta de compatibilidad (*True Positive*) en *Recommendation Engine* para un conjunto monocromático amarillo ($score = 0{,}995$).

Márgenes de discriminación y confiabilidad de los clasificadores.

No obstante, el análisis de la distribución de las puntuaciones asignadas a los candidatos en una misma pregunta FITB pone de manifiesto diferencias dinámicas sustanciales entre modelos. Como se ilustra en el acierto de la figura 8.2, el modelo HGNN asigna puntuaciones extremadamente concentradas a los cuatro elementos candidatos (comprendidas en un rango estrecho de $6{,}629$ a $6{,}710$), lo que supone una dispersión relativa inferior al $2\,\%$.

En contraposición, *Recommendation Engine* (figura 8.3) exhibe una capacidad de discriminación significativamente mayor, separando con nitidez la opción correcta del resto de



Figura 8.6: Clasificación correcta de compatibilidad (*True Positive*) en SCE-Net para el mismo conjunto monocromático amarillo ($score = 0{,}282$).

distractores ($0{,}723$ frente a $0{,}266$, $0{,}007$ y $0{,}004$). Por su parte, SCE-Net muestra un comportamiento intermedio, con una dispersión moderada de las puntuaciones ($\sim 29\,\%$ en escenarios equivalentes).

Esta variabilidad en la amplitud del margen de decisión resulta plenamente coherente con la formulación matemática de cada mecanismo de puntuación:

- La función de puntuación en **HGNN** se deriva de un sumatorio agregado sobre la totalidad de los ítems del hipergrafo (apartado 7.3.2), lo que tiende a suavizar las diferencias puntuales entre candidatos.
- Las cabezas de salida de **Recommendation Engine** (apartado 7.3.3) incorporan proyecciones optimizadas explícitamente para la tarea de clasificación binaria/ranking, lo que favorece una asignación de probabilidad más polarizada y con mayor confianza en la predicción final.

8.2.3. Análisis de Errores y Casos Límite (*Edge Cases*)

Complementariamente a las métricas agregadas, el análisis cualitativo de fallos específicos permite localizar patrones de comportamiento sistemáticos y sesgos estructurales no detectables mediante evaluaciones globales.

Desacuerdos categóricos ante entradas idénticas.

El escenario de desacuerdo más ilustrativo corresponde a un par de prendas compuesto por un jersey de cuello alto en tono azul marino y unas medias altas de encaje negro, evaluado por HGNN y SCE-Net bajo la misma etiqueta de referencia (*ground truth* incompatible). Mientras que HGNN clasifica correctamente la combinación como un verdadero negativo (*score* $= 0{,}912$, situado por debajo de su umbral de corte; figura 8.7), SCE-Net incurre en un falso positivo sobre el mismo par exacto (*score* $= 0{,}317$, por encima de su umbral; figura 8.8).

Al evaluar la entrada bajo las mismas condiciones, esta divergencia aporta evidencia directa de un criterio intrínsecamente dispar entre ambas representaciones latentes, independiente de cualquier variabilidad en el muestreo de los datos.



Figura 8.7: Clasificación correcta de compatibilidad (*True Positive*) en SCE-Net para el mismo conjunto monocromático amarillo ($score = 0{,}282$).



Figura 8.8: Clasificación correcta de compatibilidad (*True Positive*) en SCE-Net para el mismo conjunto monocromático amarillo ($score = 0{,}282$).

Ambigüedad semántica compartida en tareas de FITB.

Un segundo patrón de fallo, en este caso transversal, se registra en una pregunta de FITB contextualizada en un conjunto asociado a zapatillas de ballet. Tanto *Recommendation Engine* como SCE-Net seleccionan erróneamente el mismo distractor (una falda plisada en tono rosa palo) en lugar de la prenda objetivo (una falda de tul), asignando puntuaciones extremadamente próximas entre la opción correcta y la elegida (figuras 8.9 y 8.10). La convergencia en el error y la escasa diferencia numérica entre las puntuaciones sugieren una ambigüedad semántica intrínseca al caso, dos prendas con gran afinidad en paleta cromática, silueta y textura, más que un fallo atribuible al diseño de una arquitectura particular.



Figura 8.9: Ejemplo de fallo en la tarea FITB mediante *Recommendation Engine*. El modelo selecciona el distractor de falda rosa palo ($s = 0{,}318$) frente al conjunto de referencia GT ($s = 0{,}310$) debido a la alta afinidad estilística y cromática de las prendas.

Sesgo de cardinalidad y magnitud de puntuación en HGNN.

Por último, el análisis comparativo de los cuatro escenarios de prueba en HGNN (figuras 8.11 a 8.12) revela una correlación directa entre la magnitud de la puntuación asignada y la cardinalidad (número de prendas) del conjunto evaluado, con independencia del acierto de la predicción:

- Los conjuntos de mayor dimensión (8–9 prendas) obtienen puntuaciones elevadas tanto en el acierto (*True Positive*: $22{,}037$) como en el error (*False Positive*: $21{,}492$).
- Los conjuntos reducidos (2–3 prendas) registran puntuaciones notablemente inferiores tanto en el acierto (*True Negative*: $0{,}912$) como en el error (*False Negative*: $2{,}981$).

Este fenómeno no se replica en *Recommendation Engine* ni en *SCE-Net*. Se formula como hipótesis que esta dependencia de la cardinalidad en *HGNN* deriva de la propia formulación de la laplaciana del hipergrafo (apartado 7.3.2): a mayor número de ítems, mayor es la



Figura 8.10: Ejemplo de fallo en la tarea FITB mediante SCE-Net en el mismo escenario. El modelo asigna menor distancia latente al distractor ($d = 0{,}267$) que a la prenda de referencia de tul rosa ($d = 0{,}271$).



Figura 8.11: Evaluación de compatibilidad con acierto (*True Positive*) en el modelo HGNN. Se observa una puntuación elevada ($s = 22{,}037$) asociada a la alta cardinalidad del conjunto (9 prendas).



Figura 8.12: Evaluación de compatibilidad con fallo de clasificación (*False Negative*) en el modelo HGNN. La puntuación reducida ($s = 2{,}981$) se ve influenciada por la baja cardinalidad del conjunto (3 prendas).

acumulación de conexiones en la estructura de categorías, lo que incrementa artificialmente la magnitud del escalar de compatibilidad. En *SCE-Net*, por el contrario, el modelo parece capturar de forma más sólida una noción holística de armonía de ocasión o estilo general entre los elementos, de forma independiente a las categorias de las prendas de cada conjunto, pese a que su mecanismo de 5 máscaras de condición (apartado 7.3.1) se concibió explícitamente para aprender subespacios latentes no supervisados por categoría.

8.2.4. Análisis del Espacio de Representación

Con el fin de caracterizar la estructura semántica y la topología del espacio de representación aprendido por cada arquitectura, se extrajeron los *embeddings* finales correspondientes a una muestra común y balanceada de 300 ítems pertenecientes a 45 categorías semánticas distintas. Dichas representaciones fueron proyectadas en dos dimensiones mediante la técnica de reducción de dimensionalidad *t-SNE* (figura 8.13), evaluando cuantitativamente el grado de agrupamiento categórico mediante el cálculo del *Silhouette Score* respecto a la categoría semántica de cada prenda.

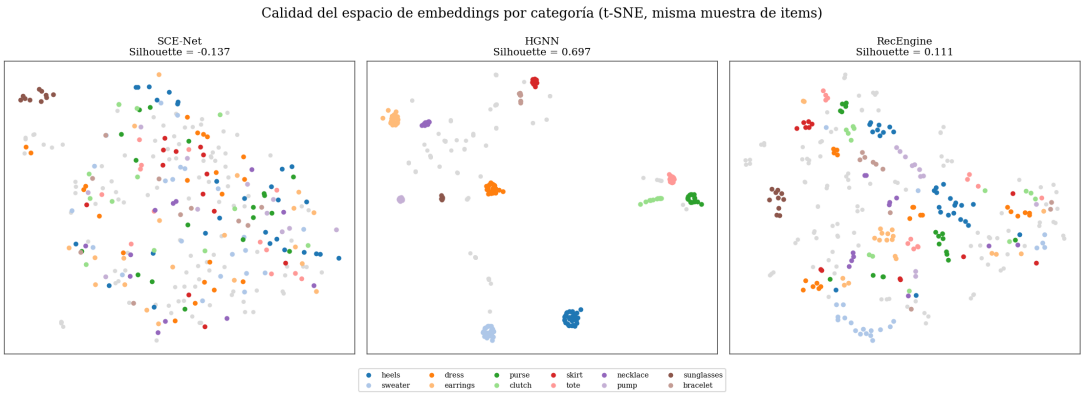


Figura 8.13: Proyección t-SNE bidimensional de los espacios latentes (300 ítems) aprendidos por HGNN ($0{,}6967$), *Recommendation Engine* ($0{,}1111$) y SCE-Net ($-0{,}1373$). Cada color representa una categoría semántica principal: azul oscuro (*heels*), azul claro (*sweater*), naranja (*dress*), melocotón (*earrings*), verde oscuro (*purse*), verde claro (*clutch*), rojo (*skirt*), rosa (*tote*), púrpura (*necklace*), lila (*pump*), marrón oscuro (*sunglasses*) y marrón claro (*bracelet*); los puntos grises corresponden a categorías secundarias.

Las tres arquitecturas exhiben topologías marcadamente dispares en sus espacios latentes, cuyos índices de silueta reflejan fielmente las hipótesis de diseño subyacentes a cada paradigma:

- **HGNN (Silhouette Score $= 0{,}6967$):** Obtiene el valor de separación más elevado con notable diferencia respecto al resto. Este comportamiento resulta plenamente coherente con su diseño estructural, dado que la arquitectura incorpora el *embedding* de categoría como un nodo explícito e indispensable en la topología del hipergrafo (apartado 7.3.2). En consecuencia, la concentración y separación por clústeres categóricos

emerge casi por construcción en el espacio proyectado.

- **Recommendation Engine (Silhouette Score $= 0{,}1111$):** Registra una cohesión categórica positiva pero moderada. Este resultado concuerda con la función de su módulo `TypeAwareGate` (apartado 7.3.3), el cual modula suavemente las representaciones mediante proyecciones conscientes del tipo de prenda, sin llegar a imponer la categoría semántica como el único eje organizador o restrictivo del espacio latente.

- **SCE-Net (Silhouette Score $= -0{,}1373$):** Arroja un coeficiente negativo, lo que evidencia una ausencia de agrupamiento por categoría semántica en la representación visual base sin enmascarar. Lejos de interpretarse como un defecto del modelo, esta dispersión constituye una confirmación empírica de su formulación teórica: a diferencia de HGNN y *Recommendation Engine*, SCE-Net no recibe la categoría semántica como entrada explícita en ninguna de sus capas (apartado 7.3.1), por lo que la ausencia de estructura por categoría en su *embedding* base es, precisamente, el comportamiento esperable de una arquitectura que aprende la noción de condición de forma completamente latente.

8.2.5. Interpretabilidad de mecanismos internos

Para profundizar en la transparencia del comportamiento de las arquitecturas, se diseñaron dos experimentos específicos orientados a auditar la estructura latente de los componentes internos en *HGNN* y *SCE-Net*.

Estructura de categorías aprendida en HGNN

Dado que *HGNN* incorpora un *embedding* de categoría como componente estructural en la topología del hipergrafo, se examinó la organización semántica interna de dicho espacio compuesto por 72 categorías. El análisis revela dos hallazgos contrapuestos de notable relevancia:

- **Desarticulación en macro-categorías:** Mientras que la discriminación entre categorías individuales de nivel fino es sumamente elevada (Silhouette Score $= 0{,}6967$, según el apartado 8.2.4), la organización de estas entidades en agrupaciones semánticas de nivel superior (tales como *tops*, *bottoms* o *footwear*) resulta deficiente, registrando un índice de silueta negativo ($-0{,}1027$). El modelo delimita con precisión cada categoría como una entidad discreta, pero fracasa al construir una jerarquía semántica global coherente.

- **Anomalías en la vecindad semántica:** La inspección de los vecinos más próximos mediante similitud coseno refuerza la falta de un espacio latente estructurado. Por ejemplo, los vecinos más cercanos a la categoría *backpack* incluyen *sunglasses* ($\cos\theta = 0{,}670$) y *tote*, mientras que para *belt* destacan *male baseball caps* ($\cos\theta = 0{,}605$) y

male jeans. Estas proximidades no responden a una jerarquía estilística ni a reglas explícitas de coocurrencia.

En conjunto, estos resultados indican que el espacio de categorías de HGNN captura una noción de identidad discreta por categoría, útil para decisiones locales en la tarea de FITB (apartado 8.2.1), pero carece de una representación semántica generalizable capaz de abstraer relaciones de alto nivel entre categorías distintas.

Interpretación semántica de las máscaras de condición en SCE-Net

El mecanismo de *SCE-Net* se concibió teóricamente para ponderar dinámicamente cinco máscaras de condición específicas ($w_1, \ldots, w_5$) en función del par de ítems evaluado (apartado 7.3.1). Para auditar este comportamiento, se examinaron los pesos asignados por la rama de ponderación sobre una muestra de $1{,}525$ pares de ítems distribuidos en ocho combinaciones de categorías distintas (e.g., *jewellery*↔*shoes*, *bags*↔*tops*). Como se ilustra en la figura 8.14, el análisis evidencia una distribución de pesos idéntica e invariable entre todos los grupos ($[0{,}248;\ 0{,}172;\ 0{,}216;\ 0{,}130;\ 0{,}235]$), la cual coincide con la media global sobre la totalidad del conjunto de prueba.

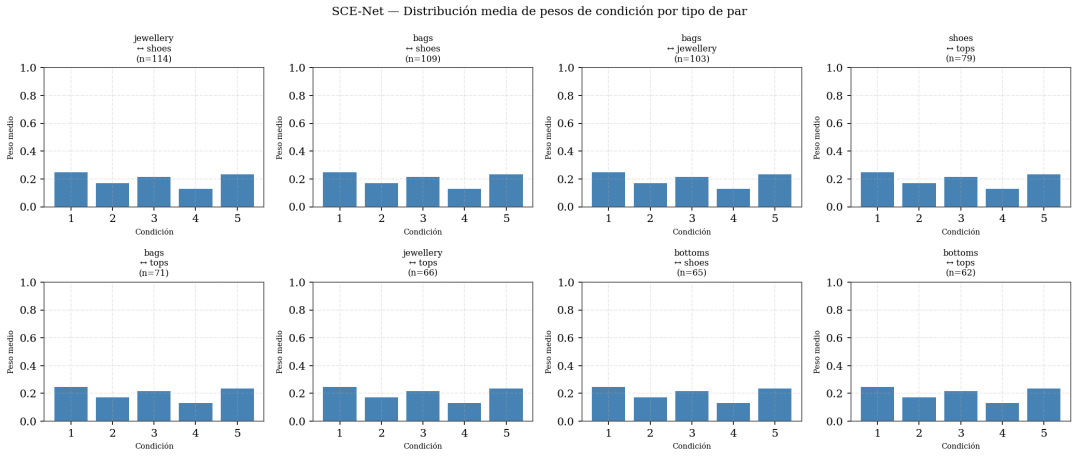


Figura 8.14: Distribución media de los pesos asignados a las cinco máscaras de condición en SCE-Net para ocho combinaciones de categorías de prendas: *jewellery*↔*shoes* ($n = 114$), *bags*↔*shoes* ($n = 109$), *bags*↔*jewellery* ($n = 103$), *shoes*↔*tops* ($n = 79$), *bags*↔*tops* ($n = 71$), *jewellery*↔*tops* ($n = 66$), *bottoms*↔*shoes* ($n = 65$) y *bottoms*↔*tops* ($n = 62$). Se observa un perfil de pesos idéntico ($[0{,}248;\ 0{,}172;\ 0{,}216;\ 0{,}130;\ 0{,}235]$) e independiente de las categorías de entrada.

Un diagnóstico exhaustivo descarta que esta insensibilidad derive de una degeneración trivial en la ingesta o en los parámetros:

1. Tanto las representaciones textuales (*Fisher Vectors*) como las visuales de los ítems muestran varianza real entre muestras.

2. La capa final del Perceptrón Multicapa (MLP) de ponderación no está dominada por el término de sesgo, registrándose una norma matricial de pesos que supera en casi ocho veces la norma del vector de sesgo.

Por consiguiente, se concluye que el mecanismo de ponderación de condición de *SCE-Net* ha colapsado experimentalmente hacia una función constante e insensible al par de entrada, en lugar de efectuar una discriminación adaptativa de subespacios latentes. Este hallazgo reinterpreta los aciertos cualitativos de SCE-Net observados en escenarios de alta coherencia estilística (apartado 8.2.2): dichos aciertos no deben atribuirse a una atención dinámica genuina de las máscaras de condición, sino más bien a la capacidad descriptiva del *embedding* visual base combinada con un esquema de ponderación estático pero funcionalmente efectivo.

8.3. Evaluación del Módulo Text-to-Outfit (Multimodal)

8.3.1. Resultados de Recuperación Semántica (*Text-to-Outfit*)

La tabla 8.2 sintetiza los resultados alcanzados por las tres variantes de la tarea de *Text-to-Outfit* desarrolladas en este trabajo sobre el conjunto de test, bajo el protocolo de evaluación multi-positivo (apartado 7.4.2). Asimismo, se incluyen como referencia externa las cifras publicadas por Jang et al. (2023) para la arquitectura *CLIP4Outfit* original.

Esta comparación asume la equivalencia funcional del protocolo de evaluación, dado que dicha publicación no especifica de forma explícita el tratamiento otorgado a la duplicidad textual entre *outfits*; los autores no reportan, en cualquier caso, las métricas Recall@1 ni MRR.

Tabla 8.2: Resultados de recuperación semántica sobre el conjunto de test (criterio multi-positivo).

Modelo	**R@1 (%)**	**R@5 (%)**	**R@10 (%)**	**MRR (%)**
RE (*zero-shot*)	2,3	8,1	11,7	5,9
RE (*fine-tuned*)	3,2	10,4	14,9	7,4
CLIP4Outfit	3,2	5,8	8,2	5,0
CLIP4Outfit (literatura)	–	7,59	11,12	–

Efecto del ajuste fino en Recommendation Engine.

Al comparar las dos adaptaciones de *Recommendation Engine*, el ajuste fino de la última capa del Transformer (apartado 7.4.1) supera de forma consistente a la adaptación *zero-shot* en las cuatro métricas evaluadas. Esta mejora evidencia la utilidad de aplicar un ajuste fino

sobre la última capa del Transformer, demostrando que el coste computacional adicional se traduce en una ganancia cuantitativa real de precisión y no en una mera adaptación marginal del `QueryEncoder` sobre un espacio latente estático. En particular, la métrica MRR se incrementa del $5{,}9\,\%$ al $7{,}4\,\%$ (una ganancia relativa aproximada del $25\,\%$), guardando proporcionalidad con los incrementos registrados en el Recall agregado.

Comportamiento anómalo y sobreajuste en CLIP4Outfit.

Por su parte, *CLIP4Outfit* alcanza el Recall@1 más elevado de las tres variantes ($3{,}2\,\%$, empatando con RE *fine-tuned*), pero resulta netamente inferior en el resto de los indicadores: sus cotas de Recall@5 ($5{,}8\,\%$) y Recall@10 ($8{,}2\,\%$) quedan por debajo de ambas adaptaciones de RE, arrojando asimismo el MRR más reducido del estudio ($5{,}0\,\%$).

Esta divergencia: un Recall@1 competitivo en paralelo a la penalización severa del MRR, resulta contraintuitiva a primera vista, dado el alto peso que el MRR otorga a los aciertos en las primeras posiciones. Sin embargo, el fenómeno resulta plenamente coherente con la dinámica de sobreajuste diagnosticada en el apartado 7.5.3: el modelo conserva la capacidad de recuperar de forma exacta la primera posición para un subconjunto específico de consultas, pero su generalización se degrada drásticamente en la consulta promedio, penalizando el cómputo global del MRR.

Transferencia de aprendizaje y comparación con el estado del arte.

Resulta de especial interés que las dos adaptaciones de *Recommendation Engine*, una arquitectura sustancialmente más liviana que *CLIP4Outfit* y desprovista de módulos dedicados de alineación multimodal por niveles, superen la cifra oficial reportada por Jang et al. (2023) en Recall@5 y Recall@10. Se formulan dos factores explicativos contiguos:

1. **Transferencia desde tareas de compatibilidad:** *Recommendation Engine* reutiliza un espacio de representación preentrenado para *Compatibility Prediction* y *Fill-in-the-Blank*. Estas tareas exigen abstraer patrones complejos de compatibilidad indumentaria, una línea con un recorrido investigador más amplio y consolidado (capítulo 5) que la propia tarea de *Text-to-Outfit*. Esta base de conocimiento previa se transfiere eficazmente hacia la tarea de recuperación por texto.

2. **Origen de los baselines de la literatura:** Los modelos frente a los que Jang et al. (2023) contrastan originalmente su propuesta (*SUPPORT-SET*, *CLIP4Clip*, *TS2-Net*) provienen en su totalidad del ámbito de la recuperación texto-vídeo, sin incorporar competidores especializados en el dominio de la moda. En consecuencia, la cifra publicada no representa necesariamente un techo de rendimiento infranqueable en este ámbito.

Por último, la réplica propia de *CLIP4Outfit* desarrollada en este trabajo no alcanza las cifras del artículo original a pesar de compartir una configuración de hiperparámetros idéntica. Esta disparidad se explica por el sobreajuste identificado, sumado a la imposibilidad de ejecutar una búsqueda sistemática de hiperparámetros o repeticiones estocásticas con semillas variadas (apartado 7.5.4), factores que impiden descartar que una ejecución alternativa hubiera alcanzado una convergencia más próxima a la reportada originalmente.

8.3.2. Inspección Cualitativa de Consultas con Referencia Conocida

Esta sección examina la aceptabilidad cualitativa del resultado *top-1* recuperado por cada modelo para cuatro consultas representativas, evaluando su comportamiento frente al conjunto de referencia (*Ground Truth*, GT) disponible en el catálogo de test. Este análisis atiende a si el conjunto recuperado resulta coherente con el registro de formalidad e intencionalidad implícitos, así como a su capacidad para capturar los atributos explícitos (color, textura o tipo de prenda) especificados en el texto.

Esta aproximación se plantea como una alternativa de menor coste a un estudio formal de preferencia con usuarios reales, como el empleado por Jang et al. (2023) mediante paneles de anotadores en Amazon Mechanical Turk, opción descartada en este trabajo por las limitaciones de alcance ya justificadas en el apartado 6.1 respecto a la pregunta de investigación RQ3.

El caso más revelador se presenta en la consulta «*Velvet Bomber Jackets*»(figura 8.15). CLIP4Outfit recupera directamente una cazadora *bomber* estampada en tonos burdeos, satisfaciendo tanto la categoría de la prenda como la textura visual evocada por el término *“velvet”*. Por el contrario, ninguna de las dos adaptaciones de *Recommendation Engine* incluye una prenda de abrigo en su recomendación *top-1*: la versión *zero-shot* propone un chaleco de borreguito, mientras que la adaptación *fine-tuned* devuelve un conjunto casual sin una cazadora reconocible.

Un patrón similar se observa en la búsqueda «*Sweater Dress and Boots*»(figura 8.16). En este caso, tanto RE *fine-tuned* como CLIP4Outfit identifican con éxito un vestido de punto combinado con botas altas, ofreciendo una interpretación casi literal de la consulta. En contraste, la versión *zero-shot* de RE diverge sustancialmente, devolviendo un bolso y unas sandalias de tacón sin incluir prendas de punto ni calzado alto.

La consulta «*Paris Fashion Week*»(figura 8.17) ilustra cómo una recomendación puede resultar altamente verosímil aun cuando difiera de la referencia conocida (*Ground Truth*). Mientras que el modelo *zero-shot* proyecta una selección inconexa sin un estilo claro, tanto la adaptación *fine-tuned* como CLIP4Outfit proponen conjuntos de estética editorial cuidada. El primero sugiere un vestido asimétrico con accesorios formales, mientras que el segundo opta

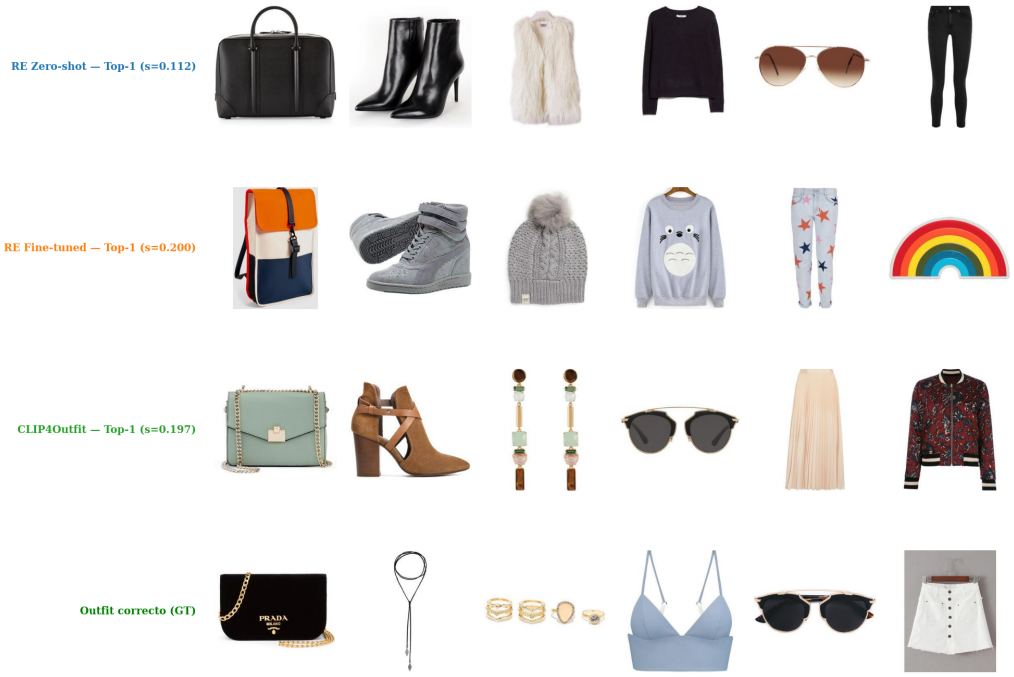


Figura 8.15: Resultados cualitativos de recomendación (*Top-1*) frente a la consulta textual *«Velvet Bomber Jackets»*. Comparativa entre las adaptaciones RE *Zero-Shot*, RE *Fine-Tuned*, CLIP4Outfit y el conjunto de referencia (*Ground Truth*).

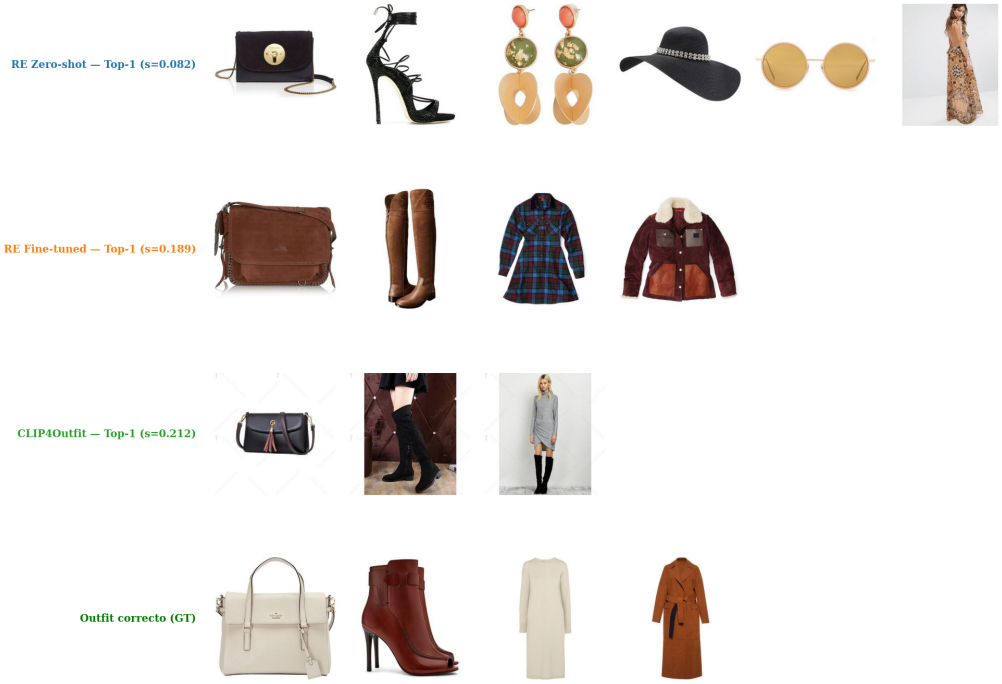


Figura 8.16: Resultados cualitativos de recomendación (*Top-1*) frente a la consulta textual *«Sweater dress and boots»*. Comparativa entre las adaptaciones RE *Zero-Shot*, RE *Fine-Tuned*, CLIP4Outfit y el conjunto de referencia (*Ground Truth*).

por un abrigo de estampado animal con gafas de sol ornamentadas. Aunque ninguno coincide con el *outfit* de referencia, ambas propuestas encajan de manera natural bajo la etiqueta conceptual en un catálogo real.



Figura 8.17: Resultados cualitativos de recomendación (*Top-1*) frente a la consulta textual *«Paris fashion week»*. Comparativa entre las adaptaciones RE *Zero-Shot*, RE *Fine-Tuned*, CLIP4Outfit y el conjunto de referencia (*Ground Truth*).

En la consulta «*Sweater Dresses*»(figura 8.18), CLIP4Outfit vuelve a aislar la prenda principal con precisión mediante un conjunto invernal totalmente coherente. Por su parte, el modelo *fine-tuned* recupera un jersey de cuello alto en lugar de un vestido, cumpliendo la consulta solo de forma parcial. Finalmente, la adaptación *zero-shot* prescinde por completo de prendas de tejido de punto.

En conjunto, el análisis de estas cuatro consultas evidencia diferencias sustanciales en la dinámica de recuperado de cada arquitectura. CLIP4Outfit demuestra una mayor tendencia a capturar de manera literal los términos léxicos explícitos (como el tipo de prenda o atributos específicos de textura), un rasgo patente en «*Velvet Bomber Jackets*»y «*Sweater Dresses*». Por el contrario, las adaptaciones de *Recommendation Engine*, y en especial la versión *fine-tuned*, priorizan la coherencia global del registro y la ocasión, aun cuando ello implique omitir menciones literales.

Esta observación cualitativa ofrece una explicación coherente para los resultados cuantitativos reportados en la tabla 8.2. CLIP4Outfit muestra una mayor varianza entre consultas:



Figura 8.18: Resultados cualitativos de recomendación (*Top-1*) frente a la consulta textual *«Sweater dresses»*. Comparativa entre las adaptaciones RE *Zero-Shot*, RE *Fine-Tuned*, CLIP4Outfit y el conjunto de referencia (*Ground Truth*).

es capaz de lograr aciertos sumamente precisos en búsquedas específicas a costa de una menor consistencia global, lo que explica su métrica de *Recall@K* agregado inferior en contraste con un rendimiento competitivo en *Recall@1*.

8.3.3. Evaluación Cualitativa de Consultas Compuestas sin Referencia

Esta sección examina cuatro consultas compuestas, con restricciones de negación, estilo o cantidad, diseñadas deliberadamente sin outfit de referencia en el catálogo: descripciones que no corresponden a ningún *outfit* concreto del dataset, sino a un perfil de búsqueda realista de un usuario final. Al no existir referencia conocida (*Ground Truth*), el criterio de evaluación es exclusivamente el de plausibilidad semántica: se analiza en qué medida la recomendación *top-1* satisface simultáneamente las diversas condiciones especificadas en la consulta.

La consulta «Casual outfit for a rainy day but not too warm»(figura 8.19) ilustra bien cómo cada modelo prioriza un subconjunto distinto de las restricciones de la consulta: la adaptación *zero-shot* recupera una cazadora vaquera y sandalias de tiras, satisfaciendo la componente “casual” pero ignorando por completo tanto la lluvia como la temperatura; el modelo *fine-tuned* sí introduce una prenda apropiada para lluvia sin resultar excesivamente abrigada, pero el conjunto resultante se aleja del registro casual solicitado. CLIP4Outfit es el único de los tres que satisface razonablemente bien las tres restricciones a la vez.



Figura 8.19: Resultados cualitativos de recomendación (*Top-1*) frente a la consulta compleja sin referencia *«Casual outfit for a rainy day but not too warm»*. Comparativa entre las adaptaciones de *Recommendation Engine Zero-Shot* y RE *Fine-Tuned* y CLIP4Outfit.

La búsqueda «Elegant evening look without black or heels»(figura 8.20) evidencia una limitación recurrente en espacios de *embeddings* contrastivos: el procesamiento de operadores lógicos de negación. RE *zero-shot* cumple solo parcialmente, propio estampado de rayas reintroduce el negro que la consulta pide excluir. RE *fine-tuned* incumple la restricción de forma más directa, con un vestido negro y bailarinas de tacón: ambos elementos vetados aparecen en el resultado. CLIP4Outfit evita el negro, pero recupera un tacón de salón, incumpliendo la restricción de calzado plano. Ninguna de las tres arquitecturas logra resolver ambas negaciones de forma simultánea, resultado consistente con las deficiencias de razonamiento composicional documentadas en la literatura para modelos basados en la arquitectura CLIP.

Por último, la consulta «Streetwear inspired by 90s hip-hop with oversized silhouette»(figura 8.21) genera la mayor divergencia en la versión RE *zero-shot*, la cual propone un conjunto ajeno a la estética urbana solicitada. En cambio, tanto RE *fine-tuned* como CLIP4Outfit capturan adecuadamente el registro *streetwear*; no obstante, ninguna de las dos opciones ofrece una silueta claramente holgada (*"oversized"*) acorde al término en la moda actual.

Cabe plantear, como hipótesis explicativa, que esta última imprecisión derive del sesgo temporal intrínseco al *benchmark*. Los contenidos de *Polyvore* reflejan las tendencias estéticas dominantes durante el periodo de actividad de la plataforma, anterior a la consolidación masiva de las siluetas *oversized* actuales. En consecuencia, las recomendaciones obtenidas pueden constituir representaciones razonables del *streetwear* noventero dentro de la distribución de datos disponible, más que un fallo estricto de recuperación semántica por parte de los modelos.

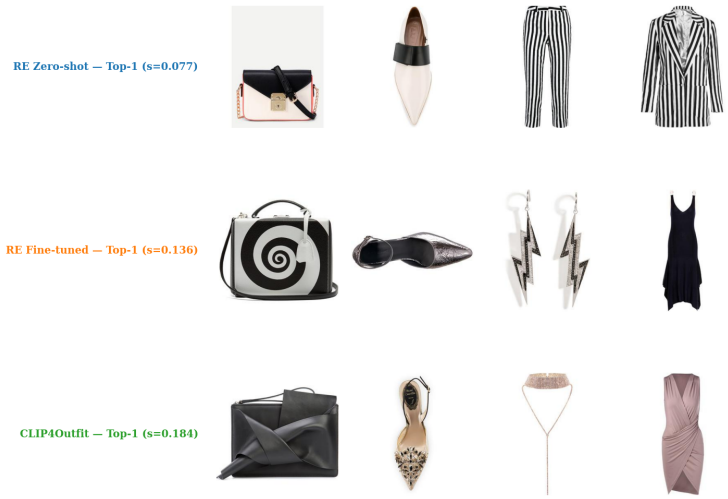


Figura 8.20: Resultados cualitativos de recomendación (*Top-1*) frente a la consulta compleja sin referencia *«Elegant evening look without black or heels a rainy day but not too warm»*. Comparativa entre las adaptaciones de *Recommendation Engine Zero-Shot* y RE *Fine-Tuned* y CLIP4Outfit.



Figura 8.21: Resultados cualitativos de recomendación (*Top-1*) frente a la consulta compleja sin referencia *«Streetwear inspired by 90s hip-hop with oversized silhouette»*. Comparativa entre las adaptaciones de *Recommendation Engine Zero-Shot* y RE *Fine-Tuned* y CLIP4Outfit.

El patrón cualitativo identificado en esta sección consolida y matiza las observaciones del apartado 8.3.2. Frente a consultas complejas con múltiples condicionantes, CLIP4Outfit demuestra una superioridad sistemática en la captura del registro estético y conceptual global (*streetwear*, elegancia o adecuación climática), propiedad atribuible a su espacio contrastivo ajustado específicamente sobre pares texto-*outfit* (apartado 7.4.2).

Sin embargo, la totalidad de las arquitecturas evaluadas muestra una marcada vulnerabilidad al resolver restricciones lógicas explícitas: las negaciones y los modificadores de forma se incumplen de manera sistemática, incluso cuando la atmósfera general de la consulta queda bien resuelta. Esta disociación entre la fidelidad al estilo global y la incapacidad para procesar operadores lógico-composicionales constituye una restricción estructural propia de la codificación semántica holística de los modelos contrastivos, cuyo alcance e implicaciones se discuten en el siguiente apartado.

8.4. Decisiones de Diseño y Diagnóstico de Componentes

Esta sección ofrece una síntesis analítica del impacto empírico y conceptual de las principales decisiones de diseño adoptadas durante el desarrollo de la memoria. Se distinguen formalmente aquellas decisiones respaldadas por estudios de ablación directa frente a aquellas justificadas sobre bases teóricas o de eficiencia computacional cuya verificación experimental independiente excede el alcance del presente trabajo.

8.4.1. Ablaciones Empíricas y Diagnóstico Interno de Arquitecturas

Estrategia de adaptación en *Recommendation Engine* (*Fine-Tuning* vs. Espacio Latente Congelado).

El ajuste fino de la última capa de la red Transformer, en comparación con el mantenimiento del espacio de representación congelado, reporta una mejora sostenida en la totalidad de las métricas de recuperación (*Recall@K* y *MRR*, Tabla 8.2), destacando una ganancia relativa en *MRR* cercana al $25\,\%$. Esta comparativa constituye el único estudio de ablación estricto del módulo de búsqueda textual, donde ambas adaptaciones fueron optimizadas y evaluadas bajo condiciones idénticas a excepción del grado de congelación de capas.

Mecanismo de propagación en *HGNN* (Ajuste del número de pasos T).

La reducción de los pasos de propagación de $T = 3$ a $T = 1$, motivada por el diagnóstico del fenómeno de sobre-suavizado (*over-smoothing*) detallado en la sección 7.3.2, no solo mitigó el colapso del entrenamiento, sino que converge con la configuración óptima identificada por Li et al. (2022). El perfil operacional de *HGNN* observado en la tabla 8.1, caracterizado por un elevado rendimiento en la tarea *FITB* frente a una métrica *AUC* más moderada, debe interpretarse en función de esta topología simplificada y no sobre la dinámica original de múltiples iteraciones.

Análisis de fallos en *SCE-Net*: Exclusión de la pérdida *VSE* y colapso funcional de ramas.

El término de supervisión *VSE* (*Visual-Semantic Embedding*) fue desestimado tras constatar una similitud cosenoidal residual entre modalidades de apenas $0{,}06$, atribuible a la reducida capacidad de discriminación semántica que presentan los vectores de Fisher ($0{,}031$ entre categorías contiguas, apartado 7.3.1). Adicionalmente, el diagnóstico interno revelado en la sección 8.2.5 demostró que la rama de ponderación de condiciones degenera hacia una salida prácticamente constante e invariable ante diferentes pares de entrada.

Estos hallazgos evidencian que *SCE-Net* opera predominantemente sustentado en el vector visual de base, teniendo sus mecanismos de atención condicional una contribución significativamente más acotada que la asumida en su formulación teórica original.

8.4.2. Justificación Conceptual y Limitaciones Transversales

Decisiones de diseño asumidas sin estudio de ablación exprofeso.

Dentro de la arquitectura *Recommendation Engine*, el diseño de la puerta condicional (`TypeAwareGate`), la asimetría funcional en la codificación de contextos frente a candidatos en la tarea *FITB*, y la combinación no ponderada de las funciones de pérdida (apartado 7.3.3) responden a criterios de diseño justificables: la analogía con embeddings sensibles al tipo Vasileva et al. (2018), la optimización de la latencia computacional y la paralelización de objetivos inspirada en la literatura de preentrenamiento lingüístico, respectivamente. No obstante, al no haberse entrenado variantes de control sin dichos módulos, su aportación cuantitativa individual al rendimiento global permanece como una hipótesis teórica no aislable empíricamente.

Limitaciones en el razonamiento composicional para *Text-to-Outfit*.

Tal como se argumentó en el análisis de casos de estudio (apartado 8.3.3), los tres modelos de recuperación semántica muestran un fallo sistemático al procesar operadores lógicos explícitos (negaciones y restricciones de cardinalidad), aun cuando capturan con precisión la estética o el contexto global de la consulta.

Esta deficiencia es inherente a la naturaleza de las pérdidas contrastivas utilizadas, las cuales proyectan la consulta a un único espacio vectorial denso sin descomponer formalmente su estructura sintáctica. Se confirma así que la incapacidad composicional constituye una limitación estructural y transversal a los dos paradigmas de *Text-to-Outfit* evaluados, y no una deficiencia puntual de una implementación concreta.

8.5. Análisis de Eficiencia Computacional y Latencia de Inferencia

Esta sección evalúa la viabilidad operacional de las arquitecturas desarrolladas en términos de volumen de parámetros y tiempos de latencia durante la inferencia. Se excluye deliberadamente el tiempo de entrenamiento de los modelos de *Text-to-Outfit* de esta comparativa: dado que *Recommendation Engine* reutiliza un espacio de representación previamente optimizado para la tarea de compatibilidad, mientras que *CLIP4Outfit* requiere un entrenamiento integral desde el punto de partida visual-textual, una comparación directa de los tiempos de convergencia no resultaría metodológicamente equitativa.

8.5.1. Modelos de Compatibilidad y Diagnóstico de Componentes

La tabla 8.3 sintetiza la parametrización total y la latencia media de respuesta para las tres arquitecturas de compatibilidad que constituyen el núcleo experimental de esta memoria.

Tabla 8.3: Volumen de parámetros y latencia de inferencia en modelos de compatibilidad.

Modelo	**Parámetros**	**Inferencia FITB**	**Inferencia Compatibilidad**
SCE-Net	12,3 M	165 ms	82,0 ms
HGNN	95 k	16 ms	3,5 ms
Recommendation Engine	3,3 M	57 ms	41,0 ms

HGNN emerge como la arquitectura de mayor eficiencia computacional y ligereza dimensional, consecuencia directa de operar sobre vectores de características visuales y textuales precomputados de manera *offline* (apartado 7.3.2). Esta cifra de latencia representa exclusivamente el tiempo de procesamiento en el hipergrafo y no es directamente extensible a escenarios de despliegue en tiempo real ante ítems fuera de catálogo: la incorporación de una nueva prenda requeriría la extracción previa de características mediante la red *InceptionV3*, un coste adicional omitido en esta medición.

Por su parte, *SCE-Net* constituye el modelo con mayor consumo de recursos, reflejo de la optimización completa (*fine-tuning*) de su *backbone* visual (apartado 7.3.1).

8.5.2. Módulo Text-to-Outfit: Latencia y Complejidad Espacial

La tabla 8.4 detalla la distribución de parámetros entre módulos específicos y arquitecturas base, así como el tiempo de respuesta por consulta textual.

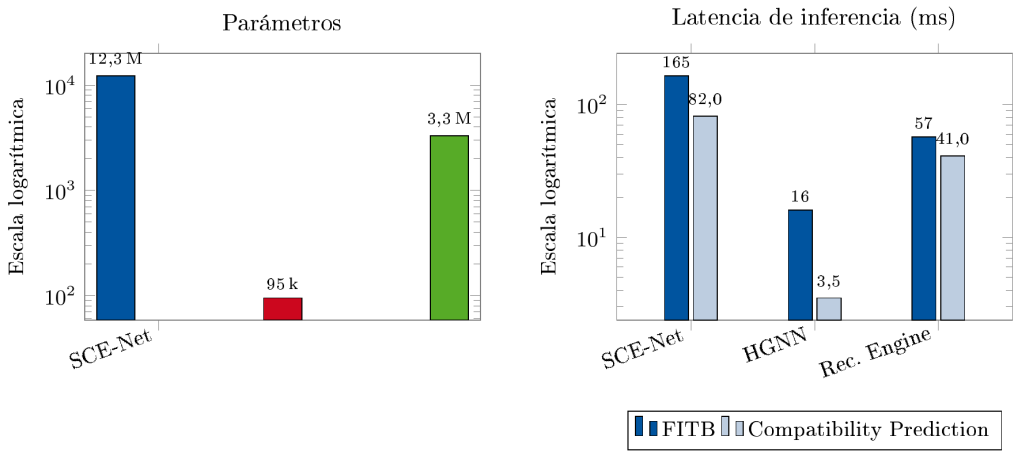


Figura 8.22: Volumen de parámetros y latencia media de inferencia en las tareas Fill-In-The-Blank y *Compatibility Prediction*, para las tres arquitecturas de compatibilidad evaluadas. Datos de la tabla 8.3.

Tabla 8.4: Desglose de parámetros y tiempo de inferencia para las variantes de *Text-to-Outfit*.

Modelo	**Parámetros específicos**	**Parámetros totales**	**Inferencia / consulta**
RE (*zero-shot*)	58 752	3,4 M*	199 ± 20 ms
RE (*fine-tuned*)	191 232	3,5 M*	260 ± 11 ms
CLIP4Outfit	163,9 M	163,9 M	$32\,451 \pm 12\,627$ ms

*Incluye los 3,3 M de parámetros del *backbone* de compatibilidad previamente entrenado (Tabla 8.3).

El análisis de la carga paramétrica exige contextualizar la ligereza de *Recommendation Engine*: aunque el volumen de parámetros entrenados exprofeso para la búsqueda textual es reducido (entre $58\,752$ y $191\,232$), la ejecución del sistema completo requiere computar los 3,3 M de parámetros de su *backbone* de compatibilidad. No obstante, la masa paramétrica total de esta aproximación se mantiene significativamente por debajo de los 163,9 M de parámetros de *CLIP4Outfit*, distribuidos principalmente entre el codificador base *CLIP* (151,3 M) y los módulos de agregación jerárquica (12,6 M).

Esta disparidad estructural se traslada de forma crítica a la latencia de inferencia en producción: las variantes adaptadas de *Recommendation Engine* resuelven la consulta en un intervalo de 200 a 260 ms, mientras que *CLIP4Outfit* promedia latencias superiores a los 32 segundos por consulta, un incremento de más de dos órdenes de magnitud, acompañado de una elevada variabilidad estocástica ($\pm 12,6$ s).

Este comportamiento es consistente con la naturaleza algorítmica de *CLIP4Outfit*, la cual requiere computar dinámicamente en tiempo de inferencia las interacciones de alineación

fine-grained (*Weight-Token Interaction*, WTI) y el agrupamiento mediante K-means para cada par consulta-*outfit*, en contraposición al cálculo algebraico directo de la similitud coseno sobre el espacio vectorial preproyectado que ejecuta *Recommendation Engine*.

Conclusiones y Trabajo Futuro

9

9.1. Consecución de Objetivos

A continuación, se presenta una evaluación crítica e individualizada del grado de cumplimiento del objetivo general y de los objetivos específicos formulados al inicio de esta investigación:

Objetivo General: *Desarrollar un sistema capaz de generar y combinar representaciones vectoriales (embeddings) de imágenes y descripciones textuales de prendas, aplicando técnicas del estado del arte al problema de la compatibilidad en moda.*

- **Grado de cumplimiento:** Alcanzado en la componente arquitectónica.
- **Evaluación analítica:** La dimensión arquitectónica y teórica del objetivo principal se ha satisfecho plenamente. Las tres arquitecturas desarrolladas (detalladas en el capítulo 5.1) integran de forma efectiva representaciones multimodales (visuales y textuales), fundamentadas en esquemas de fusión cuya justificación ha sido documentada con rigor. No obstante, la validación de su utilidad práctica sobre catálogos comerciales externos no ha podido verificarse empíricamente debido a las restricciones legales de acceso a datos analizadas en el Objetivo Específico 4 y en la formulación de la pregunta de investigación RQ_2 (sección 6.1), quedando esta componente supeditada a trabajos futuros.

Objetivo Específico 1: *Análisis comparativo de arquitecturas de vanguardia en recomendación de moda.*

- **Estado:** Cumplido.
- **Evaluación analítica:** Se han implementado, optimizado y evaluado comparativamente tres paradigmas arquitectónicos representativos del estado del arte: subespacios de condicionado métrico (*SCE-Net*), redes de hipergrafos heterogéneos (*HGNN*) y mecanismos de atención basados en *Transformer* (*Recommendation Engine*). Los resultados cuantitativos han sido sistematizados en la sección 8.2.1, mientras que el análisis de sus dinámicas de convergencia se detalla en la sección 7.5.3.

Objetivo Específico 2: *Construcción de un espacio de representación multimodal alineado.*

- **Estado:** Cumplido.
- **Evaluación analítica:** Cada arquitectura articula representaciones visuales y textuales mediante un esquema de fusión propio, documentado formalmente en el apartado 7.3. En el desarrollo de dichos esquemas se resolvieron vacíos de especificación concretos identificados en las publicaciones originales de referencia, cuyas soluciones quedan registradas en las secciones 7.3.1 a 7.3.3.

Objetivo Específico 3: *Validación experimental sobre el benchmark estándar Polyvore Outfits.*

- **Estado:** Cumplido.
- **Evaluación analítica:** Las tres arquitecturas han sido evaluadas rigurosamente en las tareas benchmark de *Fill-In-The-Blank* (FITB) y *Compatibility Prediction* (capítulo 8.2), reproduciendo con fidelidad las condiciones experimentales y métricas reportadas en la literatura sobre la partición *nondisjoint*.

Objetivo Específico 4: *Evaluación de la capacidad de generalización mediante web scraping sobre catálogos comerciales reales.*

- **Estado:** No alcanzado (Inviabilidad legal exógena).
- **Evaluación analítica:** Aunque el flujo técnico y la metodología de recolección fueron correctamente diseñados, el objetivo de evaluar los modelos con este *corpus* no pudo llevarse a cabo. Tal como se justifica en el apartado 7.2.2, la imposibilidad de ejecutar la extracción no derivó de limitaciones técnicas o de infraestructura, sino de la ausencia sistemática de autorización legal para la recolección y redistribución de datos en las plataformas evaluadas (Términos de Servicio y directivas `robots.txt`). Esta restricción se asume de manera honesta como una limitación estructural exógena, validando el planteamiento metodológico pero emplazando su ejecución práctica a la obtención de vías oficiales (APIs o acuerdos) en la agenda de trabajo futuro.

Objetivo Específico 5: *Desarrollar un modelo de Text-to-Outfit a partir de embeddings ya entrenados en tareas de compatibilidad.*

- **Estado:** Cumplido (mediante dos aproximaciones complementarias).
- **Evaluación analítica:** El módulo de recuperación semántica se abordó mediante dos vías metodológicas diferenciadas: por un lado, una adaptación ligera del espacio latente preentrenado de *Recommendation Engine* (sección 7.4.1), evaluada en sus variantes *zero-shot* y *fine-tuned*; por otro, la implementación de la arquitectura contrastiva dedicada *CLIP4Outfit* (sección 7.4.2). Ambos enfoques fueron sometidos a evaluación cuantitativa y cualitativa en la sección 8.3.

9.2. Síntesis de Resultados y Principales Contribuciones

Más allá de los resultados cuantitativos del capítulo 8, la aportación fundamental de este trabajo reside en la reconstrucción analítica del estado del arte y en la evaluación homogénea de sus paradigmas. Las contribuciones se sintetizan en los siguientes puntos:

- **Reconstrucción unificada del estado del arte:** Los capítulos 4 y 5 estructuran una década de investigación combinando dos ejes habitualmente fragmentados: la evolución de las arquitecturas de compatibilidad y el avance paralelo en la representación multimodal visual-textual. Esta perspectiva justifica de forma teórica la integración del módulo *Text-to-Outfit*.
- **Benchmarking homogéneo entre paradigmas:** Se enfrentan por primera vez bajo un mismo protocolo experimental tres paradigmas conceptualmente dispares (*SCE-Net*, *HGNN* y *Recommendation Engine*). Al imponer idéntica partición de datos (*nondisjoint*), mismo hardware y criterios unificados de evaluación, se resuelven las incoherencias comparativas de la literatura previa.
- **Transferencia de conocimiento entre compatibilidad y búsqueda semántica:** Se demuestra que un espacio latente optimizado para la compatibilidad de moda puede adaptarse eficazmente a la búsqueda por texto mediante proyecciones ligeras. Esta adaptación supera las métricas globales de arquitecturas contrastivas dedicadas (*CLIP4Outfit*) con una fracción de sus parámetros, evidenciando la complementariedad entre la transferencia de espacios de compatibilidad y el anclaje léxico específico.

9.3. Limitaciones del Trabajo

A pesar de haber consolidado un protocolo experimental riguroso y transparente sobre la partición de referencia, la interpretación de los resultados de este Trabajo Fin de Máster debe matizarse bajo las siguientes limitaciones metodológicas y estructurales:

- **Restricción ético-legal para la validación *Out-of-Domain* (RQ_2):** La principal limitación del estudio radica en la imposibilidad de evaluar empíricamente la transferencia de los modelos hacia un entorno de producción o catálogo comercial real. Tal como se justifica en el apartado 7.2.2, esta restricción no obedece a deficiencias técnicas del sistema, sino a barreras ético-legales estrictas (Términos de Servicio y directivas `robots.txt` que prohíben la extracción y redistribución de material fotográfico).
- **Exclusión acotada de la evaluación perceptual humana (RQ_3):** A diferencia de la restricción anterior, el análisis de la correlación entre las métricas de optimización automática y el juicio estético humano se omitió por decisión de diseño (sección 6.1). Una validación de constructo rigurosa en este ámbito requeriría un panel cualificado de estilistas cuyo despliegue metodológico y logístico excedía el alcance operativo de este trabajo.

- **Naturaleza no profesional y ruido del *benchmark* de referencia:** El conjunto *Polyvore Outfits* integra combinaciones generadas por usuarios anónimos en una plataforma web abierta, en lugar de selecciones curadas por expertos en moda. La presencia inherente de ruido en las etiquetas de compatibilidad establece un techo de rendimiento infranqueable que limita el aprendizaje del modelo, independientemente de la sofisticación de la arquitectura.

- **Incompletitud y escasez en la cobertura del canal textual:** El $71{,}3\,\%$ de las prendas del *dataset* carece de descripciones textuales detalladas (sección 7.2.1). Esta restricción, de carácter transversal a las tres arquitecturas de compatibilidad, requirió articular estrategias de imputación mediante campos alternativos para mitigar la asimetría informacional entre modalidades.

- **Exploración hiperparamétrica supeditada al presupuesto computacional:** Las restricciones de infraestructura y tiempo de cálculo (sección 7.1.1) impidieron ejecutar búsquedas sistemáticas (*grid search* o exploración bayesiana). En consecuencia, las configuraciones se adoptaron de forma literal a partir de la literatura de referencia o mediante ajustes heurísticos dirigidos por diagnóstico.

9.4. Líneas de Trabajo Futuro

A partir de los hallazgos y las limitaciones identificadas en este trabajo, se proponen las siguientes líneas prioritarias de investigación y desarrollo:

- **Evaluación de transferencia a entornos de producción ($\mathbf{RQ}_2$):** Resolver la restricción de acceso mediante acuerdos de investigación o la curación manual de un subconjunto de catálogo comercial. Esto permitirá auditar la capacidad de generalización *zero-shot* de las arquitecturas frente a la distribución de datos real de catálogos e-commerce.

- **Validación perceptual y juicio estético humano ($\mathbf{RQ}_3$):** Abordar la dimensión cualitativa del problema articulando un estudio con evaluadores o estilistas profesionales. Se sugiere adoptar el protocolo de Prato et al. (2020) e incorporar coeficientes de concordancia inter-anotador (p. ej., la Kappa de Fleiss) para determinar la correlación entre las métricas *offline* y la percepción humana de compatibilidad.

- **Modernización del canal textual mediante modelos *Vision-Language*:** Sustituir la extracción de descriptores tradicionales por representaciones basadas en arquitecturas preentrenadas de lenguaje y visión (tales como modelos de la familia *CLIP* o *encoders* derivados de LLMs). Esta integración enriquecerá la densidad semántica del canal textual y optimizará la alineación multimodal tanto en los modelos de compatibilidad como en la tarea *Text-to-Outfit*.

- **Mitigación del sobre-suavizado (*oversmoothing*) en hipergrafos:** Diseñar e implementar mecanismos de regularización estructural (tales como *DropEdge* o conexiones residuales heterogéneas) en la arquitectura HGNN. El objetivo es prevenir la degradación de los *embeddings* al incrementar el número de capas de propagación en la red (sección 7.3.2).
- **Razonamiento composicional en Text-to-Outfit:** Superar la limitación de los alineamientos contrastivos estándar para resolver restricciones lógicas explícitas en las consultas (negación, atributos excluyentes o cardinalidad sección 8.4). Para ello, se propone explorar arquitecturas que incorporen descomposición estructural o grafos de escena textuales de forma previa a la codificación semántica.
- **Optimización hiperparamétrica y eficiencia computacional:** Desplegar esquemas de optimización bayesiana integrados con técnicas de cuantización y canalizaciones de I/O eficientes. Esta línea permitirá mitigar los cuellos de botella de infraestructura identificados y determinar el límite superior del rendimiento de cada paradigma.

9.5. Consideraciones Finales

El desarrollo de este Trabajo Fin de Máster pone de manifiesto que el verdadero desafío en la recomendación de moda y la evaluación de compatibilidad multimodal no reside únicamente en el diseño de arquitecturas complejas de aprendizaje profundo, sino en la rigurosidad de los protocolos experimentales, el control de la calidad de los datos y la interpretación transparente de las limitaciones.

A lo largo de esta investigación, se ha demostrado que la combinación de modalidades visuales y textuales ofrece un potencial significativo para capturar la compatibilidad estilística. Sin embargo, el estancamiento observado en el rendimiento de ciertas representaciones textuales clásicas y la presencia de inconsistencias en los conjuntos de datos de referencia subrayan la necesidad de auditar críticamente los cimientos metodológicos antes de juzgar la capacidad relativa de cualquier modelo.

En definitiva, este trabajo no solo proporciona una comparación homogénea y documentada entre paradigmas prominentes del estado del arte, sino que establece un marco analítico y trazable que servirá como base para futuras investigaciones orientadas al desarrollo de sistemas de recomendación de moda más robustos y reproducibles.

Bibliografía

Bettaney, E. M., Hardwick, S. R., Zisimopoulos, O., y Chamberlain, B. P. (2019). Fashion outfit generation for e-commerce.

Bromley, J., Guyon, I., LeCun, Y., Säckinger, E., y Shah, R. (1993). Signature verification using a "siamese"time delay neural network. In Cowan, J., Tesauro, G., y Alspector, J., editors, *Advances in Neural Information Processing Systems*, volume 6. Morgan-Kaufmann.

Chen, W., Huang, P., Xu, J., Guo, X., Guo, C., Sun, F., Li, C., Pfadler, A., Zhao, H., y Zhao, B. (2019). Pog: Personalized outfit generation for fashion recommendation at alibaba ifashion.

Chia, P., Attanasio, G., Bianchi, F., Terragni, S., Magalhães, A., Goncalves, D., Greco, C., y Tagliabue, J. (2022). Contrastive language and vision learning of general fashion concepts. *Scientific Reports*, 12.

Chopra, S., Hadsell, R., y LeCun, Y. (2005). Learning a similarity metric discriminatively, with application to face verification. *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)*, 1:539–546 vol. 1.

Cui, Z., Li, Z., Wu, S., Zhang, X.-Y., y Wang, L. (2019). Dressing as a whole: Outfit compatibility learning based on node-wise graph neural networks. In *The World Wide Web Conference*, WWW '19, page 307–317. ACM.

Feng, Y., You, H., Zhang, Z., Ji, R., y Gao, Y. (2019). Hypergraph neural networks.

Gulati, S. (2024). Fashion recommendation: Outfit compatibility using gnn.

Hadsell, R., Chopra, S., y LeCun, Y. (2006). Dimensionality reduction by learning an invariant mapping. *2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'06)*, 2:1735–1742.

Han, X., Wu, Z., Huang, P. X., Zhang, X., Zhu, M., Li, Y., Zhao, Y., y Davis, L. S. (2017a). Automatic spatially-aware fashion concept discovery.

Han, X., Wu, Z., Jiang, Y.-G., y Davis, L. S. (2017b). Learning fashion compatibility with bidirectional lstms. In *Proceedings of the 25th ACM international conference on Multimedia*, MM '17, page 1078–1086. ACM.

He, R., Packer, C., y McAuley, J. (2016). Learning compatibility across categories for heterogeneous item recommendation.

He, X., Liao, L., Zhang, H., Nie, L., Hu, X., y Chua, T.-S. (2017). Neural collaborative filtering.

Jang, J., Hwang, E., y Park, S.-H. (2023). Lost your style? navigating with semantic-level approach for text-to-outfit retrieval.

Kang, W.-C., Kim, E., Leskovec, J., Rosenberg, C., y McAuley, J. (2019). Complete the look: Scene-based complementary product recommendation.

Khosla, P., Teterwak, P., Wang, C., Sarna, A., Tian, Y., Isola, P., Maschinot, A., Liu, C., y Krishnan, D. (2021). Supervised contrastive learning.

Kiros, R., Salakhutdinov, R., y Zemel, R. S. (2014). Unifying visual-semantic embeddings with multimodal neural language models.

Klein, B., Lev, G., Sadeh, G., y Wolf, L. (2015). Fisher vectors derived from hybrid gaussian-laplacian mixture models for image annotation.

Koren, Y., Bell, R., y Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37.

Li, S. (2025). Paper explained 1: Vision transformer.

Li, Z., Li, J., Wang, T., Gong, X., Wei, Y., y Luo, P. (2022). Ocphn: Outfit compatibility prediction with hypergraph networks. *Mathematics*, 10(20).

Lin, Y., Moosaei, M., y Yang, H. (2019). Learning personal tastes in choosing fashion outfits. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*.

Lin, Y.-L., Tran, S., y Davis, L. S. (2020). Fashion outfit complementary item retrieval.

McAuley, J., Targett, C., Shi, Q., y van den Hengel, A. (2015). Image-based recommendations on styles and substitutes.

Perez, E., Strub, F., de Vries, H., Dumoulin, V., y Courville, A. (2017). Film: Visual reasoning with a general conditioning layer.

Plummer, B. A., Kiapour, M. H., Zheng, S., y Piramuthu, R. (2019). Give me a hint! navigating image databases using human-in-the-loop feedback. In *IEEE Winter Conference on Applications of Computer Vision, WACV 2019, Waikoloa Village, HI, USA, January 7-11, 2019*, pages 2048–2057. IEEE.

Prato, G., Sallemi, F., Cremonesi, P., Scriminaci, M., Gudmundsson, S. F., y Palumbo, S. (2020). Outfit completion and clothes recommendation. *Extended Abstracts of the 2020 CHI Conference on Human Factors in Computing Systems*.

Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., y Sutskever, I. (2021). Learning transferable visual models from natural language supervision.

Ramzan, F., Khan, M. U., Rehmat, A., Iqbal, S., Saba, T., Rehman, A., y Mehmood, Z. (2019). A deep learning approach for automated diagnosis and multiclass classification of alzheimer's disease stages using resting-state fmri and residual neural networks. *Journal of Medical Systems*, 44.

Rendle, S., Freudenthaler, C., Gantner, Z., y Schmidt-Thieme, L. (2012). Bpr: Bayesian personalized ranking from implicit feedback.

Resnick, P., Iacovou, N., Suchak, M., Bergstrom, P., y Riedl, J. (1994). Grouplens: An open architecture for collaborative filtering of netnews. In *Proceedings of the 1994 ACM Conference on Computer Supported Cooperative Work (CSCW '94)*, pages 175–186, Chapel Hill, NC, USA. ACM.

Schroff, F., Kalenichenko, D., y Philbin, J. (2015). Facenet: A unified embedding for face recognition and clustering. In *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, page 815–823. IEEE.

Song, X., Han, X., Li, Y., Chen, J., Xu, X.-S., y Nie, L. (2019). Gp-bpr: Personalized compatibility modeling for clothing matching. *Proceedings of the 27th ACM International Conference on Multimedia*.

Tan, R., Vasileva, M. I., Saenko, K., y Plummer, B. A. (2019). Learning similarity conditions without explicit supervision.

Vasileva, M. I., Plummer, B. A., Dusad, K., Rajpal, S., Kumar, R., y Forsyth, D. (2018). Learning type-aware embeddings for fashion compatibility.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., y Polosukhin, I. (2023). Attention is all you need.

Veit, A., Kovacs, B., Bell, S., McAuley, J., Bala, K., y Belongie, S. (2015). Learning visual clothing style with heterogeneous dyadic co-occurrences.

Vo, N., Jiang, L., Sun, C., Murphy, K., Li, L.-J., Fei-Fei, L., y Hays, J. (2018). Composing text and image for image retrieval - an empirical odyssey.

Wang, X., Wu, B., y Zhong, Y. (2019). Outfit compatibility prediction and diagnosis with multi-layered comparison network. In *Proceedings of the 27th ACM International Conference on Multimedia*, MM '19, page 329–337. ACM.

Wu, H., Gao, Y., Guo, X., Al-Halah, Z., Rennie, S., Grauman, K., y Feris, R. (2020). Fashion iq: A new dataset towards retrieving images by natural language feedback.

Xu, Y., Wang, W., Feng, F., Ma, Y., Zhang, J., y He, X. (2024). Diffusion models for generative outfit recommendation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, page 1350–1359. ACM.

Yang, X., Zhang, H., Jin, D., Liu, Y., Wu, C.-H., Tan, J., Xie, D., Wang, J., y Wang, X. (2022). Fashion captioning: Towards generating accurate descriptions with semantic rewards.

Yu, M., Ma, Y., Wu, L., Wang, C., Li, X., y Meng, L. (2025). Fashiondpo:fine-tune fashion outfit generation model using direct preference optimization.

Apéndize A: Declaración de uso de la IA

A

Este apéndice describe el uso de herramientas de inteligencia artificial generativa durante el desarrollo y la redacción de este Trabajo Fin de Máster, con el objetivo de garantizar la transparencia sobre su empleo.

El proyecto fue desarrollado principalmente por el autor, incluyendo las decisiones de diseño, la implementación de las arquitecturas, la experimentación, el análisis de resultados y las conclusiones finales. No obstante, se emplearon herramientas de IA generativa como apoyo en distintas fases del trabajo, con el fin de mejorar la productividad, asistir en la resolución de problemas técnicos y perfeccionar la documentación escrita. Las principales herramientas utilizadas fueron el modelo Sonnet 5 de Claude y Flash 3.6 de Gemini. Su uso se agrupa en dos áreas principales: apoyo al desarrollo técnico y apoyo a la redacción.

A.1. Apoyo al desarrollo técnico

Durante la fase de implementación, las herramientas de IA generativa se emplearon con un rol fundamentalmente de apoyo:

- Asistencia en la revisión bibliográfica y localización de literatura relevante sobre compatibilidad de moda y representación multimodal, verificando siempre las fuentes originales.
- Aclaración de conceptos relacionados con arquitecturas de aprendizaje profundo (redes siamesas, hipergrafos, mecanismos de atención) y con las métricas de evaluación empleadas (AUC, Recall@K, MRR).
- Ayuda en la depuración de errores durante la implementación de los pipelines de entrenamiento y preprocesamiento de datos (*SCE-Net*, *HGNN*, *Recommendation Engine* y *CLIP4Outfit*).
- Sugerencias sobre el uso de librerías y frameworks concretos del proyecto (*SciPy* para la construcción de estructuras dispersas del hipergrafo en *HGNN*, *Hugging Face Transformers* para el backbone *CLIP (openai/clip-vit-base-patch32)* en *CLIP4Outfit*, y el *Hugging Face Hub* para la descarga verificada del dataset *Polyvore Outfits*).

- Apoyo en la interpretación de mensajes de error y documentación técnica.
- Mejora de la legibilidad y refactorización puntual de ciertas secciones del código.

El código final, las decisiones de arquitectura, el proceso experimental y la evaluación de los modelos fueron revisados, adaptados y validados manualmente. Las herramientas no generaron de forma autónoma el sistema completo ni sustituyeron el proceso de desarrollo y toma de decisiones del autor.

A.2. Apoyo en la redacción y documentación

Las herramientas de IA generativa también se emplearon durante la redacción de la memoria. Su uso incluyó:

- Corrección gramatical, ortográfica y de estilo del texto en español.
- Reformulación de determinados párrafos para mejorar la claridad y fluidez de la exposición.
- Apoyo en la consistencia terminológica a lo largo del documento.
- Ayuda en el formateo y estructuración del contenido en LaTeX (tablas, figuras y organización de secciones).

El contenido técnico, las explicaciones, las interpretaciones y las conclusiones presentadas en esta memoria fueron redactadas y revisadas por el autor. Todo el texto generado con apoyo de IA fue comprobado y editado manualmente para garantizar su corrección, coherencia y fidelidad con el trabajo realmente desarrollado.

A.3. Uso responsable y verificación

El uso de herramientas de IA generativa se limitó a tareas de apoyo y estuvo siempre acompañado de supervisión humana. La información, el código sugerido y las explicaciones generadas fueron revisados críticamente antes de su incorporación al proyecto. Se prestó especial atención a:

- Verificar la corrección del código generado o sugerido.
- Comprobar la validez de las explicaciones técnicas y de las referencias bibliográficas.
- Asegurar que el contenido escrito reflejara con precisión el trabajo implementado y los resultados obtenidos.
- Evitar la inclusión de información no verificada o inventada.

Las herramientas de IA generativa se trataron como asistentes que facilitaron determinadas etapas del proceso, si bien la responsabilidad sobre la implementación final, la experimentación, el análisis y el contenido redactado recae íntegramente en el autor.